



Coral

1.7.0

Livro Oficial da Linguagem

Sintaxe, bibliotecas, ferramentas, instalação, arquitetura e referência consolidada

EDIÇÃO 1.7.0

Edição consolidada - setembro de 2026

Português corrente controlado. Contratos verificáveis. Ferramentas integradas.

Como usar este livro

Esta é a documentação canônica da Coral 1.7.0. As Partes I e II formam um percurso introdutório; as Partes III e IV cobrem projetos, bibliotecas e ferramentas; a Parte V explica a implementação; a Parte VI funciona como referência de consulta.

 **Caixas laterais destacam notas, compatibilidade e limites de evidência. Exemplos em Coral são validados pelo compilador da própria árvore antes da release.**

Sumário navegável

Como usar este livro	2
Sumário navegável	3
Sobre esta edição	15
Evidência desta edição	15
Da 1.5.4 para a 1.5.5	16
Da 1.5.5 para a 1.5.6	16
Da 1.5.6 para a 1.5.7	16
Da 1.5.7 para a 1.5.8	16
Da 1.5.8 para a 1.6.1	17
Da 1.6.1 para a 1.6.2	17
Da 1.6.2 para a 1.6.3	17
Da 1.6.3 para a 1.6.4	17
Da 1.6.4 para a 1.6.5	18
Da 1.6.5 para a 1.6.6	18
Da 1.6.6 para a 1.6.7	18
Da 1.6.7 para a 1.6.8	18
Da 1.6.9 para a 1.6.10	19
Da 1.6.10 para a 1.6.11	19
Da 1.6.11 para a 1.6.12	19
Da 1.6.12 para a 1.6.13	19
Da 1.6.13 para a 1.6.14	20
Da 1.6.14 para a 1.6.15	20
O ciclo que esta edição continua	20
PARTE I: COMEÇAR COM CORAL	21
1. O que é Coral e como este livro deve ser usado	21
Da 1.6.15 para a 1.6.16	21
Da 1.6.16 para a 1.6.17	21
Da 1.6.17 para a 1.6.18	22
Comece por aqui	22

Para quem este livro foi escrito	22
2. O que vem na distribuição Coral 1.7.0	23
Artefatos correntes	23
Dependências	24
3. Formas de instalar: persistente, portátil e desenvolvimento	24
Coral Setup, recomendado para uso comum	24
Coral Manager, recomendado para automação e diagnóstico	24
Runtime portátil	24
Desenvolvimento do compilador	24
4. Instalação completa no Linux	25
4.1 Requisitos	25
4.2 Setup ou assistente	25
4.3 Validar	25
4.4 VS Code	25
4.5 Componentes opcionais	25
5. Instalação completa no Windows	25
5.1 Estado da plataforma	25
5.2 Fluxos disponíveis nesta edição	25
5.3 VS Code	26
5.4 Testes reais assistidos	26
5.5 Validação honesta	26
6. VS Code: instalação, configuração e primeiro F5	26
Instalar	26
Confirmar o runtime	26
Executar	26
Depurar com F5	26
Estudar o código	27
Componentes e diagnóstico	27
7. Criar projetos e conhecer o fluxo de trabalho	27
Fluxo recomendado de desenvolvimento	27
8. Um primeiro programa explicado	28

PARTE II: A LINGUAGEM CORAL	29
9. Valores, variáveis, tipos e entrada	29
Nome, valor e alteração	29
Tipos de valor	29
Entrada de teclado	29
10. Operadores, expressões e decisões	30
Escolher um caminho	31
Combinar condições	31
11. Repetição e controle de fluxo	31
Intervalos e condição de continuidade	31
Interromper e pular uma passagem	32
Pensar na terminação	32
12. Coleções, índices, fatias e compreensões	32
Lista: sequência mutável	32
Dicionário: valores por chave	33
Outras coleções	33
Referências e mutabilidade	33
13. Funções e chamadas naturais	34
Parâmetros opcionais e nomeados	34
Escopo e funções internas	34
14. Geradores, tipagem opcional e abstrações	35
Produzir valores sob demanda	35
Tipagem opcional	36
Funções como valores	36
15. Classes, propriedades, herança e protocolos	36
Herdar comportamento	37
Referência de membros	37
16. Erros, padrões e contextos	38
Tratar falhas esperadas	38
Selecionar por padrão	38

Liberar recursos	38
17. Programação assíncrona	39
Regras de contexto	39
18. Espaço, geometria, movimento e consultas espaciais	40
Posições e formas em 2D	40
A mesma baseline em 3D	40
Relações geométricas	40
Consultas espaciais sobre coleções comuns	40
Movimento também é geral	41
Relação com coral.jogos	41
19. Mundos, mapas, células, camadas e regiões	41
Célula e posição	41
Topologia	42
Camada e valor	42
Vizinhança	42
Região	42
Travessia	43
Eventos de mapa	43
Persistência	43
Visualização opcional com Jogos	44
PARTE III: PROJETOS, BIBLIOTECAS E RECURSOS DA LINGUAGEM	45
20. Projetos, módulos, coral.toml e testes	45
Criar a estrutura	45
Importar funções locais	45
Testes nativos	46
21. Arquivos, JSON e biblioteca padrão	46
Gravar e ler dados	46
Serializar estruturas	47
Importações explícitas	47
22. Regras Coral, eventos e temporizadores	47
Observar mudanças sem inventar eventos manuais	48

Prioridade, cancelamento e consumo	48
Limites, cooldown e debounce	49
Tempo determinístico e relógio de teste	49
Grupos e regras temporárias	49
Sequências de dois acontecimentos	50
Eventos espaciais em 2D e 3D	50
Estados observáveis	50
Histórico e explicação	50
Motores e avaliação	50
API avançada	51
23. coral.mundo: entidades, relações e simulações	51
Forma recomendada	51
Entidades	51
Relações e caminhos	51
Eventos e regras	51
API orientada a objetos	52
Precisões preservadas na linha 1.2	52
Personagens e turnos	52
Referência dos componentes	53
24. coral.rpg: dados, personagens e combate por turnos	53
Personagens	53
Dados	53
Dano e cura	53
Combate por turnos	53
API avançada	54
25. coral.jogos e a integração com Pygame CE	54
Instalação do backend gráfico	54
Exemplo recomendado	54
Recursos atuais	54
Geometria compartilhada na 1.5.5	55
Mouse e tela cheia na 1.5.2	55

Spritesheets e transformações visuais introduzidas na 1.5.3	55
BackendNulo	57
Pygame CE	57
API avançada	57
Instalar a dependência correta	57
Criar um jogo	58
Ler a arquitetura do jogo	58
Teste sem janela	58
Construir o primeiro jogo, passo a passo	59
Referência prática dos objetos gráficos	59
26. Computação científica, geração procedural, laboratório, sistema e hardware, gráficos e Web	61
Númérico	61
Gráficos científicos 2D	62
Gráficos interativos locais	62
HTML e URLs locais	63
Cliente HTTP	63
Servidor HTTP local	64
Geração procedural determinística	64
Criptografia didática	65
Laboratório	65
Sistema	66
Hardware e serial	66
Integração científica e Web na 1.6.14	66
Coesão científica consolidada	67
Persistência	67
Pontes entre domínios	67
Hardware auditado	68
27. Modo Aprender, modo Ensinar e auditoria de naturalidade	68
Ensinar	68
Ensinar integrações consolidadas	68
Modo Aprender	69

Naturalidade	69
Regra principal	69
Prefira	69
Evite como estilo introdutório	69
APIs oficiais	70
Regras e eventos	70
Naturalidade automatizada	70
Integração com a edição tolerante da 1.3	70
PARTE IV: FERRAMENTAS, CLI E VS CODE	71
28. CLI Coral: executar, verificar, formatar, testar e documentar	71
Fluxos do dia a dia	71
Diagnóstico e release 1.3	71
Inspeção do compilador	71
Cache	71
Referência completa	72
29. VS Code 0.72.0: recursos de linguagem e produtividade	72
Resolução do runtime	72
Edição tolerante	72
Componentes	72
Coral Manager	72
30. Depuração: F5, breakpoints, pilha, variáveis e limites	72
Breakpoints e módulos locais	72
Navegação	73
O que ainda depende de ambiente gráfico	73
31. Diagnóstico, logs, atualização e solução de problemas	73
Coral Manager	73
Coral Setup	73
Assistentes	73
Logs	73
Atualizar uma instalação existente para a 1.7.0	73

PARTE V: IMPLEMENTAÇÃO TÉCNICA E ARQUITETURA	75
O que é normativo nesta edição	75
Princípios de manutenção	75
32. Fundação em Python: transpiler, runtime e empacotamento	75
Camadas de distribuição	75
O que transpilear significa aqui	76
Modelo de segurança	76
Interoperabilidade	76
Extras científicos da 1.3	76
33. Pipeline real do compilador	76
Caminho real de compilação	76
Expressões e interoperabilidade	77
Execução e projetos	77
Editor	77
Bibliotecas	78
Limites arquiteturais	78
Módulos centrais e tamanho aproximado nesta distribuição	78
34. Lexer e parser de expressões	79
Nós de expressão atuais	79
Precedência conceitual	79
AST, não execução	79
35. Parser de programa, blocos e AST estrutural	79
Nós principais	80
Delimitador de bloco	80
Chamadas naturais	80
Regras declarativas	80
36. Análise semântica, símbolos e diagnósticos	80
Dívida arquitetural conhecida	81
37. Backend estrutural, runtime e mapa de linhas	81
Cache	81
38. Projetos, resolução de módulos e construção	82

Exemplo de coral.toml	82
39. Language Server Protocol: arquitetura do LSP Coral	82
Capacidades declaradas pelo servidor 1.5.5	82
Ciclo simplificado	83
Edição tolerante e cache na 1.3	83
40. Debug Adapter Protocol: arquitetura do debugger	84
Capacidades atuais	84
Fluxo	84
Limites honestos	85
41. Arquitetura da extensão VS Code e empacotamento VSIX	85
Arquivos de execução principais	85
VSIX	85
42. Arquitetura da instalação persistente, Coral Manager e Setup	86
Componentes e perfis	86
Ciclo mínimo comprovável	86
Setup 1.5	86
Launchers correntes	86
43. Qualidade, conformidade, contratos e release 1.6.13	87
Gates permanentes	87
Arquitetura de testes	87
Patches e equivalência	87
Evidência externa	88
Fechamento público da 1.7.0	88
44. Como estender Coral sem quebrar sua identidade	88
1. Português corrente, não mera tradução lexical	88
2. Naturalidade controlada	88
3. Legibilidade antes de concisão	88
4. APIs oficiais devem ter uma superfície Coral	89
5. Previsibilidade acima de adivinhação	89
6. Python como plataforma, não como aparência	89

7. Interoperabilidade progressiva	89
8. AST e semântica próprias	89
9. Ensino sem infantilização	89
10. Critério para novos recursos	89
Checklist de implementação	90
11. Separar estabilidade de experimentação	90
Autoridade documental da base estável	90

PARTE VI: REFERÊNCIA COMPLETA 91

45. Referência sintática consolidada 91

1. Escopo	91
2. Princípio lexical e de leitura	91
3. Blocos	91
4. Estado e operações básicas	91
5. Funções e chamadas naturais	91
6. Regras, eventos e tempo	92
7. Superfície natural de mundos	92
8. Superfície natural de RPG	92
9. Superfície natural de jogos	92
10. Modo Aprender	93
11. Auditoria de Naturalidade	93
12. Ferramentas	93
13. Compatibilidade	93
14. Fora do escopo	93
15. Contrato de naturalidade estável	93
Léxico e blocos	94
O que não presumir	94
16. Superfície científica e de sistema da 1.3	95

46. Referência da biblioteca padrão 95

coral.arquivos	95
coral.texto	95
coral.matematica	96

coral.datas	97
coral.json	97
coral.colecoes	98
coral.aleatorio	98
Superfícies fundamentais acrescentadas na 1.5.8	99
Assinaturas confirmadas no runtime 1.6.9	100
APIs acrescentadas na linha 1.3	101
47. Referência das APIs regras, mundo, RPG e jogos	108
Constantes de cor em coral.jogos	120
48. API Python pública para ferramentas externas	120
Funções	120
Tipos públicos	120
49. Referência do CLI	121
Ajuda integral capturada da versão 1.6.1	122
50. Configurações de referência: coral.toml, settings e launch.json	124
coral.toml	124
Instalação persistente e VS Code	124
launch.json	124
51. Solução de problemas por sintomas	125
Antes de pedir ajuda	125
52. Exercícios e percurso de prática	126
Etapa 1: estado e fluxo	126
Etapa 2: dados e funções	126
Etapa 3: projeto real	126
Etapa 4: persistência	126
Etapa 5: reatividade	126
Critério de domínio	126
Projeto final sugerido	127
Etapa 6: ciência e reprodutibilidade	127
Etapa 7: sistema e hardware sem riscos desnecessários	127
Estados de animação em desenvolvimento na linha 1.5.3	127

Transições declarativas de estados em desenvolvimento na linha 1.5.3	128
53. Glossário, limites conhecidos e referências	128
Glossário	128
Limites conhecidos da 1.6.1	129
Referências internas	129
Referências externas oficiais	129
Encerramento	130
APÊNDICE A: CONSOLIDAÇÃO DO CICLO 1.4 → 1.5.5	131
Procedimento recomendado de atualização	131
Mudanças estruturais da distribuição	132
APÊNDICE B: MATRIZ DE VALIDAÇÃO DA RELEASE	133
APÊNDICE C: CONTRATOS CONSOLIDADOS	134
APÊNDICE D: REGISTRO HISTÓRICO DA ARQUITETURA DA LINHA 1.5.3	135
Adendo da candidata 1.5.3 — Regras, eventos e apresentação	135

Sobre esta edição

Esta é a edição corrente da documentação oficial da **Coral 1.7.0**. Ela preserva a base consolidada na linha 1.5, o fechamento arquitetural da 1.6.0, a expansão científica até 1.6.5, a base Web local da 1.6.6 e as capacidades de Sistema da 1.6.7. A 1.6.8 introduziu `coral.web.http` com cliente HTTP limitado, status e corpo em bytes, interpretação sob demanda, timeout, limite de resposta, redirecionamentos finitos e HTTPS verificado por padrão. A 1.6.9 acrescenta `coral.web.servidor` como servidor HTTP local de desenvolvimento com rotas literais, porta efêmera e fechamento explícito.

A fonte deste livro é a autoridade editorial corrente. Livros e notas de release anteriores permanecem em `Livro/Historico/` e `Documentacao/Historico/`; eles explicam a evolução do projeto, mas não substituem esta edição na descrição do estado atual.

Regra editorial. Quando existir uma forma Coral natural e oficial, ela é apresentada antes da forma orientada a objetos ou Python. A interoperabilidade continua disponível, mas não substitui a identidade da linguagem.

Regra de sincronização. Uma mudança significativa em uma biblioteca ou subsistema exige revisão dos exemplos gerais em `Exemplos/` e, quando houver valor pedagógico, dos exemplos canônicos em `Livro/Exemplos/`. A relação esperada é Código ↔ Teste ↔ Documentação ↔ Exemplos ↔ Livro.

Compatibilidade dimensional. Espaço, geometria, movimento e consultas espaciais são recursos da baseline, não um módulo público adicional. A baseline aceita 2D e 3D. `coral.jogos` continua deliberadamente 2D nesta versão. Misturar dados 2D e 3D exige conversão explícita; a linguagem não inventa $z = 0$.

Evidência desta edição

A validação continua organizada em camadas. A suíte rápida protege contratos cotidianos; a suíte de integração cobre fronteiras entre componentes; a suíte slow concentra operações de maior custo; a suíte histórica roda arquivos antigos em processos isolados e restaura temporariamente a topologia lógica necessária a cada contrato.

A distribuição também contém `Testes/Executar_Testes_Reais.coral`, ponto de entrada permanente para evidências que exigem máquina física, Pygame CE, janela, áudio, teclado, mouse, GPU, sistema operacional ou outra capacidade indisponível no ambiente automatizado. Indisponibilidade ambiental é registrada separadamente de falha funcional.

A extensão oficial corrente é **Coral Language 0.72.0**. O runtime portátil desta edição é `Instalacao/Runtime/coral-1.7.0.pyz`. A 1.7.0 consolida a distribuição nativa Linux, torna o executor privado capaz de gerenciar dependências opcionais com pip próprio e revisa perfis e Setup para derivarem da mesma fonte de verdade. Esta edição do Livro incorpora a integração científica da 1.6.14, preservando a documentação da 1.6.13 sobre gráficos HTML interativos locais, exploração explícita, limites de dados, acessibilidade e integração controlada com o servidor Web. A superfície espacial dedicada permanece em 3D; capacidades públicas dedicadas a quatro dimensões espaciais foram retiradas definitivamente do escopo ativo da linha 1.6.x. A árvore física possui seis áreas oficiais: `Instalacao/`, `Documentacao/`, `Projeto/`, `Exemplos/`, `Testes/` e `Livro/`.

Da 1.5.4 para a 1.5.5

A 1.5.4 consolidou colisão configurável por sprite, automação matemática determinística de posição, escala, rotação e opacidade, novas primitivas gráficas, ensino combinado e a prévia segura de quebra de linhas no editor. A 1.5.5 usa essa fundação para introduzir espaço, geometria, movimento e consultas espaciais de forma geral, com suporte 2D e 3D fora de Jogos, enquanto `coral.jogos` reutiliza a mesma geometria como consumidor 2D.

A 1.5.5 também substitui a antiga árvore numerada por seis áreas de responsabilidade. A mudança física não redefine APIs públicas históricas: contratos antigos são preservados por resolvedores de recursos, fixtures e pelo runner histórico isolado.

Da 1.5.5 para a 1.5.6

A 1.5.6 consolida eventos e regras reativas como infraestrutura comum da linguagem. Eventos passam a carregar dados e metadados estruturados; regras podem usar prioridades, limites, cooldown, grupos e duração; mudanças de valores e relações espaciais podem ser observadas; sequências simples usam tempo lógico determinístico; e o rastreamento reativo alimenta diagnósticos e o modo Ensinar sem reexecutar condições. A superfície introdutória continua em português corrente e a API orientada a objetos permanece destinada a integração avançada.

Da 1.5.6 para a 1.5.7

A 1.5.7 acrescenta **mundos, mapas e regiões discretas** a `coral.mundo`. Mapas quadrados, hexagonais e voxel compartilham o mesmo modelo de camadas esparsas e valores genéricos; regiões podem ser definidas por células ou por formas espaciais; travessia é opcional; e mudanças usam o mesmo barramento de eventos da 1.5.6. `coral.jogos` é apenas um consumidor 2D opcional. Os exemplos de temperatura, autômato celular e robótica demonstram que o núcleo não depende de Pygame, sprites ou conceitos de RPG.

Da 1.5.7 para a 1.5.8

A 1.5.8 é uma revisão estrutural da base da linguagem. O compilador passa a compartilhar um Snapshot de análise com AST localizada por spans; entrada, conversões e erros básicos recebem superfícies Coral próprias; a persistência portátil preserva categorias de valores fundamentais; módulos expõem interfaces explícitas e chamadas naturais podem atravessar imports; projetos ganham `coral.toml` validado, recursos estáveis e build incremental; a tipagem opcional aponta incompatibilidades locais sem retirar o caráter dinâmico da linguagem; testes nativos, REPL, LSP, DAP e integração com VS Code passam a reutilizar os mesmos contratos estruturais.

A biblioteca padrão também amplia arquivos, datas, texto, matemática, formatos e aleatoriedade reproduzível. A referência sintática corrente é versionada e gerada a partir de contratos executáveis, reduzindo divergências entre runtime, ferramentas e documentação.

Da 1.5.8 para a 1.6.1

A documentação 1.5.8 permaneceu deliberadamente estável enquanto a linguagem continuou evoluindo. Entre a 1.5.9 e a 1.6.0, a extensão oficial foi aprofundada, surgiram agentes e filas de turnos gerais, a fundação `coral.procedural` passou a oferecer sementes derivadas, consultas determinísticas, sequências, campos e distribuição espacial, e Jogos recebeu uma camada maior de câmera, animação, apresentação e inspeção. A linha também consolidou um fluxo de testes com seleção por impacto, evidências reutilizáveis, higiene da árvore e checkpoints de release.

A 1.5.19 ampliou formas de português corrente e a 1.5.20 e 1.5.21 concentraram sustentabilidade e coesão. A 1.6.0 encerrou esse ciclo com fronteiras arquiteturais explícitas, serviços internos preguiçosos, fachadas compatíveis e zero ciclos ou violações de camada admitidos no fechamento. A 1.6.1 auditou a base científica e preparou contratos de evolução. A 1.6.2 entregou a expansão de `coral.matematica`. A 1.6.3 ampliou `coral.numerico` com arrays e estatística descritiva. A 1.6.4 acrescentou álgebra linear e geometria 3D. A 1.6.5 inaugurou `coral.graficos` com especificações 2D independentes de backend e exportação SVG. A 1.6.6 acrescentou a base local de `coral.web` com HTML e URLs sem rede. A 1.6.7 ampliou `coral.sistema` com capacidades do hospedeiro e processos controlados. A 1.6.8 introduziu o cliente HTTP geral. A 1.6.9 acrescenta o servidor HTTP local com rotas literais, limites explícitos e encerramento controlado. A 1.6.10 aprofunda `coral.laboratorio` com planejamento, seleção, resumo por parâmetro, JSON recuperável e comparação descritiva. A 1.6.11 acrescenta estatística avançada, reamostragem e gráficos de correlação e incerteza com exportação estática opcional. A 1.6.12 aprofunda geometria e visualização tridimensionais e retira definitivamente do escopo ativo a superfície pública espacial 4D.

Da 1.6.1 para a 1.6.2

A 1.6.2 amplia `coral.matematica` sem carregar NumPy. A nova superfície inclui combinatória (fatorial, combinacoes, arranjos), comparação aproximada e interpolação, logaritmos de base 10 e 2, trigonometria inversa, raiz n ésima real, produto de coleções, sinal e volume de bola tridimensional. Os contratos tornam explícitos domínios inválidos, valores não finitos e a diferença entre interpolação e extrapolação. As APIs existentes permanecem compatíveis.

Da 1.6.2 para a 1.6.3

A 1.6.3 amplia `coral.numerico` com identidade, remodelar, concatenar, `selecionar_coluna`, percentis, covariância, correlação, frequências, moda multimodal, quartis, medidas robustas de dispersão, médias ponderada, geométrica e harmônica e histogramas. Estatísticas numéricas rejeitam dados ausentes, NaN e infinito com `ErroNumerico`, enquanto `selecionar_coluna` exige uma política explícita para ausentes. A mesma superfície aceita séries comuns e dados extraídos de registros sem depender de `coral.laboratorio`.

Da 1.6.3 para a 1.6.4

A 1.6.4 aprofunda `coral.numerico` com norma, normalizar, `produto_vetorial`, `posto`, `minimos_quadrados`, `vetor3`, distância, ângulo, projeção, produto misto e volume. `rotacionar3` usa eixo arbitrário e ângulo em radianos. `transformar_ponto3` usa matriz homogênea 4×4 , vetor coluna, translação na última coluna e última linha afim $[0, 0, 0, 1]$. Formas erradas, vetores degenerados, NaN e infinito continuam produzindo `ErroNumerico` em português. A baseline espacial histórica permanece compatível e não é substituída por essas funções numéricas.

Da 1.6.4 para a 1.6.5

A 1.6.5 cria `coral.graficos` como camada de apresentação separada. `linha`, `dispersao` e `barras` recebem dados comuns diretamente; `series_de_registros` seleciona campos de mapeamentos e objetos sem depender de `Laboratório`. `grafico_histograma` reutiliza exatamente as contagens e bordas de `coral.numerico.histograma`, enquanto `grafico_caixa` reutiliza os quartis de `Númeroico` e declara bigodes de Tukey de $1,5 \times \text{IQR}$. `grafico_funcao` preserva avaliações inválidas como lacunas. `configurar` cria uma nova especificação com título, eixos, unidades, legenda, tema, proveniência e texto alternativo. `salvar` escreve SVG local e autossuficiente sem `Matplotlib`, `Plotly`, janela ou rede.

Da 1.6.5 para a 1.6.6

A 1.6.6 cria a base local de `coral.web`. Em `coral.web.html`, `escapar_texto` e `escapar_atributo` codificam valores conforme o contexto, enquanto `elemento`, `fragmento`, `documento`, `renderizar` e `tabela` compõem HTML sem transformar texto comum em marcação confiável. A superfície recusa atributos de evento inline e esquemas executáveis em atributos de URL. Em `coral.web.url`, `codificar_consulta` e `ler_consulta` preservam ordem, vazios e chaves repetidas em UTF 8; `decompor` e `juntar` trabalham somente com componentes textuais de endereços. Importar esses módulos não acessa a rede nem inicia servidor.

Da 1.6.6 para a 1.6.7

A 1.6.7 amplia `coral.sistema` sem transformar o módulo em inventário de hardware. `localizar_programa` e `programa_disponivel` consultam executáveis sem iniciá los; `quantidade_processadores`, `ambiente_selecionado`, `variavel_ambiente_obrigatoria`, `pasta_temporaria` e `terminal_interativo` descrevem capacidades explícitas do processo. `argumentos_programa` separa os argumentos do programa das opções internas do runtime e `caminho_executavel` identifica o hospedeiro do PYZ. `ResultadoComando` ganha leituras por linhas, aliases naturais e `exigir_sucesso`. `executar_comando` passa a aceitar codificação, política de decodificação e limite de captura aplicado durante a leitura, com flags de truncamento e preservação da execução sem shell.

Da 1.6.7 para a 1.6.8

A 1.6.8 inaugura `coral.web.http` como cliente HTTP geral e limitado. `requisitar`, `obter` e `enviar` preservam status e corpo em bytes; `RespostaHTTP.texto` e `RespostaHTTP.json` interpretam conteúdo somente sob demanda. `Timeout` e limite de resposta são finitos, redirecionamentos possuem teto e métodos com efeito não são repetidos implicitamente. `codificar_formulario` preserva campos repetidos em UTF 8. HTTPS usa verificação de certificado e hostname por padrão, enquanto proxy e contexto TLS permanecem explícitos. Importar Web continua sem abrir conexão ou servidor.

Da 1.6.9 para a 1.6.10

A 1.6.10 amplia `coral.laboratorio` com `planejar_experimento`, `selecionar_medicoes`, `resumir_por_parametro`, `validar_resultado`, `carregar_resultado_json` e `comparar_experimentos`. O JSON completo passa a usar o contrato versionado `coral.laboratorio.resultado/1`, com recuperação de tipos portáteis e rejeição de esquemas desconhecidos. A comparação é apenas descritiva e nunca apresenta significância estatística como conclusão automática.

Da 1.6.10 para a 1.6.11

A 1.6.11 amplia `coral.numerico` com erro padrão, assimetria, curtose, matrizes de covariância e correlação e regressão linear estruturada. `intervalo_bootstrap` e `teste_permutacao` usam fonte aleatória local inicializada por semente, registram método e custo e não alteram o gerador aleatório global. Em `coral.graficos`, mapas de calor e correlação, barras de erro, faixas de incerteza, painéis e o adaptador de experimentos passam pelo mesmo mecanismo de especificações gerais. PNG e PDF usam Matplotlib somente quando a exportação é solicitada.

Da 1.6.11 para a 1.6.12

A 1.6.12 aprofunda a geometria tridimensional de `coral.numerico`. Entram matrizes de rotação, translação e escala, composição de transformações, transformação de vetores, distância e projeção de ponto em plano, interseção de reta e plano, conversões entre coordenadas cartesianas e esféricas, área de triângulo e volume de tetraedro. As operações validam formas, finitude e degenerações geométricas antes do cálculo.

Em `coral.graficos`, `dispersao3d`, `trajetoria3d` e `superficie3d` preservam as coordenadas fornecidas. Superfícies exigem grade retangular explícita e não interpolam pontos ausentes. Criar a especificação não abre janela nem carrega Matplotlib. `para_svg` continua sendo o renderizador próprio 2D; para gráficos 3D, `salvar` carrega Matplotlib somente sob demanda e exporta SVG, PNG ou PDF. A linha 1.6.x passa a concentrar a superfície espacial dedicada em 3D; APIs públicas dedicadas a quatro dimensões espaciais não fazem parte do escopo ativo.

Da 1.6.12 para a 1.6.13

A 1.6.13 transforma as especificações já publicadas em `coral.graficos` em visualizações HTML interativas locais. `para_html` e `salvar_html` usam Plotly apenas sob chamada explícita e incorporam o JavaScript necessário no próprio arquivo, sem CDN. `explorar` também é explícito e, em sessão sem interface, orienta salvar o HTML em vez de iniciar servidor ou escolher uma alternativa oculta.

Gráficos 2D, `dispersao3d`, `trajetoria3d`, `superficie3d` e painéis preservam os dados da especificação. A exportação possui limite de observações e células e não reduz dados silenciosamente. `mostrar_grafico` permite redução uniforme explícita apenas para séries simples compatíveis e registra a decisão. Texto alternativo, avisos e proveniência formam a descrição acessível da figura, e o arquivo informa que contém os dados incorporados.

`coral.web.servidor.resposta_grafico` adapta uma figura para `RespostaServidorHTTP` sem abrir porta nem iniciar servidor. A publicação continua exigindo criação do servidor, registro de rota e iniciar explícitos. Plotly permanece opcional e a linha 1.6.x continua sem superfície pública dedicada a espaço 4D.

Da 1.6.13 para a 1.6.14

A 1.6.14 combina superfícies já publicadas em dois percursos completos. Registos comuns de temperatura atravessam seleção de colunas, média escalar, gráfico SVG e tabela HTML escapada; nenhum objeto experimental é exigido. Um experimento com grade finita e sementes explícitas produz JSON recuperável e um gráfico com proveniência; a apresentação HTML interativa usa Plotly opcional e o servidor atende apenas quando o programa registra rotas e inicia uma porta em loopback. Os três programas reais ficam em `Exemplos/Integracao/`.

A mesma release especifica o formato futuro de pacote local, a verificação de versão e SHA256 e a recuperação da instalação. Os assistentes desta versão ainda usam Python do hospedeiro; executáveis nativos e runtime privado são entregas previstas para a 1.6.17. A Coral 1.6.14 não oferece atualização remota automática.

Da 1.6.14 para a 1.6.15

A 1.6.15 refina `coral.procedural` sem transformar geração procedural em dependência de Jogos, RPG, Laboratório ou Gráficos. Sementes e subsementes continuam determinísticas por domínio; consultas por índice ou coordenada permanecem independentes da ordem de uso; escolhas ponderadas validam pesos finitos, não negativos e de soma positiva. Campos passam a declarar dimensionalidade e intervalo conhecido quando possível, e composição, normalização e máscaras preservam a origem sem introduzir NaN silenciosamente.

`ReceitaProcedural` registra algoritmo, versão do contrato, semente tipada e parâmetros portáteis. `receita_procedural` constrói receitas para `campo_constante`, `campo_ruido` e `distribuir_pontos`; `reconstruir_receita` valida o orçamento antes de reconstruir o resultado. `diagnosticar_custo_receita` expõe a estimativa conhecida e `SequenciaProcedural.trecho` exige limite explícito quando a materialização solicitada ultrapassa o teto corrente. Callbacks arbitrários, chunks infinitos e persistência automática de mundos continuam fora deste contrato.

O ciclo que esta edição continua

A 1.5.0 consolidou a era de solidificação. As versões seguintes passaram a aprofundar coesão e manutenção sem abandonar português corrente, determinismo e compatibilidade. A edição 1.7.0 mantém essa visão editorial, preserva as integrações publicadas até a 1.6.17 e concentra a mudança corrente na distribuição e instalação: subsementes isoladas, consultas determinísticas, campos e máscaras componíveis, distribuição espacial limitada e receitas portáteis reconstruíveis. A integração científica e Web da 1.6.14 permanece preservada sem criar novas dependências obrigatórias.

PARTE I: COMEÇAR COM CORAL

Instalação, ambiente, VS Code e primeiros programas. Nenhum conhecimento de compiladores é necessário nesta parte.

1. O que é Coral e como este livro deve ser usado

Coral é uma linguagem de programação em português corrente controlado, com gramática determinística e execução sobre Python. Esta edição documenta a **Coral 1.7.0**, preservando a compatibilidade da linha 1.5 e o fechamento arquitetural da 1.6.0. Recursos descritos como introduzidos em versões anteriores mantêm suas referências históricas; o estado corrente é identificado explicitamente quando necessário.

A ideia central é simples: um programa Coral deve poder ser lido como uma descrição precisa do algoritmo, sem depender de interpretação livre de linguagem natural.

```
defina idade como 17
se idade for maior ou igual a 18 então
    mostre "Acesso permitido"
senão
    mostre "Acesso negado"
fim
```

Da 1.6.15 para a 1.6.16

A 1.6.16 refina `coral.criptografia` sem inaugurar cifragem geral. As nove operações públicas existentes permanecem estáveis. Texto é codificado em UTF 8, bytes são preservados, SHA 256 e SHA 512 ficam restritos às operações de resumo e HMAC às operações de autenticação. Verificações distinguem conteúdo divergente de representação malformada e os diagnósticos não repetem chaves ou conteúdos sensíveis.

`gerar_bytes_seguros` e `gerar_token_seguro` continuam usando `secrets`, enquanto o catálogo didático passa a declarar operações aceitas, uso de chave, algoritmo base, tamanho, codificação textual e formatos de saída. Cifragem, KDF, certificados, assinatura digital e gestão de chaves continuam fora desta edição.

Da 1.6.16 para a 1.6.17

A 1.6.17 recupera a integração científica e Web com `Plotly` real e aprofunda `coral.procedural` antes das validações globais. `amostrar_campo` transforma coordenadas em registros gerais, `matriz_campo` produz grades 2D finitas, `anotar_pontos` combina distribuições e campos sem mutar as entradas e `proveniencia_procedural` distingue descrições reconstruíveis de objetos vivos em memória. O núcleo Procedural continua sem importar `Jogos`, `Laboratório`, `Numérico` ou `Gráficos`.

`Jogos` passa a oferecer `sprites_de_pontos`, `sprites_de_receita`, `popular_cena_procedural` e `aplicar_campo_em_sprites`. Essas operações funcionam com `Cena` e `Sprite` mesmo sem `Pygame`, permitindo que a mesma receita determine posições e que campos ambientais anotem objetos visuais. `Laboratório` acrescenta `planejar_experimento_procedural` e `executar_experimento_procedural`: sementes procedurais aparecem como fatores

experimentais explícitos, a receita base entra nos metadados e o custo máximo pode ser recusado antes da execução. Registros e matrizes produzidos por Procedural atravessam os contratos gerais de Gráficos, evitando uma dependência inversa.

Da 1.6.17 para a 1.6.18

A 1.6.18 não amplia a linguagem: ela fecha a distribuição nativa e revisa o fluxo de instalação. O catálogo de componentes e perfis passa a ser a fonte única de verdade consumida por Manager e Setup. Científico seleciona NumPy, Matplotlib, Plotly e psutil; Jogos seleciona Pygame CE; Hardware seleciona psutil e pyserial; Completo reúne todas as dependências opcionais oficiais. Seleção manual é registrada como Personalizado.

O pacote local contém Python privado e pip privado. Dependências opcionais são instaladas somente nesse executor, sem alterar o Python global, e são reaplicadas quando uma atualização substitui a árvore privada. A integridade mantém SHA256 para os bytes originais e admite novos arquivos somente nas áreas mutáveis previstas para pacotes e caches Python.

No Linux x86_64, a edição entrega Coral_Setup_1.6.18_Linux_x86_64 e coral-setup_1.6.18_amd64.deb, exercitados com PATH vazio, diagnóstico e remoção. O Setup apresenta adicionar, atualizar, manter e remover e permite restaurar um preset sem alterar escolhas independentes como VS Code. A trilha Windows está preparada, mas sua construção e validação nativas foram adiadas explicitamente para a 1.7.0.

Comece por aqui

Se este é seu primeiro contato com Coral, não é necessário conhecer compiladores nem arquitetura interna. Leia os capítulos 1 a 8 em ordem, execute os exemplos e use a Parte II quando quiser aprender a linguagem com mais profundidade. A Parte I existe para colocar um programa em execução antes de apresentar a engenharia que sustenta a linguagem.

Para quem este livro foi escrito

- **Iniciantes** podem seguir as Partes I e II em sequência.
- **Estudantes e programadores intermediários** encontram projetos, testes, computação numérica, experimentos, sistema, jogos, depuração e VS Code nas Partes III e IV.
- **Mantenedores e autores de ferramentas** podem começar pela Parte V, dedicada ao compilador, à arquitetura interna, ao LSP, ao DAP e ao empacotamento.
- **Quem precisa consultar rapidamente uma construção ou API** deve usar a Parte VI.

Para mantenedores. Detalhes normativos de versão, artefatos e validação ficam concentrados na Parte V. O percurso introdutório não depende deles.

2. O que vem na distribuição Coral 1.7.0

A distribuição completa separa responsabilidades em seis áreas principais. Código fonte, instalação, testes, documentação, exemplos e Livro Oficial não competem pelo mesmo diretório. Histórico técnico permanece dentro das áreas correspondentes, sem recriar uma segunda árvore ativa.

```

Coral/
|--- 00_LEIA_PRIMEIRO.md
|--- Coral.code-workspace
|--- Instalacao/
|   |--- Runtime/
|   |--- Extensao_VSCode/
|   |--- Assistentes/
|   |--- Ferramentas/
|--- Documentacao/
|   |--- Usuario/
|   |--- Desenvolvimento/
|   |--- Identidade_Visual/
|   |--- Historico/
|--- Projeto/
|   |--- coral/
|   |--- tools/
|   |--- vscode-coral/
|   |--- configuracao/
|   |--- auditoria/
|--- Exemplos/
|--- Testes/
|   |--- RapiDOS/
|   |--- Integracao/
|   |--- Slow/
|   |--- Historicos/
|   |--- Campo/
|   |--- Executar_Testes_Reais.coral
|--- Livro/
|   |--- Coral_1.7.0_Livro_Fonte.md
|   |--- Coral_1.7.0_Livro_Oficial.pdf
|   |--- Exemplos/
|   |--- recursos/

```

Artefatos correntes

Artefato	Arquivo	Papel
Runtime Coral	Instalacao/Runtime/coral-1.7.0.py z	aplicação Python zip portátil
Extensão VS Code	Instalacao/Extensao_VSCode/coral- language-0.72.0.vsix	editor, LSP, DAP, testes e experiência de estudo
Fonte da extensão	Instalacao/Extensao_VSCode/coral- vscode-0.72.0-source.zip	auditoria e desenvolvimento da extensão
Coral Manager	coral manager	diagnóstico, instalação, atualização, reparo e remoção
Coral Setup	Instalacao/Coral_Setup.*	interface que consome o mesmo contrato do Manager
Livro Oficial	Livro/Coral_1.7.0_Livro_Oficial.p df	documentação canônica da edição
Testes reais	Testes/Executar_Testes_Reais.cora l	evidência assistida em máquina física

Dependências

O núcleo exige **Python 3.10 ou superior**. NumPy, Matplotlib, Plotly, psutil, pyserial e Pygame CE continuam opcionais e são administráveis pelo sistema oficial de componentes e perfis. A extensão declara compatibilidade com **VS Code ^1.85.0**.

3. Formas de instalar: persistente, portátil e desenvolvimento

Há quatro caminhos oficiais, todos apoiados pela mesma base de runtime.

Coral Setup, recomendado para uso comum

Abra `Instalacao/Coral_Setup.cmd` no Windows ou `Instalacao/Coral_Setup.sh` em sistemas POSIX. A interface usa o contrato `coral.manager/1`: não existe um segundo instalador escondido na GUI.

Coral Manager, recomendado para automação e diagnóstico

```
python Instalacao/Runtime/coral-1.7.0.pyz manager diagnosticar --json
python Instalacao/Runtime/coral-1.7.0.pyz manager instalar --runtime coral-1.7.0.pyz
--perfil estudante --json
python Instalacao/Runtime/coral-1.7.0.pyz manager instalar --runtime coral-1.7.0.pyz
--perfil estudante --aplicar --json
```

Operações mutáveis produzem primeiro um plano. A alteração é aplicada apenas quando o usuário pede explicitamente.

Runtime portátil

```
python3 Instalacao/Runtime/coral-1.7.0.pyz --versao
python3 Instalacao/Runtime/coral-1.7.0.pyz meu_programa.coral
```

Desenvolvimento do compilador

```
python -m venv .venv
# Linux/macOS
source .venv/bin/activate
# Windows PowerShell
.\.venv\Scripts\Activate.ps1
python -m pip install -e .
python -m coral --self-check
```

A instalação persistente continua disponível pelos assistentes em `Instalacao/Assistentes`.

4. Instalação completa no Linux

4.1 Requisitos

O executável Linux x86_64 inclui o pacote Coral com Python e pip privados. O modo portátil continua aceitando Python 3.10 ou superior quando o usuário escolhe executar o PYZ diretamente.

4.2 Setup ou assistente

Na distribuição nativa x86_64, execute:

```
./Projeto/dist/nativos/Coral_Setup_1.7.0_Linux_x86_64
```

Também existe Projeto/dist/nativos/coral-setup_1.7.0_amd64.deb. Os launchers Instalacao/Coral_Setup.sh e os assistentes de terminal continuam disponíveis para o fluxo portátil e de desenvolvimento.

4.3 Validar

```
coral --versao
coral --self-check
coral --gate-1-3
coral --gate-1-4
coral diagnostico --completo
```

4.4 VS Code

O VSIX corrente fica em Instalacao/Extensao_VSCode/coral-language-0.72.0.vsix. Instale pela interface **Extensions: Install from VSIX...** ou pelo CLI do editor.

4.5 Componentes opcionais

Use coral componentes listar, coral componentes verificar e coral componentes instalar. Perfis como estudante, científico, hardware, jogos e completo agrupam capacidades sem transformar dependências opcionais em requisitos do núcleo.

5. Instalação completa no Windows

5.1 Estado da plataforma

A trilha Windows x86_64 está preparada, mas a construção e a validação nativas do Setup.exe foram adiadas para a 1.7.0. A 1.7.0 não trata scripts ou build cruzado como evidência de execução Windows.

5.2 Fluxos disponíveis nesta edição

Instalacao\Coral_Setup.cmd, Coral_Manager.cmd e os assistentes continuam disponíveis para quem já possui Python compatível no Windows. O executável Windows autossuficiente será qualificado no marco seguinte.

5.3 VS Code

Instale `Instalacao\Extensao_VSCode\coral-language-0.72.0.vsix`. Depois recarregue a janela e use **Coral: Mostrar runtime selecionado**.

5.4 Testes reais assistidos

Como a construção oficial não possui um host Windows/GUI real, a distribuição inclui:

 `Testes\Executar_Testes_Reais.coral`

Esse ponto de entrada seleciona testes que dependem de ambiente físico, como janela, Pygame, teclado, mouse, áudio, GPU ou comportamento específico do sistema. O runner executa automaticamente o que puder, pede confirmação humana apenas quando necessário, distingue falha de indisponibilidade ambiental e grava `resumo.txt`, `resultados.json`, `ambiente.json` e `detalhes.log` em `Testes/Resultados/`.

5.5 Validação honesta

Contratos automatizados cobrem a parte reproduzível do fluxo Windows, mas capacidades nativas só recebem evidência real quando `Testes/Executar_Testes_Reais.coral` é executado no host apropriado. Ausência de uma capacidade é registrada como indisponível ou não executada, não como aprovação artificial.

6. VS Code: instalação, configuração e primeiro F5

A extensão oficial é **Coral Language 0.72.0**.

Instalar

Use `coral-language-0.72.0.vsix` da pasta `Instalacao/Extensao_VSCode`. Depois execute **Developer: Reload Window**.

Confirmar o runtime

Use **Coral: Mostrar runtime selecionado**. A extensão respeita configuração explícita e, na ausência dela, consulta a instalação persistente.

Executar

Coral: Executar arquivo no terminal usa o runtime selecionado e mantém entrada e saída no terminal.

Depurar com F5

Abra um arquivo `.coral`, coloque um breakpoint e pressione F5. O DAP suporta breakpoints no arquivo principal e em módulos `.coral` locais importados, pilha, variáveis, evaluate e step in/over/out.

Estudar o código

O Modo Aprender e o Coral Ensinar explicam o programa sem exigir alteração do código fonte. O estudo automático no editor é opcional e pode ser ligado ou desligado pelo usuário.

Componentes e diagnóstico

A extensão também oferece comandos para verificar suporte opcional, instalar componentes pelo gerenciador oficial, abrir o Coral Manager e diagnosticar a integração VS Code.

7. Criar projetos e conhecer o fluxo de trabalho

Coral oferece templates oficiais:

Template	Objetivo
basico	Ponto de partida mínimo
terminal	Aplicativo interativo de terminal
jogo-2d	Projeto usando coral.jogos
aventura	Mundo, RPG e Regras Coral
simulacao	Estados e entidades
didatico	Exemplos graduais

Liste e crie:

```
coral templates
coral novo MeuProjeto --template basico
cd MeuProjeto
coral executar
```

Projetos usam coral.toml para identificar entrada, versão e módulos. O VS Code oferece **Coral: Novo Projeto** e um Project Explorer próprio.

Fluxo recomendado de desenvolvimento

```
Editar .coral
↓
coral verificar / diagnósticos do LSP
↓
coral testar
↓
F5 / Debug Adapter
↓
coral naturalidade .
↓
coral documentar
```

8. Um primeiro programa explicado

Considere:

```
defina idade como 17
defina mensagem como ""
se idade for maior ou igual a 18 então
    defina mensagem como "maior de idade"
senão
    defina mensagem como "menor de idade"
fim
mostre mensagem
```

- defina cria ou atualiza um nome.
- como separa o nome do valor.
- se ... então abre uma condição.
- for maior ou igual a é uma comparação escrita como português.
- senão fornece o caminho alternativo.
- fim encerra o bloco.
- mostre produz saída.

Experimente:

```
coral explicar primeiro.coral
coral ensinar primeiro.coral
coral naturalidade primeiro.coral
```

explicar/Modo Aprender **executa** e rastreia valores. ensinar faz análise estática e não executa o programa. naturalidade verifica estilo e idioma da superfície Coral; não é uma prova de correção lógica.

PARTE II: A LINGUAGEM CORAL

Do básico ao intermediário: valores, fluxo, coleções, funções, objetos, erros e concorrência assíncrona.

9. Valores, variáveis, tipos e entrada

Nome, valor e alteração

Uma variável relaciona um nome a um valor. `defina pontos como 10` cria ou atribui o valor; `adicione` altera esse estado. A sequência importa: a linha posterior vê o resultado das anteriores.

Arquivo: `Exemplos/Fundamentos/01_fluxo_e_colecoes.coral`.

```
defina pontos como 10
adicione 5 a pontos
subtraia 2 de pontos
mostre pontos
```

Saída esperada:

```
13
```

Após definir, o valor é 10; após adicionar, 15; após subtrair, 13. A sintaxe alternativa `pontos recebe 20` também faz atribuição. Um nome deve existir antes de ser consultado, salvo quando é introduzido pelo próprio contexto, como um parâmetro de função.

Tipos de valor

Categoria	Exemplos	Observação
Inteiro	0, 19, -3	Contagens e valores sem parte fracionária
Decimal	2.5, 0.01	Usa ponto decimal no código
Texto	"Coral", 'Ana'	Preserva os espaços dentro das aspas
Booleano	verdadeiro, falso	Condições e resultados lógicos
Ausência	nulo	Representa ausência de valor
Coleções	[1, 2], {"nome": "Ana"}	Agrupam vários valores

A execução usa objetos Python. Por isso, a impressão direta de certos valores segue sua representação, como `True`, `False` e `None`. Isso não muda a grafia recomendada no código Coral.

Entrada de teclado

A 1.5.8 fornece uma superfície Coral própria para entrada. O caminho principal é `coral.entrada`; não é necessário importar `input` de Python para um programa de terminal comum.

Arquivo: `Exemplos/Dados/01_entrada_conversoes_e_texto.coral`.

```

de coral.entrada importe ler_linha
de coral.conversoes importe inteiro e decimal e texto

defina nome como ler_linha("Nome: ")
defina idade como inteiro(ler_linha("Idade: "))
defina altura como decimal(ler_linha("Altura: "))
mostre "Olá, " mais nome
mostre idade
mostre altura
mostre texto(idade)

```

`ler_linha` sempre devolve texto e traduz o fim da fonte para o erro `Coral FimDeEntrada`. Ferramentas como testes, REPL e debugger podem fornecer uma fonte de entrada sem trocar a API usada pelo programa.

As conversões oficiais são `inteiro`, `decimal`, `texto` e `booleano`. Elas são estritas quando uma conversão seria ambígua e aceitam um valor padrão opcional quando o programa deseja tratar uma entrada inválida sem capturar uma exceção.

```

de coral.conversoes importe inteiro e booleano

defina quantidade como inteiro("12")
defina confirmado como booleano("sim")
mostre quantidade
mostre confirmado

```

Interoperação com `int`, `float`, `str` e outras funções Python continua possível no exemplo específico de interoperabilidade, mas não é o caminho didático principal para entrada e conversão na 1.5.8.

Exercício: peça um número com `ler_linha`, converta com `inteiro` e mostre o dobro. Depois teste uma entrada não numérica com e sem um valor padrão.

10. Operadores, expressões e decisões

Operadores combinam valores. `mais`, `menos`, `vezes` e `dividido por` são formas de leitura corrente. Operadores simbólicos continuam disponíveis, especialmente `%`, `//` e `**`.

Intenção	Forma Coral	Forma simbólica
Soma	<code>a mais b</code>	<code>a + b</code>
Subtração	<code>a menos b</code>	<code>a - b</code>
Multiplicação	<code>a vezes b</code>	<code>a * b</code>
Divisão	<code>a dividido por b</code>	<code>a / b</code>
Igualdade	<code>a for igual a b</code>	<code>a == b</code>
Diferença	<code>a for diferente de b</code>	<code>a != b</code>
Ordem	<code>a for maior que b</code>	<code>a > b</code>
Limite incluso	<code>a for menor ou igual a b</code>	<code>a <= b</code>
Pertencimento	<code>a estiver em valores</code>	Forma natural recomendada

Não use `=` como teste de igualdade. Ele aparece em argumentos nomeados, enquanto comparações usam `for igual a` ou `==`.

Escolher um caminho

Arquivo: Exemplos/Fundamentos/01_fluxo_e_colecoes.coral.

```
defina media como 7
se media for maior ou igual a 7 então
  mostre "Aprovado"
senão se media for maior ou igual a 5 então
  mostre "Recuperação"
senão
  mostre "Reprovado"
fim
```

Saída esperada:

```
Aprovado
```

O programa testa os ramos em ordem e executa apenas o primeiro cuja condição for verdadeira. senão cobre os demais casos e é opcional. fim fecha a decisão inteira. A indentação comunica a estrutura, mas o encerramento explícito é indispensável.

Combinar condições

Use e, ou e não. Prefira parênteses ao combinar comparações com negação: não (idade for menor que 18). O parser não interpreta toda frase possível em português; siga uma forma reconhecida.

A precedência implementada, da menor para a maior, é: ou, e, comparação, soma e subtração, produto e divisão, potência, operadores unários e operações de chamada, atributo e índice. Não presume que todo caso de precedência seja idêntico ao Python; use parênteses para tornar explícita sua intenção, principalmente com potência e sinal negativo.

Exercício: altere a média para 6 e depois para 4. Antes de executar, escreva qual mensagem espera. Em seguida, acrescente uma condição que recuse valores menores que zero.

11. Repetição e controle de fluxo

Intervalos e condição de continuidade

Arquivo: Exemplos/Fundamentos/01_fluxo_e_colecoes.coral.

```
defina total como 0
para numero de 1 até 4 faça
  adicione numero a total
fim
mostre total
defina restante como 3
enquanto restante for maior que 0 faça
  subtraia 1 de restante
fim
mostre restante
```

Saída esperada:

```
10
0
```

para numero de 1 até 4 faça inclui o limite final: o total é 1 + 2 + 3 + 4. até antes de 4 excluiria 4. O laço enquanto repete enquanto a condição permanece verdadeira; ele precisa

alterar algum estado ou ter outra forma de encerramento quando a condição depende desse estado.

para cada item em colecao faça percorre os elementos de uma coleção ou de outro iterável.
repita 3 vezes repete um bloco por quantidade, sem exigir que você mantenha o contador manualmente.

Interromper e pular uma passagem

Arquivo: Exemplos/Fundamentos/01_fluxo_e_colecoes.coral.

```
para n de 0 até antes de 5 faça
    se n for igual a 1 então
        continue
    fim
    se n for igual a 3 então
        pare
    fim
    mostre n
fim
repita 2 vezes
    mostre "Coral"
fim
```

Saída esperada:

```
0
2
Coral
Coral
```

continue pula o restante da passagem atual. pare encerra o laço mais próximo. Ambos exigem um contexto de repetição. O último bloco demonstra repetição por quantidade. Não escreva pare para encerrar uma função; nesse caso, use retorne.

Pensar na terminação

Antes de executar um enquanto, responda: qual estado muda? Qual condição permite sair? O corpo pode deixar de alcançar essa mudança? Isso evita laços infinitos. Para interromper um programa de terminal que não termina, use Ctrl+C.

Exercício: some somente os números pares de 1 a 10. Uma solução pode usar % 2 para testar o resto da divisão. Verifique se o limite final está incluso.

12. Coleções, índices, fatias e compreensões

Lista: sequência mutável

Arquivo: Exemplos/Fundamentos/01_fluxo_e_colecoes.coral.

```
defina notas como [7, 8, 9]
coloque 10 em notas
mostre quantidade de notas
mostre primeiro item de notas
mostre notas[1:3]
```

Saída esperada:

```
4
7
[8, 9]
```

Uma lista preserva a ordem e pode crescer. coloque acrescenta um elemento; estenda incorpora os elementos de outra sequência. Os índices começam em zero. Na fatia [1:3], o início é incluído e o fim é excluído; a lista original não é alterada por essa consulta.

Dicionário: valores por chave

Arquivo: Exemplos/Fundamentos/01_fluxo_e_colecoes.coral.

```
defina aluno como {"nome": "Ana", "nota": 8}
defina a chave "cidade" de aluno como "Timon"
mostre valor de "nome" em aluno
mostre aluno["cidade"]
```

Saída esperada:

```
Ana
Timon
```

Uma chave identifica um valor. A expressão `aluno["nome"]` é equivalente à consulta natural mostrada. Consultar uma chave inexistente gera erro; confira a existência ou use uma operação apropriada ao seu caso. Remova com `remove a chave "nota" de aluno`.

Outras coleções

Tuplas, como `(10, 20)`, são sequências imutáveis. Uma tupla com um único item precisa da vírgula: `(10,)`. Conjuntos guardam elementos únicos, sem contrato de ordem para apresentação. `{}` representa dicionário vazio; use um `conjunto vazio` para um conjunto sem elementos.

Arquivo: Exemplos/Fundamentos/01_fluxo_e_colecoes.coral.

```
defina numeros como [1, 2, 3, 4]
defina pares como [n vezes 2 para cada n em numeros se n % 2 for igual a 0]
mostre pares
defina unicos como {1, 1, 2}
mostre quantidade de unicos
```

Saída esperada:

```
[4, 8]
2
```

A compreensão produz uma nova coleção a partir de outra. Primeiro percorre os valores, aplica o filtro e só então transforma cada valor aceito. No exemplo, apenas 2 e 4 passam pelo filtro e são dobrados.

Referências e mutabilidade

Atribuir uma lista a outro nome não cria necessariamente uma cópia independente. Os dois nomes podem apontar para o mesmo objeto. Quando a separação é importante, use uma cópia explícita, como uma fatia completa `valores[:]`, e considere que objetos aninhados continuam compartilhados numa cópia superficial.

Desempacotamento é permitido, como `defina x e y como (10, 20)`, respeitando a quantidade de valores. Evite misturar elementos de tipos incompatíveis em operações de ordenação ou aritmética.

13. Funções e chamadas naturais

Uma função dá nome a uma operação reutilizável. Parâmetros são nomes que recebem argumentos a cada chamada. `retorne` encerra a função e devolve um valor; `mostre` um resultado na tela não é a mesma coisa que `retorná` lo.

Arquivo: Exemplos/Fundamentos/02_funcoes_tipos_e_escopo.coral.

```
crie a função dobro com numero chamada como "dobro {numero}"
  retorne numero vezes 2
fim
mostre dobro 21
mostre (dobro 4) mais (dobro 5)
```

Saída esperada:

```
42
18
```

`chamada` como declara uma forma natural. Os campos entre chaves precisam corresponder aos parâmetros. A leitura natural pode participar de outras expressões; os parênteses no segundo resultado deixam a composição evidente. Não existe interpretação livre por IA.

Parâmetros opcionais e nomeados

Arquivo: Exemplos/Fundamentos/02_funcoes_tipos_e_escopo.coral.

```
crie a função saudar com nome e prefixo opcional "Olá"
  retorne prefixo mais ", " mais nome
fim
mostre saudar("Ana")
mostre saudar(nome="Bia", prefixo="Oi")
crie a função contar com vários valores
  retorne quantidade de valores
fim
mostre contar(10, 20, 30)
```

Saída esperada:

```
Olá, Ana
Oi, Bia
3
```

Um valor padrão permite omitir o argumento. Argumentos nomeados associam explicitamente o nome ao valor. `vários valores` aceita uma quantidade variável de argumentos posicionais. A forma `várias opcoes` nomeadas aceita argumentos nomeados variáveis.

Use `execute funcao(...)` quando deseja chamar uma função apenas por seu efeito. Use a chamada dentro de `defina` ou `mostre` quando deseja consumir o retorno. Uma expressão de chamada solta não é uma instrução universalmente aceita na superfície Coral.

Escopo e funções internas

Arquivo: Exemplos/Fundamentos/02_funcoes_tipos_e_escopo.coral.

```

crie a função criar_contador
  defina valor como 0
  crie a função proximo
    use valor do escopo externo
    adicione 1 a valor
    retorne valor
  fim
  retorne proximo
fim
defina contar como criar_contador()
mostre contar()
mostre contar()

```

Saída esperada:

```

1
2

```

O estado `valor` pertence à função externa e continua acessível pela função retornada. `use valor do escopo externo` permite alterar essa ligação. Sem a declaração adequada, uma atribuição interna pode criar outra variável ou produzir um erro de escopo.

As formas declarativas de chamada natural são registradas durante a análise da fonte. Para módulos separados, a forma mais previsível de usar uma função importada é a chamada convencional; não presuma que importar um módulo publique automaticamente todos os seus padrões naturais no arquivo chamador.

Exercício: crie `triplo` com a leitura `triplique {numero}`. Teste com zero, um número negativo e uma expressão entre parênteses.

14. Geradores, tipagem opcional e abstrações

Produzir valores sob demanda

Arquivo: `Exemplos/Fundamentos/04_geradores_e_assincrono.coral`.

```

crie a função numeros com limite
  para n de 0 até antes de limite faça
    produza n
  fim
fim
para cada n em numeros(3) faça
  mostre n
fim

```

Saída esperada:

```

0
1
2

```

`produza` suspende a execução de uma função geradora e entrega um valor por vez. Na próxima solicitação, a função continua de onde parou. Isso permite percorrer uma sequência sem construir todos os valores antecipadamente. `produza de outra_colecao` delega a produção a outro iterável.

Tipagem opcional

Arquivo: Exemplos/Fundamentos/02_funcoes_tipos_e_escopo.coral.

```
defina idade do tipo inteiro como 19
crie a função dobro com numero do tipo inteiro retornando inteiro
    retorne numero vezes 2
fim
mostre dobro(idade)
```

Saída esperada:

```
38
```

As anotações são metadados de intenção. A Coral continua dinamicamente tipada; uma anotação não converte automaticamente os dados nem garante a rejeição estática de toda incompatibilidade. Use conversão explícita quando o dado chega como texto.

Tipos reconhecidos nas anotações incluem inteiro, decimal, texto, booleano, lista, dicionário, conjunto, tupla, qualquer e nomes de classes. Os tipos concretos e as operações seguem os objetos usados em execução.

Funções como valores

Uma função pode ser atribuída a uma variável, recebida como argumento ou retornada. Uma função anônima pode ser escrita como função anônima com `x` que retorna `x` vezes 2. Isso é uma expressão; atribua o resultado a um nome antes de usar, quando precisar de uma operação identificável.

Decoradores são declarados com `decore` com decorador antes da função. Eles exigem um objeto chamável previamente definido e são um recurso avançado de interoperabilidade. Prefira funções comuns até existir uma razão concreta para modificar o comportamento de outras funções.

Não existe uma promessa de cobertura estática equivalente a linguagens com verificação obrigatória de tipos. Testes e validação de entradas continuam necessários.

15. Classes, propriedades, herança e protocolos

Objetos combinam estado e operações. Uma classe define como criar e manipular esses objetos. Cada instância pode manter valores próprios.

Arquivo: Exemplos/Fundamentos/03_objetos_e_erros.coral.

```
crie a classe Contador
    ao criar uma Contador com inicial
        defina seu valor como inicial
    fim
    método somar com quantidade
        adicione quantidade a seu valor
    fim
    propriedade atual
        retorne seu valor
    fim
fim
crie contador como uma Contador com 10
execute contador.somar(5)
mostre contador.atual
```

Saída esperada:

```
15
```

O construtor ao criar recebe o estado inicial. seu valor acessa o atributo da instância atual. O método altera esse atributo; a propriedade oferece leitura sem exigir uma chamada com parênteses.

Herdar comportamento

Arquivo: Exemplos/Fundamentos/03_objetos_e_erros.coral.

```

crie a classe Pessoa
  ao criar uma Pessoa com nome
    defina seu nome como nome
  fim
fim
crie a classe Aluno herda de Pessoa
  ao criar uma Aluno com nome e nota
    inicialize a classe pai com nome
    defina sua nota como nota
  fim
fim
crie aluno como uma Aluno com "Ana" e 9
mostre aluno.nome
mostre aluno.nota

```

Saída esperada:

```

Ana
9

```

Aluno reutiliza a estrutura de Pessoa. O comando `inicialize a classe pai` chama a inicialização herdada, e o construtor acrescenta a nota. Herança é útil quando a relação de especialização é real; para apenas guardar ou usar outro objeto, composição costuma ser mais direta.

Referência de membros

Construção	Papel
método nome com parametros	Operação de instância; self é implícito
método estático nome	Operação sem instância implícita
método de classe nome	Operação que recebe a classe como cls
propriedade nome	Getter acessado como atributo
ao definir a propriedade nome com valor	Setter
ao converter para texto	Representação textual do objeto
ao obter tamanho	Protocolo de tamanho
ao iterar e ao obter próximo	Protocolo de iteração

Esses cabeçalhos são trechos de referência: devem aparecer dentro de uma classe, com corpo e fim próprios. As formas `seu`, `sua`, `seus` e `suas` referem-se a atributos do objeto atual. O nome do atributo precisa coincidir exatamente com o que foi definido.

16. Erros, padrões e contextos

Tratar falhas esperadas

Arquivo: Exemplos/Fundamentos/03_objetos_e_erros.coral.

```
tente
  defina resultado como 10 dividido por 0
se der erro do tipo ZeroDivisionError
  mostre "Não é possível dividir por zero"
finalmente
  mostre "Operação encerrada"
fim
```

Saída esperada:

```
Não é possível dividir por zero
Operação encerrada
```

se der erro do tipo seleciona a exceção a tratar. finalmente é executado ao sair do bloco, inclusive quando houve erro. Uma captura ampla pode esconder falhas que você deveria corrigir; prefira capturar o tipo que seu programa realmente espera.

Uma exceção própria pode ser declarada com `crie a exceção Nome`, corpo e `fim`. Use `lance um erro do tipo Nome com "mensagem"` para lançá-la. garanta que condicao com mensagem `"..."` expressa uma verificação, frequentemente usada em testes.

Selecionar por padrão

Arquivo: Exemplos/Fundamentos/01_fluxo_e_colecoes.coral.

```
defina comando como "iniciar"
combinar comando
  caso "iniciar"
    mostre "Começando"
  caso "sair" ou "parar"
    mostre "Encerrando"
  caso padrão
    mostre "Comando desconhecido"
fim
```

Saída esperada:

```
Começando
```

`combinar` avalia casos em ordem. Há suporte a literais, alternativas, capturas, sequências, mapeamentos e outros padrões estruturais. O caso padrão cobre o que não foi reconhecido. Não confunda uma captura de nome com uma comparação contra uma variável: padrões possuem regras próprias.

Liberar recursos

Arquivo: Exemplos/Dados/02_arquivos_json_e_caminhos.coral.

```
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "Coral")
com arquivo "dados.txt" aberto para leitura como arquivo faça
  defina conteudo como arquivo.read()
fim
mostre conteudo
```

Saída esperada:

 Coral

O gerenciador de contexto fecha o arquivo ao sair do bloco. A leitura por `arquivo.read()` é uma chamada avançada do objeto de arquivo. Para operações simples, a biblioteca padrão de arquivos geralmente é suficiente.

Distingua erros de parsing, que impedem compreender o código, de erros de execução, como divisão por zero ou arquivo ausente. `coral verificar` não executa o programa e, por isso, não comprova que um caminho existe ou que uma operação é válida para todos os dados.

17. Programação assíncrona

Programação assíncrona permite suspender uma operação enquanto outra pode avançar no mesmo ciclo de eventos. Ela não torna automaticamente todo cálculo paralelo nem elimina o custo de operações bloqueantes.

Arquivo: Exemplos/Fundamentos/04_geradores_e_assincrono.coral.

```
 crie a função assíncrona dobro com numero
    espere 0 segundos
    retorne numero vezes 2
fim
crie a função assíncrona principal
    defina resultado como aguarde dobro(21)
    mostre resultado
fim
execute assincronamente principal()
```

Saída esperada:

 42

A função `dobro` é assíncrona. `aguarde` obtém seu resultado dentro de outra função assíncrona. `execute assincronamente` inicia a execução no nível principal. `espere 0 segundos` cede ao ciclo de eventos sem introduzir uma demora perceptível no exemplo.

Regras de contexto

`aguarde`, laços assíncronos e contextos assíncronos exigem função ou método assíncrono. Não copie apenas uma linha `aguarde` para o topo de um arquivo e espere que funcione. O exemplo inclui a estrutura necessária.

`crie tarefa` como uma tarefa para `operacao()` registra uma tarefa dentro de um contexto com ciclo de eventos ativo. Depois, `aguarde tarefa` obtém o resultado. Laços podem usar para cada item assincronamente em `fonte` faça; a fonte precisa implementar um iterador assíncrono.

Classes avançadas podem implementar `ao iterar assincronamente`, `ao obter próximo assincronamente`, `ao entrar assincronamente` e `ao sair assincronamente`. Esses nomes não transformam um objeto comum em assíncrono sem um comportamento correspondente.

Para programas iniciais de terminal, prefira fluxo síncrono. Use o modelo assíncrono quando houver uma necessidade concreta de coordenar espera e atividades, e teste tratamento de erro e encerramento das tarefas.

18. Espaço, geometria, movimento e consultas espaciais

A Coral 1.5.5 trata posição, distância, direção, formas, movimento e relações espaciais como conceitos gerais da linguagem. Eles pertencem à baseline e estão disponíveis **sem importação**. Não existe um módulo público separado para habilitar esses recursos. Jogos é apenas um consumidor 2D dessa infraestrutura.

Posições e formas em 2D

```
defina origem como posição(0, 0)
defina destino como posição(3, 4)
defina distancia_total como distância entre origem e destino
defina area_util como retângulo(0, 0, 100, 60)
mostre distancia_total
mostre area_util
```

Exemplo completo: Livro/Exemplos/Espaco/01_espaco_2d_e_3d.coral.

A mesma baseline em 3D

Adicionar uma terceira coordenada cria um valor tridimensional real. A dimensão faz parte do contrato do valor; uma posição 2D não ganha $z = 0$ implicitamente.

```
defina origem como posição(0, 0, 0)
defina destino como posição(2, 3, 6)
defina volume como caixa(0, 0, 0, 20, 30, 40)
defina distancia_total como distância entre origem e destino
mostre distancia_total
```

Exemplo completo: Livro/Exemplos/Espaco/01_espaco_2d_e_3d.coral.

Relações geométricas

As relações são matemáticas e independentes de jogos. **intersecta** aceita contato de borda; **sobrepõe** exige interior comum; **toca** descreve contato sem sobreposição interior. Formas e regiões podem ser usadas em simulações, dados científicos, interfaces, mapas abstratos ou qualquer outro domínio que precise raciocinar espacialmente.

Consultas espaciais sobre coleções comuns

Uma consulta não exige cena, mundo nem classe base especial. A coleção pode conter valores espaciais diretamente ou objetos dos quais a Coral consiga obter posição ou forma.

```
defina sensores como [posição(1, 1), posição(4, 4), posição(20, 0)]
defina proximos como procure em sensores os que estão a até 6 de posição(0, 0)
defina mais_perto como procure em sensores o mais próximo de posição(3, 3)
mostre proximos
mostre mais_perto
```

Exemplo completo: Livro/Exemplos/Espaco/02_consultas_e_movimento.coral. O mesmo arquivo mostra a consulta em 2D e 3D; a dimensionalidade vem dos dados.

Movimento também é geral

Velocidade, aceleração, integração por dt e limites não pertencem a `coral.jogos`. Isso permite modelar sondas, partículas, robôs, corpos simulados ou trajetórias sem abrir uma janela.

```
defina posicao_inicial como posicao(0, 0, 200)
defina velocidade_inicial como velocidade(12, 4, menos 2)
defina gravidade como aceleracao(0, 0, menos 9.8)
defina estado como integrar_movimento(posicao_inicial, velocidade_inicial, gravidade,
  dt=0.5)
mostre estado.posicao
```

Exemplo completo: Livro/Exemplos/Espaco/02_consultas_e_movimento.coral.

Relação com coral.jogos

`coral.jogos` permanece 2D nesta versão. Sprites e colisores reutilizam as formas e relações gerais, mas uma forma geométrica não é sinônimo de colisor. Isso permite que a mesma matemática seja útil dentro e fora de jogos sem transformar o restante da linguagem em uma API de game engine.

19. Mundos, mapas, células, camadas e regiões

A Coral 1.5.7 introduz uma infraestrutura discreta de mapas em `coral.mundo`. Ela não é uma engine de jogos: um mapa organiza **valores no espaço**, permite consultar relações locais e comunica mudanças usando a infraestrutura reativa já existente. O mesmo contrato serve para simulações, ciência, robótica, logística, autômatos celulares e, opcionalmente, jogos.

A ideia central é separar cinco coisas que costumam ser confundidas:

- uma **posição** é uma localização contínua no espaço, como (2.4, 7.1);
- uma **célula** é uma localização discreta inteira, como (2, 7);
- uma **camada** diz que tipo de informação está sendo armazenada;
- o **valor** é o dado guardado naquela célula e camada;
- uma **região** agrupa células ou associa uma forma espacial a uma parte do mapa.

Célula e posição

Células usam coordenadas inteiras. Uma topologia quadrada usa coluna e linha; uma topologia hexagonal usa q e r; e um mapa voxel usa x, y e z. Posições contínuas continuam pertencendo à baseline espacial apresentada no capítulo anterior.

```
crie um mapa chamado clima com topologia quadrada
adicione uma camada chamada temperatura ao mapa clima
defina 31.5 na célula (4, 2) da camada temperatura do mapa clima
defina leitura como o valor da célula (4, 2) da camada temperatura do mapa clima
garanta que leitura for igual a 31.5
```

Ausência não significa nulo, zero, falso ou texto vazio. Uma célula pode simplesmente não possuir conteúdo em determinada camada.

Topologia

A topologia define a dimensionalidade, a forma de vizinhança e a conversão entre célula e coordenada de mundo.

```
crie um mapa chamado colmeia com topologia hexagonal
defina vizinhas como as células vizinhas de (2, menos 1) no mapa colmeia
garanta que quantidade de vizinhas for igual a 6

crie um mapa chamado volume com topologia voxel
adicione uma camada chamada densidade ao mapa volume
defina 0.82 na célula (3, 4, 5) da camada densidade do mapa volume
defina amostra como o valor da célula (3, 4, 5) da camada densidade do mapa volume
garanta que amostra for igual a 0.82
```

A topologia quadrada possui quatro vizinhos ortogonais por padrão. A hexagonal possui seis vizinhos axiais. A voxel possui seis vizinhos ortogonais em três dimensões. Mapas podem ser finitos ou não limitados; uma célula ausente em um mapa não limitado continua sendo uma coordenada válida, apenas sem conteúdo.

Camada e valor

Camadas são nomeadas, ordenadas e esparsas. Elas podem armazenar números, textos, booleanos, listas, dicionários e outros valores portáteis reconhecidos pela persistência Coral. BlocoMapa existe como conveniência para terreno ou material, mas não é a unidade obrigatória do sistema.

```
crie um mapa chamado laboratorio com topologia quadrada
adicione uma camada chamada temperatura ao mapa laboratorio
adicione uma camada chamada umidade ao mapa laboratorio
defina 22.0 na célula (1, 0) da camada temperatura do mapa laboratorio
defina 0.61 na célula (1, 0) da camada umidade do mapa laboratorio
defina t como o valor da célula (1, 0) da camada temperatura do mapa laboratorio
defina u como o valor da célula (1, 0) da camada umidade do mapa laboratorio
garanta que t for igual a 22.0
garanta que u for igual a 0.61
```

Vizinhança

Vizinhança pertence à topologia, não ao domínio do programa. Isso permite que o mesmo comando seja usado por um autômato celular, um robô ou um jogo.

```
crie um mapa chamado grade com topologia quadrada
defina proximas como as células vizinhas de (4, 2) no mapa grade
garanta que quantidade de proximas for igual a 4
```

Consultas de mapa são expressões e podem participar de cálculos e comparações. Por exemplo, soma mais o valor da célula `celula` da camada `temperatura` do mapa `laboratorio` permanece uma única expressão estrutural da linguagem.

Região

Uma região pode ser formada por células explícitas ou, pela API avançada, por uma forma espacial serializável. Regiões discretas servem para setores de armazém, biomas, zonas de segurança, territórios ou qualquer agrupamento irregular.

```
crie um mapa chamado clima com topologia quadrada
adicione uma camada chamada temperatura ao mapa clima
crie uma região chamada zona_quente no mapa clima com as células [(4, 2), (5, 2), (5, 3)]
defina presentes como as regiões da célula (5, 2) no mapa clima
garanta que quantidade de presentes for igual a 1
```

Em mapas não limitados, uma região definida por forma não precisa materializar uma quantidade infinita de células. Quando uma enumeração puder ser infinita, a API exige limites explícitos.

Travessia

Travessia é opcional e genérica. Um conteúdo pode permitir ou bloquear passagem, declarar custo não negativo e registrar um motivo. A 1.5.7 fornece adjacência e custos, mas não introduz um algoritmo oficial de caminhos.

```
crie um mapa chamado deposito com topologia quadrada
adicione uma camada chamada ocupacao ao mapa deposito
coloque o bloco "estante" na célula (8, 3) da camada ocupacao do mapa deposito
bloquee a travessia pela célula (8, 3) da camada ocupacao do mapa deposito porque "corredor
ocupado"
```

Uma consulta pura de travessia não publica eventos. Uma tentativa explícita pode publicar `mapa.travessia_bloqueada` no barramento já utilizado pelo mundo.

Eventos de mapa

Mapas associados a Mundo usam **o mesmo barramento** `mundo.regras.eventos`; não existe um motor de regras oculto por mapa. Um mapa independente também funciona sem barramento quando o programa não precisa de reatividade.

```
crie um mapa chamado clima com topologia quadrada
adicione uma camada chamada temperatura ao mapa clima
defina ultima como 0

quando o valor da célula (4, 2) da camada temperatura do mapa clima mudar como evento
  defina ultima como evento.valor_novo
fim

defina 31.5 na célula (4, 2) da camada temperatura do mapa clima
garanta que ultima for igual a 31.5
```

Também é possível observar entrada e saída de regiões. O resolvidor de localização é relido a cada avaliação, portanto mover o alvo não congela a posição inicial.

Persistência

Mapa participa do envelope `coral.persistencia/1` com o identificador `coral.mundo/Mapa`. O formato recomendado para um mapa isolado é `.coralmapa`. Topologia, camadas, conteúdos portáteis e regiões são preservados; Pygame, superfícies, callbacks e recursos gráficos abertos não fazem parte do arquivo.

Na API avançada, a persistência genérica já é suficiente:

```
from coral.persistencia import carregar, salvar

salvar(mapa, "clima.coralmapa")
restaurado = carregar("clima.coralmapa")
```

Mundo também persiste seus mapas nomeados. Documentos de versões anteriores que não possuem o campo `mapas` continuam sendo carregados com zero mapas.

Visualização opcional com Jogos

`coral.jogos` não define o significado de um mapa. Ele oferece apenas conversões de tela e um adaptador 2D para mapas quadrados e hexagonais. Mapas voxel continuam válidos no núcleo, mas não recebem uma projeção 3D implícita.

```
de coral.jogos importe Camera e RenderizadorMapa2D e BackendNulo

crie um mapa chamado sala com topologia quadrada
adicione uma camada chamada terreno ao mapa sala
coloque o bloco "parede" na célula (1, 1) da camada terreno do mapa sala

defina camera como Camera(0, 0, 64, 64)
defina backend como BackendNulo()
defina renderizador como RenderizadorMapa2D(sala, camera=camera)
defina desenhadas como renderizador.desenhar(backend)
garanta que desenhadas for igual a 1
```

O conjunto de exemplos oficiais inclui uma grade de temperatura, um autômato celular e um mapa de ocupação robótica justamente para provar que visualização é um consumidor opcional e não a fundação da API.

PARTE III: PROJETOS, BIBLIOTECAS E RECURSOS DA LINGUAGEM

Projetos, biblioteca padrão, Regras Coral, mundos, RPG, jogos, ferramentas de estudo e a linha científica e de hardware introduzida na Coral 1.3 e consolidada nesta edição.

20. Projetos, módulos, coral.toml e testes

Criar a estrutura

```
coral novo MinhaFerramenta --template basico
cd MinhaFerramenta
coral executar
coral testar
```

`coral.toml` identifica a raiz do projeto. Ele pode definir nome, versão, arquivo de entrada, requisito de versão Coral, caminhos de módulos e caminhos de testes.

```
[projeto]
nome = "minha-ferramenta"
versao = "0.1.0"
[coral]
versao = ">=1.3"
[entrada]
arquivo = "principal.coral"
[modulos]
caminhos = ["."]
[testes]
caminhos = ["testes"]
```

Caminhos são relativos à raiz configurada. `coral executar` compila os módulos para `.coral_build` e executa a entrada. Essa pasta contém saída regenerável. Não a use como local dos seus arquivos fonte.

Importar funções locais

Um arquivo `calculos.coral` pode declarar uma função `media`. Em `principal.coral`, use de `calculos importe media` e depois `media(notas)`. Nomeie seus módulos de forma clara e evite colidir com `coral`, `json`, `sys` e outros módulos do ambiente.

Na 1.5.8, a interface pública do módulo também carrega formas naturais exportadas. Uma função declarada com uma forma como `"dobre {numero}"` pode ser usada pelo consumidor após uma importação seletiva. Alias de importação preserva a forma adaptada, enquanto duas formas externas ambíguas produzem diagnóstico em vez de uma escolha silenciosa.

O build guarda hashes de fonte e de interface. Sem mudança, módulos já válidos são reutilizados. Uma mudança apenas de implementação recompila o módulo afetado; uma mudança de interface também invalida os consumidores. O resultado de um build incremental deve ser equivalente ao de um build limpo.

Recursos de projeto precisam ser declarados no `coral.toml`. O build copia somente os recursos declarados e o runtime resolve seus caminhos a partir do pacote construído, de modo que a pasta de build possa ser movida sem perder o vínculo com esses arquivos.

A execução de um arquivo isolado não equivale à construção de um projeto: para módulos locais Coral, prefira `coral executar` na raiz do projeto. O comando `coral arquivo.coral` é apropriado a programas independentes.

Testes nativos

Desde a linha 1.2, um bloco teste "nome" contém verificações e termina com `fim`. Execute com `coral testar`; simplesmente executar um arquivo de testes como programa não é o mesmo fluxo. Na 1.5.8, o runner prepara o projeto uma vez por sessão e reutiliza esse build entre arquivos de teste.

O bloco de teste pode declarar casos parametrizados, erro esperado, motivo de pulo, semente, limite de duração, grupo e recursos temporários. Hooks explícitos de preparação e encerramento continuam separados do teste básico, de modo que um caso simples não precisa carregar toda a infraestrutura avançada.

O conjunto Exemplos/Projetos_Completos/Notas/ traz um projeto completo com `calculos.coral`, `principal.coral` e `testes/basico.coral`. Copie essa pasta para seu diretório de trabalho e execute:

```
coral executar
coral testar
coral dependencias
coral documentar
```

A execução imprime 8.0 e o teste da média deve passar. `coral documentar` gera uma referência de símbolos; isso não substitui o manual de uso nem inventa explicações das regras de negócio.

O runner isola os módulos locais durante o carregamento de cada arquivo de testes. Testes no mesmo arquivo ainda podem compartilhar objetos importados. Evite depender da ordem de execução.

21. Arquivos, JSON e biblioteca padrão

Gravar e ler dados

Arquivo: Exemplos/Dados/02_arquivos_json_e_caminhos.coral.

```
de coral.arquivos importe escrever_texto e ler_texto
execute escrever_texto("mensagem.txt", "Olá, arquivo!")
defina conteudo como ler_texto("mensagem.txt")
mostre conteudo
```

Saída esperada:

```
Olá, arquivo!
```

O caminho relativo é resolvido a partir da pasta de trabalho, não necessariamente da pasta em que o arquivo `.coral` está salvo. Se você executa de outra pasta, um caminho como `mensagem.txt` aponta a essa outra pasta. Para um projeto, comece pelo terminal na raiz e organize dados em diretórios conhecidos.

Serializar estruturas

Arquivo: Exemplos/Dados/02_arquivos_json_e_caminhos.coral.

```
de coral.json importe para_json e de_json
defina original como {"nome": "Ana", "nota": 9}
defina documento como para_json(original)
defina recuperado como de_json(documento)
mostre recuperado["nome"]
mostre recuperado["nota"]
```

Saída esperada:

```
Ana
9
```

JSON preserva estruturas básicas como objetos, listas, textos, números, booleanos e ausência. Ele não serializa automaticamente qualquer classe personalizada. `de_json` interpreta dados JSON; não executa código Coral.

Importações explícitas

Arquivo: Exemplos/Dados/05_utilitarios_e_interoperabilidade.coral.

```
de coral.matematica importe media
de coral.texto importe maiusculas
mostre media([7, 8, 9])
mostre maiusculas("Coral")
```

Saída esperada:

```
8.0
CORAL
```

Use o namespace `coral.<módulo>` para indicar que deseja a biblioteca padrão Coral. O backend direciona esses nomes para os módulos distribuídos. Imports curtos, como `de matematica importe media`, existem por conveniência, mas podem ser substituídos por um módulo local com o mesmo nome.

A tabela de referência adiante registra as funções públicas da biblioteca padrão. Os nomes de argumentos são os da API Python exposta à Coral; chamadas convencionais são adequadas quando ainda não há uma frase natural específica para a operação.

Exercício: grave um dicionário com nome e nota em JSON, leia novamente e confira os valores. Use uma pasta temporária durante os testes para não sobrescrever um documento seu.

22. Regras Coral, eventos e temporizadores

A ideia principal da programação reativa em Coral é simples: **algo acontece e uma regra reage**. O programa não precisa transformar cada comportamento em um loop manual nem em callbacks de baixo nível. A superfície natural descreve o acontecimento, a condição opcional e a reação; o runtime mantém ordem determinística, tempo lógico e rastreamento.

O exemplo canônico mais curto está em

Livro/Exemplos/Regras/01_eventos_observacao_e_tempo.coral:

```

defina ultimo_dano como 0
quando ocorrer "dano" como evento se evento.quantidade for maior que 3
    defina ultimo_dano como evento.quantidade
    mostre evento.origem
fim

emita o evento "dano" com quantidade 5 e origem "espada"

```

O objeto evento expõe os dados nomeados do acontecimento. Nesse exemplo, quantidade é carga útil e origem identifica quem publicou o evento. A emissão é local e determinística: ela não cria uma thread, não inicia uma tarefa em segundo plano e não transforma o programa em um sistema distribuído.

Observar mudanças sem inventar eventos manuais

Quando a intenção é reagir a um valor que mudou, diga isso diretamente:

```

defina vida como 10
quando vida mudar
    mostre vida
fim

avale as regras
defina vida como 9
avale as regras

```

Também existem cruzamentos de limite:

```

quando temperatura passar de 40
    mostre "Temperatura alta."
fim

quando temperatura cair abaixo de 30
    mostre "Temperatura baixa."
fim

```

O exemplo executável é `Livro/Exemplos/Regras/01_eventos_observacao_e_tempo.coral`. Uma atribuição que mantém o mesmo valor não conta como mudança. `passar de` e `cair abaixo de` reagem ao cruzamento da fronteira, não a toda avaliação posterior que permaneça do mesmo lado.

Prioridade, cancelamento e consumo

Prioridades nomeadas evitam números mágicos: baixa, normal, alta e crítica. Reações de maior prioridade são consideradas primeiro; empates preservam a ordem de registro.

```

quando ocorrer "rpg.dano.antes" como evento com prioridade crítica
    cancele evento
fim

```

Cancelar só tem efeito sobre acontecimentos declarados como canceláveis e deve ocorrer **antes** do efeito. A Coral não tenta desfazer uma alteração já aplicada. O RPG usa esse contrato em `rpg.dano.antes`: se o evento for cancelado, a vida não é modificada. Veja `Livro/Exemplos/Regras/02_prioridade_e_cancelamento.coral`.

`consume evento` tem outro significado: impede que o acontecimento continue sua propagação para níveis superiores, mas não representa por si só o cancelamento da operação de domínio.

Limites, cooldown e debounce

Uma reação pode executar somente uma vez ou um número máximo de vezes:

```
uma vez quando ocorrer "visita"
  mostre "Primeira visita."
fim
```

Cooldown limita a frequência de um evento:

```
quando ocorrer "ataque" no máximo uma vez a cada 2 segundos
  mostre "Ataque aceito."
fim
```

Debounce é adequado quando a ação deve ocorrer somente depois de um período sem novas mudanças:

```
quando texto mudar espere 300 milissegundos sem novas mudanças faça
  mostre texto
fim
```

Essas formas usam o tempo lógico do motor. Não usam sleep escondido.

Tempo determinístico e relógio de teste

Temporizadores continuam disponíveis com `depois de` e `a cada`. Em testes, o relógio pode ser avançado instantaneamente:

```
defina pulsos como 0
a cada 2 segundos faça
  adicione 1 a pulsos
fim

avance o relógio de teste em 5 segundos
mostre pulsos
```

O exemplo `Livro/Exemplos/Regras/01_eventos_observacao_e_tempo.coral` valida tempo sem esperar cinco segundos reais. Isso mantém testes rápidos e reproduzíveis.

Grupos e regras temporárias

Regras nomeadas podem ser reunidas em grupos:

```
grupo tempestade faça
  use chuva_venenosa
  use raios
fim

desative grupo tempestade
ative grupo tempestade
```

Uma regra desativada também pode ser ligada por duração ou até uma condição:

```
ative alerta durante 30 segundos
ative alerta até temperatura for menor que 30
```

A duração é cooperativa: o motor aplica a expiração quando seu tempo lógico avança.

Sequências de dois acontecimentos

A 1.5.6 reconhece sequências deliberadamente simples $A \rightarrow B$:

```
quando ocorrer "botao_a" e depois ocorrer "botao_b" dentro de 2 segundos
  mostre "Combinação reconhecida."
fim
```

A janela temporal é opcional. Esta edição não transforma a linguagem em um processador geral de eventos complexos; sequências arbitrariamente longas permanecem fora do escopo.

Eventos espaciais em 2D e 3D

Os observadores espaciais reutilizam a baseline geral da linguagem:

```
defina jogador como posição(-1, 5)
defina floresta como região(retângulo(0, 0, 10, 10))

quando jogador entrar em floresta
  mostre "Entrou na floresta."
fim
```

Também há sair de, proximidade, toque e interseção. As mesmas ideias funcionam em 3D; misturar 2D e 3D continua exigindo conversão explícita. O exemplo canônico é `Livro/Exemplos/Regras/03_eventos_espaciais.coral`.

Estados observáveis

Para uma pequena máquina de estados, declare a intenção sem construir uma classe auxiliar:

```
estado porta começa como fechada

quando ocorrer "abrir_porta"
  mude porta para aberta
fim
```

Esse recurso é um estado observável leve. Ele não é um statechart hierárquico e não cria estados paralelos implicitamente.

Histórico e explicação

O comando:

```
mostre histórico de eventos
```

ativa rastreamento detalhado desde o início do programa e apresenta a cadeia de decisões reativas. O runtime registra o resultado de uma condição quando ela é avaliada; o modo Ensinar usa esse registro para explicar por que uma regra executou ou foi suprimida **sem reavaliar a condição** e sem repetir efeitos colaterais.

Motores e avaliação

avalie as regras processa condições e observadores no motor padrão. atualize as regras por ... segundos também avança tempo lógico, timers, cooldowns, debounce e ativações temporárias. Jogos podem atualizar seu próprio motor durante o ciclo de quadros, enquanto Mundo e RPG publicam acontecimentos de domínio no mesmo modelo de eventos.

Quando nenhum motor específico é informado, a superfície natural usa o motor padrão. Formas históricas como quando ocorrer "moeda" recebendo valor continuam válidas por compatibilidade.

API avançada

A API orientada a objetos permanece em `coral.regras`, `coral.tempo_eventos` e módulos auxiliares para ferramentas, integrações e casos que a superfície natural não cobre. Código introdutório deve preferir as frases deste capítulo. regra, eventos, observadores, sequências, grupos e controles reativos são nós do parser e da AST Coral; não dependem de substituição textual frágil nem de interpretação por IA.

23. coral.mundo: entidades, relações e simulações

`coral.mundo` é a API oficial da Coral 1.2 para mundos, entidades, atributos, inventários, relações e pequenos modelos de simulação. Internamente, relações formam um grafo dirigido, mas a superfície recomendada é escrita como Coral corrente.

Forma recomendada

```
crie um mundo chamado campanha com nome "Minha Campanha"
crie uma entidade chamada ana com nome "Ana" no mundo campanha
crie uma entidade chamada castelo com nome "Castelo" no mundo campanha
relacione ana com castelo como "viaja para" no mundo campanha
encontre um caminho de ana até castelo no mundo campanha como rota
```

A intenção é que a leitura revele o modelo sem exigir que o iniciante conheça construtores, métodos ou a implementação em grafos.

Entidades

Entidades podem guardar atributos, tags e inventário. A API de baixo nível permanece disponível para código avançado, interoperabilidade e extensão da biblioteca.

Relações e caminhos

`relacione` cria uma relação dirigida. `encontre um caminho` usa a busca de caminho fornecida por Mundo, atualmente baseada em busca em largura para caminhos não ponderados.

Eventos e regras

Cada mundo possui um `MotorRegras`, portanto pode hospedar regras e eventos próprios:

```
regra "perigo" em campanha.regras
    quando vida for menor ou igual a 3
    então
        mostre "Perigo!"
fim
```

API orientada a objetos

Formas como `Mundo(...)`, `mundo.criar_entidade(...)` e `mundo.relacionar(...)` continuam válidas. Elas são consideradas superfície avançada e não a forma principal dos templates e tutoriais oficiais quando existe frase Coral equivalente.

Precisões preservadas na linha 1.2

As relações são dirigidas, mas `Mundo.caminho` e a frase natural de busca consideram ambas as direções por padrão (`bidirecional=True`). A API avançada aceita `bidirecional=falso` numa expressão Coral para seguir somente saídas. A implementação atual rejeita entidades externas que tenham apenas o mesmo nome de uma entidade local. `para_json` exporta o estado genérico do mundo; não é um formato completo de persistência de personagens e combates.

Um mundo reúne entidades e relações. Entidades têm identidade própria: dois objetos com o mesmo nome não são intercambiáveis. A biblioteca recusa relações com objetos externos que apenas repetem o nome de uma entidade registrada.

Arquivo: `Exemplos/Mundo/01_mundo_e_rpg.coral`.

```
crie um mundo chamado campanha com nome "Campanha"
crie uma entidade chamada vila no mundo campanha
crie uma entidade chamada bosque no mundo campanha
relacione vila com bosque como "estrada" no mundo campanha
encontre um caminho de bosque até vila no mundo campanha como rota
mostre quantidade de rota
```

Saída esperada:

```
2
```

A relação criada é dirigida, mas a busca padrão considera ambas as direções. Por isso, o exemplo encontra um caminho de bosque para vila. A API avançada `campanha.caminho(bosque, vila, bidirecional=falso)` permite seguir apenas a direção de saída. A busca usa BFS para caminhos não ponderados.

Personagens e turnos

Arquivo: `Exemplos/Mundo/01_mundo_e_rpg.coral`.

```
crie um mundo chamado campanha com nome "Campanha"
crie um personagem chamado heroína no mundo campanha com 12 de vida
crie um personagem chamado monstro no mundo campanha com 6 de vida
role um dado de 6 lados usando a semente 7 como ataque
cause ataque.total de dano a monstro
cure 1 de vida de monstro
mostre monstro.vida
crie um combate chamado batalha com heroína e monstro
inicie o combate batalha usando a semente 4
mostre batalha.rodada
```

Saída esperada:

```
4
1
```

O dano reduz vida sem deixá-la negativa; a cura respeita o limite máximo. A semente torna a rolagem reproduzível para o exemplo. O combate organiza iniciativa e turnos, sem fornecer um sistema completo de regras de RPG. Use `avance o combate batalha` para avançar ao participante seguinte.

Referência dos componentes

Componente	Elementos principais
<code>coral.mundo</code>	Mundo, Entidade, Relacao, <code>criar_mundo</code>
Entidade	atributos, tags, inventário, obter, definir, guardar
Mundo	<code>criar_entidade</code> , relacionar, relacionados, caminho, atualizar
<code>coral.rpg</code>	Personagem, Rolagem, CombateTurnos, rolar
Rolagem	expressão, resultados individuais, modificador e total
Combate	participantes, ordem, índice, rodada e participante atual

`Mundo.para_json` exporta o estado genérico do mundo. Não é um salvamento integral de campanha: vida específica de personagem, estado de combate e lógica de retorno precisam de modelagem própria. Não prometa persistência completa usando somente essa função.

24. coral.rpg: dados, personagens e combate por turnos

`coral.rpg` é uma camada pequena construída sobre `coral.mundo`. O objetivo é oferecer peças úteis para aventuras, protótipos e demonstrações, sem tentar substituir sistemas completos de RPG.

Personagens

```
crie um personagem chamado heroína com nome "Coralina" no mundo campanha com 12 de vida
crie um personagem chamado slime com nome "Slime de Musgo" no mundo campanha com 6 de vida
defina o atributo "força" de heroína como 2
```

Dados

```
role um dado de 20 lados como ataque
role 3 dados de 6 lados como dano
role um dado de 6 lados usando a semente 7 como dado
```

Sementes permitem exemplos e testes reproduzíveis.

Dano e cura

```
cause 3 de dano a slime
cause dado.total de dano a slime como dano_causado
cure 2 de vida de heroína
```

Combate por turnos

```
crie um combate chamado batalha com heroína e slime
inicie o combate batalha usando a semente 4
avance o combate batalha
```

A camada fornece rolagens, personagens, vida, atributos, testes e ordem de turnos. O modelo de mundo, relações e eventos continua vindo de `coral.mundo`.

API avançada

Classes e funções como `Personagem`, `CombateTurnos` e `rolar(...)` permanecem disponíveis para extensão e interoperabilidade. A documentação introdutória prioriza as frases naturais acima.

25. coral.jogos e a integração com Pygame CE

A Coral 1.2 usa Pygame CE como backend gráfico oficial, mas a superfície principal é `coral.jogos`. O programa deve poder ser lido como uma descrição do jogo, sem obrigar o usuário a escrever o boilerplate típico do backend gráfico.

Instalação do backend gráfico

```
python -m pip install pygame-ce
coral jogos-check
```

Exemplo recomendado

```
de coral.jogos importe AZUL e AMARELO

crie um jogo chamado jogo com largura 800 e altura 450
crie uma cena chamada principal
crie um sprite chamado jogador em 100 e 180 com largura 40 e altura 40 e cor AMARELO

adicione jogador à cena principal
mude jogo para a cena principal

quando o jogo jogo atualizar com dt
    se a tecla "direita" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a direita
    fim

    se a tecla "esquerda" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a esquerda
    fim
fim

quando o jogo jogo desenhar
    limpe jogo com AZUL
    desene o texto "Use as setas" em 20 e 20 no jogo jogo
fim

inicie jogo
```

Recursos atuais

A API inclui:

- janela e loop de jogo;
- teclado;
- desenho de formas e texto;
- sprites;
- cenas;
- colisão retangular;
- assets;
- animações;

- câmera simples;
- áudio;
- regras e eventos integrados ao loop.

Geometria compartilhada na 1.5.5

Na 1.5.5, `coral.jogos` continua 2D, mas colisões de sprites reutilizam as relações geométricas gerais da baseline. A direção da dependência permanece única: Jogos conhece a infraestrutura espacial; a baseline não conhece sprites, cenas nem Pygame. O contrato histórico de Jogos também é preservado: sobreposição interior conta como colisão, enquanto apenas encostar pelas bordas não conta.

O exemplo `Livro/Exemplos/Jogos/01_geometria_compartilhada.coral` executa sem janela e sem Pygame. Ele compara dois sprites sobrepostos, um terceiro apenas tangente e usa, no mesmo programa, uma consulta espacial geral sobre posições comuns. A versão equivalente na biblioteca geral fica em `Exemplos/Jogos/04_geometria_compartilhada.coral`.

Mouse e tela cheia na 1.5.2

A Coral 1.5.2 torna mouse e tela cheia parte do contrato público de `coral.jogos`. `criar_jogo(..., tela_cheia=verdadeiro)` permite iniciar no modo de tela cheia pela API avançada, e `Jogo` pode consultar ou alternar esse estado durante a execução. O comportamento padrão permanece em janela quando a opção é omitida.

A entrada de mouse diferencia estado contínuo de eventos do quadro. `posicao_mouse()` e `botao_mouse_pressionado(...)` consultam o estado atual; `mouse_clicado(...)`, `mouse_solto(...)` e `roda_mouse()` representam eventos observados no quadro corrente. Essa distinção evita confundir manter um botão pressionado com um novo clique. Os nomes de botão aceitos incluem esquerdo, meio e direito.

No `BackendNulo`, `definir_mouse(...)` fixa um estado e `programar_mouse(...)` descreve quadros determinísticos, permitindo testar interação sem abrir janela. O mesmo backend registra mudanças de tela cheia. A abertura real de janela, a entrada do sistema e o modo de tela cheia continuam dependendo do Pygame CE e do ambiente gráfico.

Spritesheets e transformações visuais introduzidas na 1.5.3

A linha 1.5.3 acrescentou uma camada headless de spritesheets sem tornar Pygame uma dependência do núcleo. `SpriteSheet` descreve uma folha ou atlas, `QuadroSprite` representa um recorte imutável, e animações podem consumir esses quadros sem abandonar o formato antigo baseado em caminhos de imagens.

No mesmo ciclo, `Sprite` ganha transformação visual independente da colisão e do estado de `coral.mundo`: escala uniforme, rotação em graus, espelhamento horizontal e vertical, origem e pivô local. As origens estruturadas reconhecidas são `topo_esquerdo`, `centro` e `centro_inferior`. Um pivô personalizado é informado como `(x, y)` nas unidades locais anteriores à escala. O ponto escolhido permanece ancorado na posição do sprite enquanto a representação visual é transformada.

A chamada estruturada `Jogo.desenhar_imagem(...)` aceita os mesmos parâmetros opcionais. `Omiti` los preserva a semântica anterior. O `BackendNulo` registra comandos transformados sem abrir arquivos ou janela; o backend Pygame aplica as operações à imagem carregada ou ao

quadro recortado. A abertura de janela e o resultado visual físico continuam sujeitos a teste de campo com Pygame CE.

Estas interfaces pertencem à camada avançada e interoperável. Elas foram mantidas como fundação técnica até os gates de contrato, erros e regressão ficarem verdes; só então partes com ganho pedagógico claro começaram a receber frases Coral naturais.

No DEV3, a integração com `coral.mundo` ganha uma ponte visual opcional do lado de `coral.jogos`. `Cena.representar(...)` associa uma entidade a um `Sprite` por atributos de posição explicitamente nomeados, mantendo a entidade como fonte de verdade. A dependência não é invertida: `coral.mundo` continua sem importar jogos, sprites ou Pygame, de modo que a mesma entidade pode existir em simulações headless e em aplicações sem interface.

A biblioteca atual não mantém mais o antigo laboratório versionado de desenvolvimento. O exemplo `Exemplos/Jogos/02_sprites_transformacoes_e_animacao.coral` concentra a superfície estável de sprites, transformações e automações em modo headless; validações históricas de spritesheets, recortes e estados permanecem na suíte de regressão.

No DEV4, uma primeira fatia da superfície natural é promovida para parser/AST: carregue a spritesheet ..., pegue o quadro ..., crie uma animação ..., use o quadro ..., use a animação ... e represente ... com o sprite ... na cena Essas frases compilam para a API estrutural já estabilizada e são conhecidas pela semântica, tabela de símbolos, Naturalidade, Ensinar e LSP. Métodos diretos continuam válidos como API avançada, mas a documentação para iniciantes prioriza as frases Coral.

O exemplo `Exemplos/Jogos/02_sprites_transformacoes_e_animacao.coral` demonstra a superfície estável de sprites sem janela e sem Pygame CE.

No DEV5, as transformações visuais estabilizadas recebem a superfície natural ajuste a escala do sprite ..., gire o sprite ... em ... graus, espelhe o sprite ... e ancore o sprite A ancoragem por origem aceita topo esquerdo, centro e centro inferior; selecionar uma origem limpa um pivô personalizado anterior. `ancore ... no pivô X e Y` volta a definir um ponto local personalizado que prevalece na transformação. Essas frases também são conhecidas por semântica, Naturalidade, Ensinar, LSP, corretor, realce e snippets da extensão.

O mesmo exemplo demonstra escala, rotação, espelhamento, ancoragem, colisão associada e automação matemática sem janela.

No DEV8, o modelo direcional deixa de tratar cima, baixo, esquerda e direita como uma lista estrutural fixa. A base compartilhada passa a ser um conjunto arbitrário de direções nomeadas, cada uma associada a um vetor bidimensional. Duas, quatro e oito direções são apenas predefinições convenientes do mesmo mecanismo. Vetores diagonais são normalizados para preservar a distância efetiva do movimento.

```

crie um conjunto de direções chamado diagonais
  direção nordeste com vetor 1 e -1
  direção sudoeste com vetor -1 e 1
fim
use movimento com diagonais em jogador
mova jogador 10 para nordeste

```

A animação direcional usa os mesmos nomes livres e continua em bloco declarativo:

```

crie uma animação direcional chamada andar usando folha
  direção esquerda na linha 0
  direção direita na linha 1
  quadros 0 até 5
  velocidade 10 quadros por segundo
  repetir
fim

```

Movimento e apresentação não precisam possuir a mesma quantidade de direções. use movimento em 8 direções no jogador pode ser ligado a uma animação com apenas esquerda e direita por ligue a animação andar ao movimento de jogador no sprite visual. A ponte visual escolhe a direção disponível mais próxima do vetor lógico. O estado também separa direção de movimento e direção de olhar: pare o movimento de jogador limpa somente o movimento atual e conserva para onde o alvo estava olhando.

O núcleo compartilhado do DEV8 vive internamente em `coral.direcoes` e ainda não constitui uma nova biblioteca pública estável. `coral.mundo` continua sem importar sprites, animações ou Pygame, enquanto `coral.jogos` consome o estado direcional apenas para apresentação. O exemplo `Exemplos/Jogos/02_sprites_transformacoes_e_animacao.coral` reúne a demonstração atual em modo headless.

BackendNulo

BackendNulo executa a lógica sem abrir uma janela. Ele existe para testes automatizados, CI, simulações e desenvolvimento em ambientes sem vídeo. Isso permite testar a maior parte da API mesmo sem Pygame CE instalado.

Pygame CE

Pygame CE permanece o backend gráfico oficial de `coral.jogos` na 1.5.5. A documentação do Pygame CE cobre janela/display, desenho, eventos, teclado, imagens, áudio, tempo e sprites, que são as capacidades sobre as quais a camada Coral é construída.

API avançada

O runtime também pode ser manipulado por classes e métodos (Jogo, Cena, Sprite, `jogo.executar()`, etc.). Essa camada é válida para extensão da engine e interoperabilidade, mas os templates oficiais priorizam a sintaxe natural Coral quando ela existe.

Instalar a dependência correta

Pygame CE é o backend gráfico oficial. O runtime básico funciona sem ele. Instale Pygame usando o Python da instalação persistente. No Linux, com a pasta padrão:

```

"$HOME/.local/share/coral/ambiente/bin/python" -m pip install pygame-ce
coral jogos-check

```

Se você mudou `XDG_DATA_HOME` ou o destino, use o caminho mostrado pelo diagnóstico. A detecção de pygame comprova importação, mas não comprova abertura de janela, áudio ou comportamento de drivers.

Criar um jogo

```
coral novo MeuJogo --template jogo-2d
cd MeuJogo
coral executar
```

O template contém as importações de cores, uma cena e um sprite controlado pelo teclado. No evento de atualização, `dt` representa o tempo do quadro em segundos. Multiplicar a velocidade por `dt` permite expressar deslocamento por segundo em vez de deslocamento fixo por quadro.

Ler a arquitetura do jogo

Peça	Responsabilidade
Jogo	janela, eventos, ritmo de atualização e encerramento
Cena	agrupar sprites e organizar entrada, saída, atualização e desenho
Sprite	posição, dimensões, aparência, velocidade e colisão retangular
Assets	localizar arquivos de imagem e som dentro de uma raiz
Animacao	alternar quadros ao longo do tempo
Camera	transformar coordenadas do mundo para a tela
MotorRegras	avaliar regras e temporizadores durante o ciclo
BackendNulo	exercitar lógica e registrar comandos sem janela

Cores como AZUL e AMARELO precisam ser importadas. Um sprite com imagem depende de um arquivo existente. Mantenha caminhos de assets relativos à pasta de trabalho prevista. O build de projetos não copia automaticamente os arquivos de mídia.

Teste sem janela

Arquivo: Exemplos/Jogos/01_jogo_e_entrada_sem_janela.coral.

```
de coral.jogos importe criar_jogo e BackendNulo
defina backend como BackendNulo()
defina jogo como criar_jogo("Teste", backend=backend)
execute jogo.executar(max_quadros=2)
mostre backend.quadros
```

Saída esperada:

```
2
```

Essa é uma demonstração deliberada da API avançada para testes. Ela limita a execução a dois quadros e verifica o contador do backend. Não prova que o mesmo ambiente possui suporte gráfico. Para desenvolvimento de um jogo real, teste também janela, teclado, encerramento e áudio no seu computador.

A sintaxe natural completa de criação e callbacks está exemplificada no template `jogo-2d` e em `Documentacao/Desenvolvimento/Projeto/JOGOS.md`. Os componentes `Sprite`, `Cena`, `Assets`, `Animacao` e `Camera` possuem interfaces avançadas em `coral/jogos/objetos.py`.

Construir o primeiro jogo, passo a passo

O arquivo principal.coral do template jogo-2d tem o programa abaixo. Ele abre uma janela e permanece em execução até o usuário fechá-la. Requer Pygame CE e ambiente gráfico; não é um dos exemplos de saída textual automática.

```
# Requer pygame-ce para abrir a janela.
de coral.jogos importe AZUL e AMARELO

crie um jogo chamado jogo com largura 800 e altura 450
crie uma cena chamada principal
crie um sprite chamado jogador em 100 e 180 com largura 40 e altura 40 e cor AMARELO
adicione jogador à cena principal
mude jogo para a cena principal

quando o jogo jogo atualizar com dt
    se a tecla "direita" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a direita
    fim
    se a tecla "esquerda" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a esquerda
    fim
fim

quando o jogo jogo desenhar
    limpe jogo com AZUL
    desenha o texto "Use ← e → para mover" em 20 e 20 no jogo jogo com tamanho 24
fim

inicie jogo
```

1. A importação disponibiliza as duas cores. A criação do jogo define uma janela de 800 por 450 pixels.
2. A cena reúne o conteúdo visível. O sprite começa em $x=100$ e $y=180$, com largura e altura de 40 pixels. Adicioná-lo à cena e selecionar a cena são operações separadas.
3. O evento de atualização consulta o estado das setas a cada quadro. 180 vezes dt desloca o jogador de acordo com o tempo decorrido. As coordenadas de tela crescem para a direita e para baixo.
4. O evento de desenho limpa o fundo e escreve a instrução. Em seguida, o ciclo desenha a cena, incluindo seus sprites. Desenho deve ocorrer novamente a cada quadro.
5. inicie jogo entra no ciclo de execução. Para começar, altere a cor, a velocidade e a posição inicial, uma mudança por vez.

O programa não impede o jogador de sair da janela: limites e regras de colisão pertencem à lógica do jogo. Também não carrega imagens ou sons. Acrescente esses recursos depois de conferir o movimento básico.

Referência prática dos objetos gráficos

A superfície natural é a primeira opção para iniciar jogos. As assinaturas abaixo documentam a API de componentes para integração e comportamentos mais específicos. `objeto.metodo(...)` usa a instância que você criou. `self` não é passado nas chamadas.

Operação	Contrato
<code>criar_jogo(titulo, largura=800, altura=450, fps=60, backend=None)</code>	Cria o jogo; backend omitido seleciona Pygame

Operação	Contrato
<code>jogo.executar(max_quadros=None)</code>	Executa até o encerramento; o limite é útil em testes
<code>jogo.parar()</code>	Solicita encerramento do ciclo
<code>jogo.mudar_cena(cena)</code>	Sai da cena anterior e entra na nova
<code>jogo.tecla_pressionada(tecla)</code>	Consulta uma tecla no backend
<code>jogo.tocar_som(caminho, volume=1.0)</code>	Reproduz o arquivo pelo backend; depende de áudio disponível
<code>jogo.desenhar_imagem(caminho, x, y, largura=None, altura=None, ...)</code>	Desenha imagem ou <code>QuadroSprite</code> ; no desenvolvimento 1.5.3 aceita escala, rotação, espelhamento, origem e pivô opcionais
<code>Cena(nome)</code>	Cria uma coleção de sprites e callbacks
<code>cena.adicionar(sprite)</code>	Adiciona e devolve o sprite
<code>cena.remover(sprite)</code>	Remove se estiver presente
<code>Sprite(x, y, largura, altura, imagem=None, cor=BRANCO)</code>	Parâmetros iniciais do sprite; também aceita velocidades, visibilidade, nome, dados, animação e, no desenvolvimento 1.5.3, escala, rotação, espelhamento, origem e pivô
<code>sprite.mover(dx, dy)</code>	Soma deslocamentos à posição
<code>sprite.atualizar(dt)</code>	Avança velocidades e animação; a cena chama essa operação
<code>sprite.colide_com(outro)</code>	Testa sobreposição retangular; não resolve fisicamente a colisão
<code>Assets(raiz="assets")</code>	Define a raiz dos arquivos de mídia
<code>assets.imagem(nome) / assets.som(nome)</code>	Resolve um arquivo existente dentro da raiz; rejeita saída da raiz
<code>Animacao(quadros, fps=10.0, repetir=True)</code>	Exige ao menos um quadro e fps positivo
<code>animacao.atualizar(dt)</code>	Avança o tempo e devolve o quadro atual
<code>animacao.reiniciar()</code>	Retorna ao primeiro quadro e zera o tempo
<code>Camera(x=0, y=0, largura=800, altura=450)</code>	Define o deslocamento e a área da câmera
<code>camera.seguir(sprite, suavidade=1.0)</code>	Define o alvo; suavidade é limitada ao intervalo de 0 a 1
<code>camera.tela(x, y)</code>	Converte uma posição do mundo em posição da tela

Para usar uma câmera, atribua sua instância a `cena.camera`. A cena atualiza a câmera e transforma as posições ao desenhar. Para animar, atribua uma `Animacao` a `sprite.animacao`; seus quadros representam caminhos de imagens. A atribuição de `sprite.velocidade_x` dispensa movimentá-lo manualmente por essa mesma velocidade no callback: fazer ambos soma os dois deslocamentos.

Callbacks do jogo recebem (jogo), exceto atualização, que recebe (jogo, dt). Callbacks da cena recebem (jogo, cena); atualização da cena recebe (jogo, cena, dt). Registre funções com `quando_iniciar`, `quando_atualizar`, `quando_desenhar` e `quando_encerrar` no jogo, ou com os registradores correspondentes de entrada, saída, atualização e desenho na cena. Uma assinatura incompatível causa erro na execução.

26. Computação científica, geração procedural, laboratório, sistema e hardware, gráficos e Web

A frente científica da Coral é composta por `coral.numerico`, `coral.graficos`, `coral.laboratorio`, `coral.sistema` e `coral.hardware`. A geração procedural é uma fundação transversal separada. A base `coral.web` fornece HTML e URLs locais para apresentação e composição sem rede. Essas áreas permanecem desacopladas: a ausência de NumPy, psutil ou pyserial não impede o núcleo de funcionar.

Numérico

A API normaliza erros públicos relevantes como `ErroNumerico`, valida matrizes, estatísticas, intervalos e reduções vazias e evita vaziar detalhes acidentais do backend NumPy para a superfície Coral. Na 1.6.3, estatísticas numéricas passam a rejeitar dados ausentes, NaN e infinito com diagnóstico público em português. `selecionar_coluna` permite escolher explicitamente se registros ausentes geram erro ou são ignorados. Percentis e quartis usam interpolação linear declarada; correlação rejeita variância zero; moda preserva empates em vez de inventar uma única categoria vencedora.

Na 1.6.4, a mesma política passa a cobrir álgebra linear e geometria 3D. `norma` e `normalizar` trabalham com vetores finitos; `posto` e `minimos_quadrados` validam formas; `produto_vetorial` é restrito a três dimensões. `rotacionar3` usa a fórmula de Rodrigues e ângulo em radianos. `transformar_ponto3` recebe uma matriz afim homogênea 4×4 e usa vetor coluna, mantendo a coordenada homogênea apenas como detalhe interno.

Na 1.6.11, `erro_padrao` usa desvio padrão amostral e exige pelo menos duas observações. `assimetria` e `curtose` usam momentos centrais padronizados e rejeitam variância zero; `curtose` é devolvida em excesso. `matriz_covariancia` interpreta linhas como observações e colunas como variáveis e `matriz_correlacao` rejeita colunas constantes. `regressao_linear` devolve inclinação, intercepto, resíduos, R^2 e quantidade de observações.

`intervalo_bootstrap` implementa intervalo percentil por reamostragem e `teste_permutacao` implementa teste bilateral por diferença da estatística escolhida pelo programa. Ambos usam um gerador local por semente, registram repetição e custo estimado e recusam trabalho acima do limite interno. Intervalos e valores p são resultados numéricos; a biblioteca não os transforma automaticamente em causalidade, superioridade ou significância substantiva.

Na 1.6.12, a camada tridimensional ganha `matriz_rotacao3`, `matriz_translacao3`, `matriz_escalas3`, `compor_transformacoes3` e `transformar_vetor3`. As relações ponto plano são cobertas por `distancia_ponto_plano3`, `projetar_ponto_plano3` e `intersecao_reta_plano3`. `coordenadas_esfericas3` e `ponto_de_esfericas3` fazem a conversão explícita entre representações, enquanto `area_triangulo3` e `volume_tetraedro3` tratam medidas geométricas sem recorrer ao backend gráfico.

Gráficos científicos 2D

`coral.graficos` recebe coleções comuns e produz especificações imutáveis. A criação da especificação é separada da renderização e não abre janela. Linhas e dispersão exigem séries alinhadas e finitas; barras mantêm zero na escala vertical por padrão. Registros comuns podem ser adaptados por campos escolhidos com `series_de_registros`.

Histogramas e caixas não mantêm uma segunda implementação estatística: `grafico_histograma` usa o contrato de `coral.numerico.histograma`, e `grafico_caixa` usa os quartis de `Numérico`. Isso mantém contagens, bordas e quartis consistentes entre cálculo e apresentação.

`grafico_funcao` limita a quantidade de amostras e representa avaliações inválidas como lacunas para não ligar uma falha por uma linha enganosa. `configurar` retorna cópia com título, rótulos, unidades, legenda, tema, proveniência e texto alternativo. `para_svg` e `salvar` produzem SVG 2D autossuficiente usando somente a biblioteca padrão.

```
de coral.graficos importe linha, configurar, salvar

defina figura como linha([0, 1, 2], [20, 21, 23])
defina figura como configurar(figura, titulo="Temperatura", eixos=["Tempo", "Temperatura"],
    unidades=["min", "°C"])
execute salvar(figura, "temperatura.svg")
```

Na 1.6.11, `mapa_calor` representa matrizes retangulares e `mapa_correlacao` reutiliza o cálculo de `coral.numerico.matriz_correlacao`. `barras_erro` e `faixa_incerteza` representam incerteza sem chamar toda faixa de intervalo de confiança. `grafico_experimento` importa Laboratório apenas durante a chamada explícita e converte medições em séries comuns preservando proveniência. `painel` combina múltiplos gráficos sem alterar as especificações filhas. SVG 2D continua disponível somente com a biblioteca padrão; PNG e PDF usam Matplotlib opcional carregado sob demanda.

Na 1.6.12, `dispersao3d` e `trajetoria3d` criam séries tridimensionais sem alterar a ordem ou as coordenadas de entrada. `superficie3d` representa uma grade retangular explícita e não interpola dados ausentes. A especificação 3D continua independente do backend. `para_svg` recusa reduzir 3D a uma projeção implícita; `salvar` usa Matplotlib sob demanda para SVG, PNG e PDF tridimensionais. A superfície espacial dedicada da linha 1.6.x termina em 3D; não há uma API pública planejada para quatro dimensões espaciais.

Gráficos interativos locais

Na 1.6.13, `para_html` gera HTML interativo autossuficiente e `salvar_html` grava o resultado localmente. Plotly é carregado somente nessas operações ou em `explorar`; importar `coral.graficos` continua sem carregar o backend. O arquivo final não usa CDN e contém os dados e o JavaScript necessários à interação.

```
de coral.graficos importe dispersao, configurar, salvar_html

defina figura como dispersao([1, 2, 3], [2, 4, 6])
defina figura como configurar(figura, titulo="Relação entre medidas")
execute salvar_html(figura, "medidas_interativas.html")
```

`explorar` abre a visualização somente quando solicitado. A ausência de sessão gráfica produz diagnóstico que orienta `salvar_html`. `mostrar_grafico` é a única forma pública de redução automática planejada nesta edição e exige decisão explícita; a exportação nunca elimina pontos silenciosamente.

Painéis mistos podem conter gráficos 2D e 3D. Zoom, inspeção e rotação da câmera são recursos de apresentação e não alteram coordenadas, malhas ou resultados estatísticos. Texto alternativo, avisos e proveniência permanecem disponíveis na saída, e o HTML informa que os dados ficam incorporados no arquivo.

HTML e URLs locais

`coral.web.html` oferece `escapar_texto`, `escapar_atributo`, `elemento`, `fragmento`, `documento`, `renderizar` e `tabela`. Texto comum é escapado por padrão, atributos de evento inline são recusados e atributos de URL rejeitam esquemas executáveis. Uma tabela recebe registros gerais e colunas explícitas; campos ausentes são erro, nunca descarte silencioso.

`coral.web.url` oferece `codificar_consulta`, `ler_consulta`, `decompor` e `juntar`. Consultas preservam ordem, valores vazios e chaves repetidas em UTF 8. Decompor e juntar endereços são operações textuais: não resolvem DNS e não abrem conexão.

```
de coral.web.html importe elemento, documento, renderizar
de coral.web.url importe codificar_consulta

defina corpo como elemento("p", {}, "A < B & C")
defina pagina como documento("Exemplo", corpo, "pt-BR")
defina consulta como codificar_consulta(["tag", "coral"], ["tag", "web"])
mostre renderizar(pagina)
mostre consulta
```

Na 1.6.6, cliente HTTP e servidor local ainda não faziam parte da base Web. A 1.6.8 acrescentou o cliente HTTP geral; a 1.6.9 acrescenta o servidor HTTP local com rotas literais, limites explícitos e encerramento controlado. Upload multipart, cookies, WebSocket e hospedagem de produção permanecem fora do escopo.

Cliente HTTP

`coral.web.http` oferece `requisitar`, `obter`, `enviar` e `codificar_formulario`. `RespostaHTTP` preserva status, cabeçalhos, URL final, método e corpo em bytes. `texto(codificacao)` e `json()` interpretam a resposta sob demanda, mantendo erros de formato separados de falhas de transporte.

Toda requisição usa timeout finito e `limite_resposta` impede materialização acima do teto declarado. GET pode seguir redirecionamentos com número máximo de saltos. POST e outros métodos com efeito não repetem a requisição após redirecionamento sem opção explícita. HTTPS usa verificação de certificado e hostname por padrão. Importar o módulo não abre conexão.

```
de coral.web.http importe obter

defina resposta como obter("https://exemplo.test/dados", tempo_limite=5,
    limite_resposta=1048576)
mostre resposta.status
mostre resposta.corpo_bytes
```

O exemplo do cliente continua usando apenas loopback e porta efêmera. Nesta edição, o servidor HTTP local e as rotas literais passam a fazer parte da superfície pública; multipart, cookies persistentes, WebSocket e hospedagem de produção permanecem fora do escopo.

Servidor HTTP local

A Coral 1.6.9 acrescenta `coral.web.servidor` como servidor de desenvolvimento local. `ServidorHTTP` registra rotas literais e só abre uma porta quando `iniciar()` é chamado. O padrão é 127.0.0.1 com porta efêmera; exposição externa exige opção explícita. `RequisicaoHTTP` preserva método, caminho, consulta, cabeçalhos e corpo em bytes, com leitura opcional de texto, JSON e formulários. `resposta_texto`, `resposta_html`, `resposta_json` e `resposta_arquivo` constroem respostas sem conversões silenciosas. `limite_corpo` é aplicado antes do tratador, arquivos permanecem sob raiz declarada e `encerrar()` fecha o socket explicitamente. O módulo é voltado a desenvolvimento local, não a hospedagem de produção.

Na 1.6.13, `resposta_grafico(grafico)` recebe uma especificação de `coral.graficos` e produz uma resposta HTML interativa autossuficiente. A função não cria servidor, não registra rota e não abre porta. Essa fronteira mantém o cliente `coral.web.http` separado da apresentação e preserva a decisão explícita do programa sobre quando publicar conteúdo localmente.

Geração procedural determinística

`coral.procedural` é uma fundação geral introduzida na 1.5.11. Ela não depende de Jogos, RPG nem interface gráfica. `Semente`, `GeradorProcedural`, `SequenciaProcedural` e `CampoProcedural` permitem derivar domínios independentes, consultar valores por chave ou coordenada sem depender da ordem de consultas anteriores, compor campos e distribuir pontos sob limites finitos. Na 1.6.15, pesos inválidos recebem diagnóstico explícito, campos declaram dimensionalidade e intervalos conhecidos, e a ordem de operações não comutativas permanece observável.

`ReceitaProcedural`, `receita_procedural`, `reconstruir_receita` e `diagnosticar_custo_receita` formam a camada declarativa portátil. As receitas suportadas registram algoritmo, versão, semente e parâmetros e podem ser preservadas por `coral.persistencia`. Callbacks não entram na forma portátil. Reconstrução e materializações grandes possuem limites de custo explícitos, sem esconder trabalho ou alocação potencialmente desproporcional.

O módulo serve a simulações, mapas, ecologia, dados sintéticos e outros domínios. Jogos e Laboratório podem consumi-lo, mas não são sua finalidade obrigatória.

Na 1.6.17, `amostrar_campo` e `matriz_campo` tornam a observação do campo uma operação explícita e limitada. `anotar_pontos` combina distribuições espaciais com grandezas como altitude, umidade ou risco. `proveniencia_procedural` fornece metadados portáteis sem afirmar que um callback arbitrário pode ser reconstruído.

A ponte com Jogos vive no consumidor: receitas de `distribuir_pontos` podem preencher cenas com sprites reproduzíveis e campos podem escrever valores em `sprite.dados`, sem abrir janela. A ponte com Laboratório trata receita e semente procedural como parte do desenho experimental, separadas da semente usada pelo próprio protocolo de medição. `planejar_experimento_procedural` calcula execuções e custo máximo antes de reconstruir a receita; `executar_experimento_procedural` preserva a receita base e as sementes nos metadados do resultado.

Criptografia didática

`coral.criptografia` oferece `resumir`, `verificar_resumo`, `autenticar`, `verificar_autenticacao`, `gerar_bytes_seguros`, `gerar_token_seguro`, `comparar_com_seguranca`, `listar_algoritmos` e `explicar_algoritmo`. Texto usa UTF 8 e valores binários preservam bytes. `resumir` e `verificar_resumo` aceitam SHA 256 ou SHA 512; HMAC pertence a `autenticar` e `verificar_autenticacao`.

Resultados podem ser hex ou bytes. Uma comparação com resumo válido porém diferente retorna falso, enquanto comprimento, tipo ou representação hexadecimal inválidos produzem erro de formato. Chaves vazias são recusadas e mensagens de erro não reproduzem chaves, dados autenticados ou resultados recebidos.

Bytes e tokens seguros usam a fonte criptográfica da plataforma por meio de secrets; eles não reutilizam o gerador pseudoaleatório de `coral.aleatorio` ou `coral.procedural`. O catálogo diferencia hash de autenticação e deixa explícito que nenhum desses mecanismos cifra dados.

```
de coral.criptografia importe resumir, autenticar, verificar_autenticacao

defina mensagem como "Coral ç"
defina chave como "chave de exemplo"
defina resumo como resumir(mensagem, "SHA-256")
defina etiqueta como autenticar(mensagem, chave, "sha256")
garanta que verificar_autenticacao(mensagem, chave, etiqueta, "sha256")
mostre resumo
```

A 1.6.16 não oferece cifragem simétrica ou assimétrica, derivação de senha, certificados, assinaturas digitais, armazenamento de credenciais ou gestão de chaves. Essas capacidades exigem um roadmap próprio.

Laboratório

Experimentos aceitam repetições, parâmetros, sementes e métricas. A implementação preserva os estados aleatórios globais do chamador e não encerra um `tracemalloc` que já estivesse ativo antes do experimento. Na 1.6.10, `planejar_experimento` calcula dimensões, combinações e execuções previstas antes da execução; `selecionar_medicoes` filtra resultados sem alterar o original; e `resumir_por_parametro` apresenta contagens, falhas, tempos e memória por cenário.

`validar_resultado` produz diagnósticos estruturados antes de persistir ou comparar. O JSON completo usa `coral.laboratorio.resultado/1`, esquema 1, e `carregar_resultado_json` reconstrói tipos portáteis e reconhece explicitamente o formato histórico conhecido até a 1.6.9. CSV e Markdown continuam formatos de relatório, enquanto JSON é a representação recuperável. `comparar_experimentos` descreve diferenças de configuração, sucessos, tempo e memória usando `b - a`, mas não calcula significância estatística, causalidade nem superioridade geral.

Na 1.6.17, receitas procedurais podem ser planejadas e executadas como fator experimental. O plano registra quantidade de sementes, grade, repetições e custo preventivo. Cada medição preserva `semente_procedural`, enquanto o resultado guarda a receita base em metadados. Esse mecanismo mede configurações; ele não escolhe automaticamente a melhor semente nem transforma diferenças descritivas em conclusão científica.

Sistema

`coral.sistema` separa inspeção de capacidade de execução. `localizar_programa` e `programa_disponivel` consultam executáveis sem iniciá-los; `quantidade_processadores`, `pasta_temporaria` e `terminal_interativo` descrevem capacidades do hospedeiro sem prometer recursos efetivos sob quotas. `ambiente_selecionado` copia apenas nomes explícitos e `variavel_ambiente_obrigatoria` falha sem incluir valores em diagnósticos. `argumentos_programa` expõe somente os argumentos do programa Coral, enquanto `caminho_executavel` identifica o executável que hospeda o runtime.

`executar_comando` continua sem shell implícito e agora aceita `codificacao`, política explícita de erros de decodificação e `limite_saida_bytes`. O limite é aplicado enquanto `stdout` e `stderr` são drenados, e `saida_truncada` e `erro_truncado` registram descarte sem transformar truncamento em falha. `Timeout` tem prioridade sobre código de saída. `ResultadoComando` preserva seus campos históricos e acrescenta leituras por linhas, `codigo_saida`, `tempo_decorrido` e `exigir_sucesso`, que produz `ComandoFalhou` preservando o resultado completo.

Hardware e serial

Inventário de hardware degrada de forma previsível quando dependências opcionais não estão instaladas. A serial possui um backend simulado determinístico usado por contratos e testes. Comunicação com Arduino ou ESP32 físicos continua uma evidência de campo separada e não é declarada aprovada por simulação.

Integração científica e Web na 1.6.14

`Exemplos/Integracao/01_registros_cientificos.coral` demonstra que a seleção de registros em `coral.numerico` pode ser combinada com `coral.matematica.media`, gráficos SVG 2D e `coral.web.html` sem Laboratório, NumPy ou Plotly. As temperaturas 20, 22, 24 e 26 produzem média 23 °C. O texto de um sensor contendo < aparece escapado na tabela.

`Exemplos/Integracao/02_experimento_cientifico.coral` planeja quatro execuções, com sementes de 42 a 45, salva `coral.laboratorio.resultado/1` e produz SVG com proveniência. `03_experimento_web.coral` carrega o JSON, salva HTML interativo autossuficiente e publica duas rotas HTTP em loopback, uma de dados e outra de gráfico. Plotly é opcional, mas necessário nesse último programa. O servidor é encerrado em `finalmente`; o arquivo HTML incorpora os dados e merece revisão antes de ser compartilhado.

O contrato da instalação local é descrito em

`Documentacao/Desenvolvimento/Projeto/INSTALADOR_MULTIPLATAFORMA_1_6_14.md`. SHA256 permite conferir integridade, mas não comprova autoria do pacote. A atualização por pacote com executor privado ainda não integra os assistentes desta edição.

A Coral 1.7.0 parte da 1.6.17 congelada. A integração científica, Web, Procedural e Criptografia permanece preservada; a mudança corrente está no empacotamento e na instalação. O executável Linux e o pacote Debian possuem evidência nativa. Windows permanece explicitamente não executado nesta edição e será exercitado na 1.7.0.

Coesão científica consolidada

Na base 1.5, Laboratório pode anexar informações de Sistema como metadados portáteis e pode expor retornos de um experimento para análise Numérica sem criar dependência obrigatória de NumPy. Quando NumPy não está instalado, o núcleo, Laboratório e os diagnósticos continuam utilizáveis; somente a capacidade Numérico que depende dele fica indisponível.

O fluxo recomendado mantém responsabilidades separadas:

1. Hardware ou uma fonte simulada produz dados.
2. Laboratório controla repetição, semente, parâmetros, métricas e falhas.
3. Sistema fornece contexto de ambiente quando solicitado explicitamente.
4. Numérico transforma e analisa séries quando sua capacidade está disponível.
5. Persistência ou os exportadores registram resultados em formatos portáteis.

As formas adicione ambiente aos metadados do experimento e extraia os retornos do experimento ... são construções estruturais da linguagem. O E2E científico usa BackendSerialSimulado, exporta JSON, CSV e Markdown e mantém dados determinísticos separados de informações naturalmente voláteis como PID, tempo de parede e uso momentâneo de memória.

Persistência

A superfície pública `coral.persistencia/1` usa um envelope versionado com contrato, versão de esquema, tipo Coral e carga útil compatível com JSON. Mundo, RPG, Jogos e Laboratório podem persistir estado sem serializar callbacks, janelas, listeners vivos ou identidade acidental de objetos Python.

Em Jogos, por exemplo:

```
salve o jogo aventura em "partida.coral-save"
carregue o jogo de "partida.coral-save" como aventura_reaberta
```

Um jogo reaberto usa BackendNulo, começa parado e restaura somente estado persistente. Combates contextuais preservam participantes, ordem, índice e rodada utilizando as entidades reconstruídas do Mundo.

Pontes entre domínios

A Coral não funde bibliotecas diferentes para criar coesão. O contrato `coral.pontes/1` descreve integrações que já possuem evidência executável e serve como fonte única para runtime, Manager, diagnósticos, LSP, VS Code e Coral Ensinar.

As pontes consolidadas cobrem Mundo + RPG, Mundo + Jogos, RPG + Jogos, Laboratório + Numérico, Laboratório + Sistema, Laboratório + Hardware Serial e Persistência com Mundo, RPG, Jogos e Laboratório. A matriz declara capacidades necessárias, dependências opcionais, exemplos e estado de consolidação, mas **não executa lógica de domínio**.

Hardware auditado

A superfície de Hardware distingue informação de CPU/memória, serial real e BackendSerialSimulado. psutil e pyserial continuam opcionais. Dependência ausente produz diagnóstico Coral e plano do Manager em vez de crash ou instalação implícita. Timeout, desconexão, fechamento e erro de backend têm contratos testados; hardware físico permanece evidência de campo.

27. Modo Aprender, modo Ensinar e auditoria de naturalidade

Coral possui três ferramentas diferentes que às vezes parecem semelhantes, mas têm finalidades e riscos distintos.

Ferramenta	Objetivo	Executa o programa?
coral ensinar	Explica estaticamente as construções do texto	Não
Estudo automático do VS Code	Reexecuta a análise estática enquanto o arquivo é editado	Não
coral explicar / Modo Aprender	Executa e rastreia passos, valores, escopos e saída	Sim
coral naturalidade	Audita estilo e preferência por formas idiomáticas	Não

Ensinar

```
coral ensinar programa.coral
coral ensinar-json programa.coral
```

A análise é local e determinística: não abre jogos, não pede entrada, não grava os arquivos que o programa escreveria e não percorre loops como execução real. No VS Code, use **Coral: Ativar estudo automático (ensinar)** e feche o painel ou use **Desativar estudo automático** para encerrar.

Na linha 1.5.1, quando a validação estática encontra um erro de compilação, o Ensinar apresenta primeiro uma explicação didática em português e mantém o tipo e a mensagem técnica em um campo separado para diagnóstico. Essa melhoria não transforma o Ensinar em executor: executado permanece falso e nenhum efeito do programa é realizado.

Ensinar integrações consolidadas

O Ensinar consulta a mesma matriz de pontes usada pelas demais ferramentas. Ele pode explicar os fluxos Mundo → RPG → Jogos, Hardware → Laboratório → Numérico, Sistema → Laboratório e Persistência → estado dos domínios. A explicação informa capacidades opcionais e falhas possíveis mesmo quando uma dependência não está instalada, sem executar o programa e sem inventar suporte que não exista no runtime.

Modo Aprender

```
coral explicar programa.coral
coral explicar-json programa.coral
```

Como esse modo executa código, efeitos colaterais são reais. Use em programas confiáveis.

Naturalidade

```
coral naturalidade programa.coral
coral naturalidade . --json
```

A métrica existe para proteger a identidade da linguagem. Ela prefere construções como `adicione 5 a pontos`, frases naturais das APIs oficiais e evita ensinar `python: onde` já existe forma Coral.

A Coral aceita interoperabilidade e programação orientada a objetos convencional, mas a identidade da linguagem depende de o código principal continuar legível em português.

Use:

```
coral naturalidade programa.coral
```

A auditoria não é um verificador de correção. Ela procura sinais de que um programa está usando escapes ou APIs de baixo nível onde a biblioteca oficial já oferece uma forma Coral.

Exemplo preferido:

```
crie um mundo chamado campanha
crie um personagem chamado heroína no mundo campanha com 12 de vida
role um dado de 20 lados como ataque
cause 3 de dano a heroína
```

A forma orientada a objetos continua válida para casos avançados:

```
defina heroína como criar_personagem(campanha, "Heroína", 12)
```

O objetivo da auditoria é preservar a experiência da linguagem em exemplos, aulas e projetos que desejam seguir o estilo idiomático do Coral; ela não tenta proibir interoperabilidade.

Regra principal

Escreva intenção em português primeiro. Use a forma de interoperabilidade somente quando a superfície Coral não oferecer a operação necessária.

Prefira

```
defina energia como 10
se energia for menor ou igual a 3 então
    mostre "Energia baixa"
senão
    subtraia 1 de energia
fim
```

Evite como estilo introdutório

```
python: energia = 10
```

APIs oficiais

Prefira `crie um mundo`, `crie um personagem`, `role um dado`, `cause ... de dano`, `crie um jogo`, `crie uma cena`, `mova ...` e os blocos quando o `jogo ...` quando essas formas expressarem a intenção. Chamadas de método continuam válidas para interoperabilidade e casos avançados, mas não são a voz principal da documentação.

Regras e eventos

Use `regra`, quando `ocorrer`, `depois de` e `a cada` quando o domínio é reativo. Evite esconder essa intenção em callbacks Python se a construção Coral já existe.

Naturalidade automatizada

```
coral naturalidade principal.coral
coral naturalidade .
coral naturalidade . --json
```

A auditoria não substitui compilação, lint ou testes. Ela protege a identidade editorial da linguagem. Pontuações baixas não significam necessariamente código inválido; indicam código menos idiomático para a superfície oficial da versão.

Integração com a edição tolerante da 1.3

O estudo automático acompanha documentos ainda não salvos. A linha 1.3 distingue a validação tolerante durante a digitação da validação estrita aplicada ao salvar. Trechos reconhecidos como incompletos podem receber dicas de edição em vez de uma cascata de erros enquanto uma palavra ou cabeçalho ainda está sendo escrito.

PARTE IV: FERRAMENTAS, CLI E VS CODE

Fluxo diário de execução, verificação, formatação, documentação, depuração, LSP, edição tolerante e diagnóstico.

28. CLI Coral: executar, verificar, formatar, testar e documentar

O comando `coral` reúne execução, inspeção do compilador, projetos, testes, documentação, editor e diagnósticos da linha 1.3.

Fluxos do dia a dia

```
coral programa.coral
coral verificar programa.coral
coral lint programa.coral
coral formatar programa.coral
coral testar
coral documentar
coral explicar programa.coral
coral ensinar programa.coral
coral naturalidade .
```

Diagnóstico e release 1.3

```
coral --numerico-check
coral --sistema-info
coral --hardware-check
coral --laboratorio-check
coral --contratos-1-3
coral --conformidade-1-3
coral --gate-1-3
coral --release-check-1-3
```

Inspeção do compilador

```
coral --tokens-expressao "1 mais 2 vezes 3"
coral --ast-expressao "1 mais 2 vezes 3"
coral --ast-programa programa.coral
coral --semantica programa.coral
coral --simbolos programa.coral
coral --mapa-linhas programa.coral
```

Cache

```
coral --cache-info programa.coral
coral --sem-cache programa.coral
coral --limpar-cache programa.coral
```

Referência completa

Este capítulo mantém apenas os comandos necessários ao fluxo cotidiano. A listagem integral das opções da CLI fica concentrada no **Capítulo 49, Referência do CLI**, evitando duplicar no percurso didático um bloco extenso de documentação de referência. Para conferir a instalação atual, use também `coral --help`.

29. VS Code 0.72.0: recursos de linguagem e produtividade

A extensão 0.72.0 distribuída com a Coral 1.7.0 reúne realce, snippets, formatação, LSP, DAP, Test Explorer, Project Explorer, criação de projetos, Modo Aprender, Coral Ensinar, diagnóstico, tokens semânticos, navegação e integração com componentes e Coral Manager.

Resolução do runtime

A ordem de preferência é explícita: configuração do workspace/usuário quando fornecida, instalação persistente quando disponível e mecanismos de fallback controlados. **Coral: Mostrar runtime selecionado** apresenta a decisão efetiva.

Edição tolerante

Durante digitação, o LSP reduz cascatas de erros em cabeçalhos e blocos parciais, mantém diagnósticos versionados e reanalisa o documento completo quando necessário para preservar consistência.

Componentes

Coral: Verificar suporte opcional e **Coral: Instalar suporte opcional** delegam ao gerenciador da própria Coral. A extensão não implementa uma segunda política de pip.

Coral Manager

A extensão pode abrir o Manager e disparar diagnóstico da instalação, mantendo editor e runtime sobre o mesmo contrato de manutenção.

30. Depuração: F5, breakpoints, pilha, variáveis e limites

O DAP Coral é implementado pelo runtime. F5 no VS Code inicia uma sessão que compila o projeto, preserva mapa de linhas e expõe pausa, pilha, variáveis e avaliação.

Breakpoints e módulos locais

A linha 1.3.11 eliminou uma limitação importante: breakpoints podem ser registrados em módulos `.coral` locais antes do launch e permanecem válidos quando a execução atravessa imports do projeto.

Navegação

`continue`, `next`, `stepIn` e `stepOut` possuem contratos automatizados. `stackTrace`, `scopes`, `variables` e `evaluate` permitem inspecionar o estado disponível na pausa.

O que ainda depende de ambiente gráfico

A lógica do protocolo é testada automaticamente. O comportamento completo da interface de uma janela real do VS Code em Windows é verificado pela suíte de campo e só recebe estado de aprovado mediante evidência externa.

31. Diagnóstico, logs, atualização e solução de problemas

A linha 1.4 possui três superfícies complementares: Coral Manager, Coral Setup e os assistentes autocontidos.

Coral Manager

`coral manager` é a API operacional principal. Ele diagnostica, planeja instalação, atualização, reparo e remoção. Operações mutáveis só aplicam mudanças com confirmação explícita pelo parâmetro apropriado.

Coral Setup

`coral setup` é uma interface gráfica construída sobre `coral.manager/1`. O modo `coral setup --diagnostico` permite testar o contrato sem abrir janela.

Assistentes

`Instalar_Coral.pyz` e `Diagnosticar_Extensao.pyz`, com lançadores `.sh` e `.cmd`, permanecem como entradas autocontidas. O diagnosticador coleta evidências de runtime, DAP, ensino, VSIX e, quando possível, Extension Host.

Logs

A extensão possui o canal **Coral: Log** e exportação de log. A suíte Windows/VS Code cria um ZIP único de evidências tanto em sucesso quanto em falha, incluindo resumo JSON, eventos, `stdout/stderr` e registros relevantes do editor.

Atualizar uma instalação existente para a 1.7.0

1. Faça backup de projetos e configurações pessoais.
2. Use a distribuição completa 1.7.0 ou um patch cuja base seja declarada pelo fluxo de release; não aplique patches sobre uma base diferente da indicada.
3. Execute `Coral_Setup` ou `coral manager atualizar` para obter o plano da mudança.
4. Revise o plano antes de usar a forma com `--aplicar`.

5. Confirme `coral --versao` e `coral --self-check`. Os gates amplos de release pertencem ao processo de desenvolvimento e congelamento, não ao uso cotidiano.

A atualização não instala dependências opcionais silenciosamente. Perfis e componentes continuam sob o contrato do Manager e o modo Personalizado permanece disponível.

PARTE V: IMPLEMENTAÇÃO TÉCNICA E ARQUITETURA

Esta parte é voltada a mantenedores, autores de ferramentas e leitores que querem compreender como a Coral é construída. Aqui aparecem os termos de engenharia deliberadamente evitados na porta de entrada: contratos executáveis, gates, evidência de release, parser, AST, backend, LSP, DAP e empacotamento.

O que é normativo nesta edição

A autoridade técnica desta edição é a árvore **Coral 1.7.0**, aberta sobre a release 1.6.17 congelada. `Projeto/pyproject.toml` e `Projeto/coral/__init__.py` devem declarar a mesma versão, `Projeto/dist/` contém apenas artefatos correntes e runtimes históricos necessários aos testes ficam em `Testes/Fixtures/runtimes/`. O runtime portátil corrente é `coral-1.7.0.pyz` e a extensão distribuída é **Coral Language 0.72.0**.

Documentos de releases anteriores permanecem como histórico técnico, mas não têm precedência sobre código, contratos executáveis, gates e documentação corrente.

Princípios de manutenção

1. Português corrente, não tradução lexical mecânica.
2. Naturalidade controlada, sem interpretação livre por IA.
3. Legibilidade antes de concisão.
4. APIs oficiais devem oferecer uma superfície Coral quando isso fizer sentido.
5. Previsibilidade acima de adivinhação.
6. Python é plataforma de execução, não aparência obrigatória do código.
7. `python:` é escape explícito para interoperabilidade.
8. Sintaxe oficial deve chegar ao parser, à AST e aos diagnósticos.
9. Ensino deve explicar estado, fluxo, funções, objetos e erros sem esconder os conceitos.
10. Novos recursos devem ser determinísticos, diagnosticáveis, testáveis e documentados.

32. Fundação em Python: transpiler, runtime e empacotamento

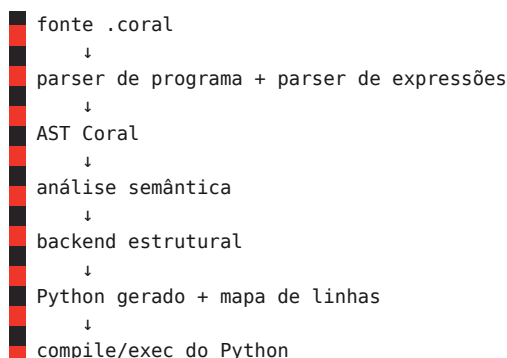
Coral é implementada em Python e **transpila** programas `.coral` para Python. Isso permite aproveitar portabilidade, biblioteca padrão, ecossistema, mecanismos de exceção, `asyncio` e empacotamento Python sem obrigar a superfície da linguagem a parecer Python.

Camadas de distribuição

- `Projeto/pyproject.toml` define o pacote `coral-linguagem 1.5.5`, requer Python `>=3.10` e registra o comando `coral = coral.cli:main`.
- `coral-1.6.1.pyz` é um **Python zip application**: um arquivo ZIP executável por Python que contém o pacote e um ponto de entrada.

- a instalação persistente cria um venv, copia o .pyz e cria launchers.
- a extensão VS Code não contém um segundo compilador; ela chama o mesmo runtime por CLI/LSP/DAP.

O que transpilar significa aqui



O Python gerado é uma representação de backend. Ele pode ser inspecionado com `--mostrar-python` ou salvo com `--compilar`, mas não é a API editorial principal da linguagem.

Modelo de segurança

Coral **não é um sandbox**. Um programa Coral, inclusive código trazido por `python:`, executa com as permissões do processo Python. Nunca execute código Coral não confiável supondo isolamento.

Interoperabilidade

`python:` permanece o escape explícito:

```
python: print("interoperabilidade direta")
```

Imports também permitem usar módulos Python quando a sintaxe Coral os referencia. Para a biblioteca oficial, aliases como `matematica` e `arquivos` são resolvidos para `coral.stdlib` quando apropriado; módulos locais têm prioridade. Quando há colisão, o namespace explícito `coral.json`, por exemplo, remove a ambiguidade.

Extras científicos da 1.3

O `pyproject.toml` registra extras opcionais `numerico`, `sistema`, `hardware`, `jogos` e `completo`. O núcleo permanece instalável sem essas dependências. Isso mantém o compilador leve e permite que um curso introdutório não dependa de NumPy, Pygame ou acesso serial.

33. Pipeline real do compilador

Caminho real de compilação

`coral.api.compile` e `traduzir_codigo` convergem em `coral.tradutor.compile_validado`. A função executa análise semântica e depois geração estrutural. Nesta implementação, a semântica analisa o programa e o backend volta a analisá-lo; não há ainda uma única AST compartilhada entre todas as etapas.

Módulo	Responsabilidade
coral/parser_programa.py	Reconhecer instruções, blocos e construções naturais a partir das linhas
coral/lexico.py	Tokenizar expressões, preservando literais
coral/parser_expressoes.py	Precedência e composição da AST de expressões
coral/ast_programa.py	Nós de programa, blocos e localização de fonte
coral/nos.py	Nós de expressão e sua representação Python
coral/semantica.py	Diagnósticos contextuais, como retorno fora de função e await fora de contexto assíncrono
coral/backend_estrutural.py	Gerar Python por nós estruturais e produzir mapa de linhas
coral/runtime.py	Operações auxiliares usadas pelo programa gerado
coral/diagnosticos.py	Normalizar diagnósticos para CLI e ferramentas
coral/simbolos.py	Símbolos e escopos para ferramentas de linguagem

O parser de programa usa reconhecedores de linha, incluindo expressões regulares, para construir nós próprios. Isso não equivale a um fallback de tradução textual: o caminho oficial precisa reconhecer a construção estruturalmente. O lexer de expressões não é uma etapa única aplicada previamente ao programa inteiro.

Expressões e interoperabilidade

Chamadas naturais são declaradas e analisadas deterministicamente. `python:` é escape explícito. `traduzir_expressao` mantém espaços internos de literais na 1.2.1; a normalização textual anterior alterava dados do usuário. O arquivo `tradutor.py` continua sendo uma fachada pública e contém auxiliares; seu nome não indica a permanência do tradutor legado 0.x.

Execução e projetos

O CLI compila e executa Python no processo do runtime. `projeto.py` lê `coral.toml`, coleta módulos e produz `.coral_build`. A 1.2.1 rejeita diretórios de saída que contenham fontes, configuração ou raiz do projeto, e rejeita módulos distintos que gerariam o mesmo arquivo Python, antes de limpar a saída.

O cache em `cache.py` depende da fonte, nome de arquivo, versão Coral e formato de serialização. É uma otimização: entradas inválidas são descartadas e recompiladas. Escritas usam temporários individuais com substituição final do arquivo.

Editor

A extensão `Projeto/vscode-coral/extension.js` inicia `coral --lsp` para linguagem e `coral --dap` para depuração. O cliente LSP está em `coral-lsp-client.js`. `lsp.py` implementa os recursos de linguagem; `dap.py` usa o mapa de linhas para conectar a execução Python à fonte Coral. Test Explorer e Modo Aprender consomem interfaces do runtime.

Bibliotecas

`regras.py` fornece motor, condições, ações, eventos e temporizadores. `mundo.py` representa entidades e relações; a busca BFS é bidirecional por padrão apesar de as relações serem dirigidas. `rpg.py` oferece rolagens e turnos. `jogos/base.py` define o contrato do backend e o ciclo de vida; `pygame_backend.py` realiza as operações gráficas e `BackendNulo` registra comandos sem abrir janela.

Limites arquiteturais

A análise semântica não é uma prova de correção completa nem um sistema obrigatório de tipos estáticos. A gramática EBNF 1.0 permanece histórica; a linha 1.6.1 mantém uma referência sintática versionada apoiada pelo parser e por testes sentinela. LSP e DAP reutilizam as estruturas correntes do compilador. A execução Coral tem os acessos do processo Python e não oferece isolamento de código não confiável.

Referências internas: `Projeto/coral/tradutor.py`, `Projeto/coral/api.py`, `Projeto/coral/parser_programa.py`, `Projeto/coral/backend_estrutural.py` e os testes correspondentes em `Testes/`.

Módulos centrais e tamanho aproximado nesta distribuição

Módulo	Linhas
<code>coral/ast_programa.py</code>	238
<code>coral/backend_estrutural.py</code>	588
<code>coral/cache.py</code>	143
<code>coral/cli.py</code>	821
<code>coral/dap.py</code>	552
<code>coral/diagnosticos.py</code>	84
<code>coral/lexico.py</code>	93
<code>coral/lsp.py</code>	1223
<code>coral/nos.py</code>	207
<code>coral/parser_expressoes.py</code>	736
<code>coral/parser_programa.py</code>	835
<code>coral/projeto.py</code>	361
<code>coral/runtime.py</code>	7
<code>coral/semantica.py</code>	184
<code>coral/simbolos.py</code>	191
<code>coral/tradutor.py</code>	214

A fachada pública `tradutor.py` permanece por compatibilidade de API, mas o caminho estável é estrutural. O nome histórico do arquivo não significa que o compilador 0.x de substituições textuais continue como fallback oficial.

34. Lexer e parser de expressões

Expressões são analisadas por uma camada própria, separada do reconhecimento de blocos do programa. `coral.lexico` tokeniza, preservando strings; `coral.parser_expressoes` aplica precedência e constrói nós de `coral.nos`.

Nós de expressão atuais

NoExpressao, Literal, Nome, Lista, Tupla, Conjunto, Dicionario, ArgumentoChamada, Chamada, Lambda, Atributo, Indice, Fatia, Aguarde, Unario, Binario, Comparacao, CompreensaoLista, CompreensaoConjunto, CompreensaoDicionario.

Precedência conceitual

Da menor para a maior ligação:

1. ou
2. e
3. comparações (for igual a, for maior que, etc.)
4. adição/subtração
5. multiplicação/divisão
6. potência
7. operadores unários
8. chamada, atributo, índice e fatia

O parser aceita formas naturais e algumas formas simbólicas para interoperabilidade. A identidade editorial favorece palavras em português quando a construção oficial existe.

AST, não execução

O parser produz objetos como `Literal`, `Nome`, `Binario`, `Comparacao`, `Chamada`, `Atributo` e `Indice`. Isso permite LSP, análise, formatação e backend raciocinarem sobre estrutura em vez de depender de `eval` ou de substituições de texto.

35. Parser de programa, blocos e AST estrutural

`coral.parser_programa` percorre as linhas do arquivo, reconhece construções por regras determinísticas e produz nós de `coral.ast_programa`.

Nós principais

InstrucaoSimples, Bloco, RamoCondicional, Condicional, RamoTente, Tente, Caso, Combinar, FormaChamadaNatural, RegraDeclarativa, EventoDeclarativo, TemporizadorDeclarativo, Programa.

O parser usa reconhecedores de linha, inclusive expressões regulares. Isso não é equivalente a um tradutor textual: uma construção só entra no caminho oficial depois de ser classificada e representada como nó estruturado.

Delimitador de bloco

Coral usa `fim` como delimitador explícito. A indentação é parte do estilo oficial e das ferramentas, mas `fim` é o sinal estrutural principal.

Chamadas naturais

Uma função pode declarar uma frase de chamada:

```
crie a função somar com a e b chamada como "some {a} com {b}"
  retorne a mais b
fim
defina total como some 4 com 7
```

O template é validado no parser: placeholders precisam corresponder a parâmetros e templates ambíguos/duplicados são rejeitados. Não há interpretação livre por IA.

Regras declarativas

RegraDeclarativa, EventoDeclarativo e TemporizadorDeclarativo são nós próprios. A forma longa de regra exige a sequência regra, quando, então, corpo e fim, com erros P130/P131 quando etapas obrigatórias estão ausentes.

36. Análise semântica, símbolos e diagnósticos

Depois do parse, `coral.semantica` verifica regras contextuais que não são apenas sintaxe.

Exemplos de erros:

Código	Exemplo de categoria
S101	retorno/produção fora de função
S102	pare fora de laço
S103	continue fora de laço
S104	aguarde fora de contexto assíncrono
S105	laço assíncrono fora de função assíncrona
S106	contexto assíncrono fora de função assíncrona
S107	método/propriedade fora de classe
S110	teste em escopo inválido

Código	Exemplo de categoria
S120-S123	restrições contextuais de regras/eventos/temporizadores

O parser usa códigos Pxxx para problemas estruturais, e o linter usa Cxxx para estilo/geração, incluindo:

- C101: tabulação em vez de espaços.
- C102: linha acima de 150 caracteres.
- C201: uso de python: reduz portabilidade.
- C202: captura genérica de exceção.
- C203: importação com *.

A tabela de símbolos em `coral.simbolos` registra escopos e localizações e alimenta recursos de IDE como definição, referências, rename, completion e hierarquias.

Dívida arquitetural conhecida

Na arquitetura atual, a análise semântica parseia o programa e o backend estrutural volta a parsear antes de gerar Python. Ainda não há uma única instância de AST compartilhada por todas as etapas. O comportamento é testado, mas essa duplicação é uma oportunidade futura de simplificação interna.

37. Backend estrutural, runtime e mapa de linhas

`coral.backend_estrutural` gera Python a partir dos nós do programa. Durante a geração, mantém um mapa entre linhas Python e linhas Coral.

Esse mapa é essencial para:

- erros de execução que apontam para o arquivo Coral;
- breakpoints e stack frames do DAP;
- Modo Aprender;
- diagnósticos e ferramentas de inspeção.

Inspeção:

```
coral --mostrar-python programa.coral
coral --mapa-linhas programa.coral
coral --compilar programa.py programa.coral
```

O módulo `coral.runtime` contém operações auxiliares usadas pelo Python gerado. `coral.regras`, `mundo`, `rpg` e `jogos` são runtimes públicos especializados, não macros textuais.

Cache

`coral.cache` fornece cache persistente endereçado por conteúdo e versão, com escrita por arquivo temporário e substituição final. O parser de expressões também usa cache em memória. Cache é otimização: dados inválidos devem ser descartados e recompilados.

```
coral --cache-info
coral --limpar-cache
coral --sem-cache programa.coral
```

38. Projetos, resolução de módulos e construção

`coral.projeto` lê `coral.toml`, descobre módulos locais, monta o grafo de importações, detecta ciclos e constrói saída em `.coral_build`.

Módulos locais têm prioridade sobre aliases curtos da `stdlib`. A construção rejeita situações perigosas como um diretório de saída contendo a própria fonte/configuração e módulos diferentes que colidiriam no mesmo arquivo Python gerado.

Exemplo de `coral.toml`

```
[projeto]
nome = "meu-projeto"
versao = "0.1.0"

[coral]
versao = ">=1.3"

[entrada]
arquivo = "principal.coral"

[modulos]
caminhos = ["."]

[testes]
caminhos = ["testes"]
```

Comandos relacionados:

```
coral projeto-info
coral dependencias
coral construir
coral executar
coral testar
coral documentar
```

39. Language Server Protocol: arquitetura do LSP Coral

O LSP separa inteligência da linguagem da interface do editor. Coral inicia o servidor com:

```
coral --lsp
```

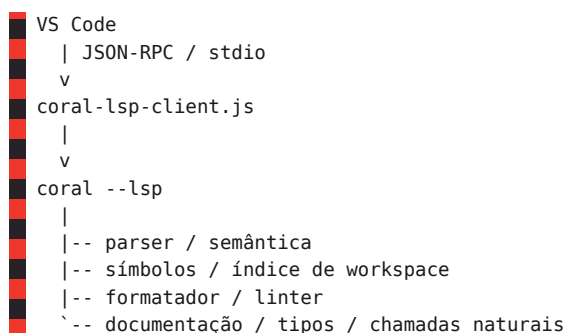
O processo conversa por **JSON-RPC sobre stdio**. O cliente do VS Code é implementado no próprio projeto, sem dependência runtime de `vscode-languageclient`.

Capacidades declaradas pelo servidor 1.5.5

- sincronização de documento
- hover
- go to definition
- references
- document symbols

- workspace symbols
- rename com prepare
- completion
- signature help
- formatação de documento
- range formatting
- on type formatting
- document highlights
- selection ranges
- call hierarchy
- type hierarchy
- code actions
- semantic tokens
- folding
- inlay hints
- document links
- CodeLens

Ciclo simplificado



O uso de LSP reduz duplicação: a mesma análise pode, em princípio, ser consumida por outros editores que implementem o protocolo.

Edição tolerante e cache na 1.3

O LSP diferencia o diagnóstico tolerante usado enquanto a pessoa digita do diagnóstico estrito aplicado ao salvar. Cabeçalhos e construções parcialmente reconhecidas podem ser tratados como edição incompleta, reduzindo falsos positivos durante a escrita.

O índice de símbolos mantém cache limitado por conteúdo e tamanho. Documentos acima do limite configurado são processados sem permanecer no cache. Essa estratégia não equivale a um parser incremental completo; é uma otimização controlada com recuperação parcial.

40. Debug Adapter Protocol: arquitetura do debugger

O debugger é iniciado por:

```
coral --dap
```

O VS Code conversa com `coral.dap` pelo **Debug Adapter Protocol**. O adaptador prepara o arquivo, compila Coral, usa o mapa de linhas e controla a execução Python.

Capacidades atuais

- configurationDone
- breakpoints condicionais
- hit conditions
- log points
- breakpoint locations
- exceções não tratadas/todas
- exceptionInfo
- setVariable
- stackTrace
- scopes
- variables
- evaluate
- continue
- step over
- step in
- step out
- pause
- terminate/disconnect

Fluxo

```

VS Code Run and Debug
  | DAP (JSON)
  v
coral-debug.js
  | stdio
  v
coral --dap
  |
  |-- compilação Coral -> Python
  |-- mapa de linhas
  |-- breakpoints
  |-- inspeção de variáveis
  |-- controle de execução

```

Limites honestos

O adaptador anuncia `supportsStepBack: false` e `supportsRestartRequest: true`. O comando de restart reinicia a sessão preservando a configuração de lançamento aplicável. Breakpoints podem ser associados aos módulos Coral conhecidos pelo projeto e são traduzidos pelo mapa de linhas correspondente; fontes que não pertencem ao projeto continuam sendo rejeitadas ou marcadas como não verificadas em vez de receber suporte fictício. Retroceder a execução continua fora do contrato da 1.6.1.

41. Arquitetura da extensão VS Code e empacotamento VSIX

A extensão está em `Projeto/vscode-coral/` e é empacotada como VSIX. O manifesto atual declara:

Campo	Valor
Nome	Coral Language
ID	<code>coral-lang.coral-language</code>
Versão	0.64.0
VS Code	<code>^1.85.0</code>
Runtime dependencies	nenhuma
Dev dependency	<code>@vscode/vsce ^3.0.0</code>

Arquivos de execução principais

- `extension.js`: ativação, comandos, tarefas, Project Explorer, Test Explorer e integração geral.
- `coral-runtime.js`: descoberta e invocação do runtime Coral.
- `coral-lsp-client.js`: cliente LSP JSON-RPC autocontido.
- `coral-debug.js`: configuração e ponte de depuração.
- `coral-teaching.js`: estudo automático estático.
- `coral-log.js`: canal **Coral: Log** e exportação de logs.

Há ainda gramática TextMate, snippets, configuração da linguagem, ícones e recursos de UI.

VSIX

A distribuição inclui `coral-language-0.67.0.vsix`. O VS Code permite instalar VSIX pela Command Palette/Extensions e por `code --install-extension caminho.vsix`. Instalações via VSIX têm atualização automática desativada por padrão segundo a documentação do editor; uma atualização da Coral deve portanto considerar runtime e extensão.

42. Arquitetura da instalação persistente, Coral Manager e Setup

A instalação possui uma única autoridade de mutação: o contrato `coral.manager/1`. O **Coral Setup** implementa `coral.setup/1` como interface gráfica para esse mesmo contrato; launchers e VS Code delegam às mesmas operações, sem manter uma segunda tabela de pacotes ou chamar `pip` diretamente.

Componentes e perfis

O núcleo exige apenas Python 3.10 ou superior. NumPy, Matplotlib, Plotly, psutil, pyserial e Pygame CE permanecem opcionais e são administrados pelo sistema oficial de **componentes e perfis**. Perfis como Estudante, Científico, Hardware, Jogos e Completo agrupam intenções comuns; o modo Personalizado continua disponível para escolher capacidades individualmente.

Uma operação mutável produz primeiro um plano. Instalação, atualização, reparo ou remoção só alteram o ambiente quando o usuário solicita explicitamente a aplicação. Diagnóstico e planejamento nunca instalam dependências como efeito colateral.

Ciclo mínimo comprovável

A prova automatizada de instalação usa um destino isolado e o perfil Estudante. Ela percorre planejamento, instalação, diagnóstico, launcher, reparo, atualização e remoção sem alterar PATH global. Isso prova o contrato operacional sem fingir validar propriedades visuais de uma janela real.

Setup 1.5

O visual consolidado deriva do redesign da 1.4.11: identidade da cobra Coral, hierarquia mais limpa, indicador das cinco etapas, navegação por teclado, tamanho mínimo e suporte a redução de movimento. O Setup não possui um motor de instalação próprio; toda mutação continua passando pelo Manager.

`coral setup --diagnostico` informa disponibilidade de Tkinter e contexto gráfico. Em Linux sem DISPLAY/Wayland e em qualquer ambiente onde a janela real não possa ser exercitada, a evidência visual permanece `nao_executado`, não “aprovada”.

Launchers correntes

Windows e Linux apontam para `Instalacao/Runtime/coral-1.7.0.pyz`. Os assistentes de instalação e diagnóstico são reconstruídos a partir da mesma árvore da release.

43. Qualidade, conformidade, contratos e release 1.6.13

A Coral não considera uma release pronta apenas porque o runtime inicia. A linha 1.6 usa checkpoints observacionais, validação proporcional ao impacto, sustentabilidade, coesão, higiene, documentação, exemplos, testes reais assistidos e equivalência dos pacotes antes do congelamento.

Gates permanentes

```
coral --gate-1-3
coral --gate-1-4
coral --gate-1-4-9
coral --gate-1-4-10
coral --gate-1-4-11
coral --gate-1-5
```

O gate `coral.gate/1.5` possui vinte critérios de promoção. Ele verifica versão corrente, bases de patch, gates anteriores, limpeza de dist, inventário de fixtures, dependências opcionais, pontes, ferramentas, ciclo mínimo, identidade do Setup, documentação, Livro Oficial, roadmap encerrado, pipeline, ausência da edição 1.4.1 como autoridade corrente, limpeza mecânica, relatório de promoção e presença da release.

Arquitetura de testes

A suíte **rápida** cobre contratos cotidianos. A camada **slow** concentra PYZ, subprocessos e contratos externos automatizáveis. A suíte **histórica** permanece obrigatória para release, mas cada arquivo é executado em processo próprio pelo runner isolado para impedir que recursos vazados por um marco antigo contaminem o seguinte.

Evolução estrutural da linha 1.5.1

A linha 1.5.1 substitui o padrão de criar um arquivo de gate para cada marco por **contratos de regressão armazenados como dados**. Casos permanentes, chamados **specification tests**, entram em um registro parametrizado; casos que apenas congelam o comportamento observado, chamados **characterization tests**, permanecem separados e possuem expiração explícita. Antes de qualquer arquivamento, a equivalência é provada por snapshot comportamental e pode ser complementada por cobertura contextual. O material substituído é movido para o histórico, nunca apagado.

Essa mudança também torna a classificação de custo parte do contrato da suíte: testes com subprocessos, PYZ, instalação real ou runners históricos pertencem à camada **slow**, mesmo quando o custo é indireto por meio de um helper. A camada rápida fica reservada ao feedback cotidiano.

Patches e equivalência

A distribuição exige três ZIPs: `Coral_1.5.5_Completo.zip`, `Coral_1.5.5_Patch_desde_1.5.4.zip` e `Coral_1.5.5_Patch_desde_estavel_1.5.4.zip`. Cada patch é aplicado sobre uma base 1.5.4 limpa e a árvore resultante é comparada arquivo por arquivo e por SHA256 com o pacote completo. Depois da aprovação final, a 1.5.5 pode assumir o papel de nova base estável.

Evidência externa

Windows nativo, GUI real do VS Code e hardware físico nunca recebem aprovação automática a partir de mocks, Xvfb ou backends simulados. `coral.campo/2` diferencia aprovado, falhou e `nao_executado` para que limitações do ambiente sejam registradas sem perder logs úteis.

Fechamento público da 1.7.0

A 1.7.0 é o marco público de fechamento do ciclo 1.6.x. Ela não adiciona novas famílias de API: promove a identidade do runtime e da extensão, refece a distribuição Linux, exige a validação nativa de Windows transferida pela 1.6.18 e executa os gates finais de sustentabilidade, coesão, higiene e empacotamento.

Enquanto os gates Windows e a qualificação pesada do notebook não estiverem concluídos, esta edição é candidata. A identidade técnica 1.7.0 não substitui a necessidade de evidência real de plataforma nem autoriza declarar a release congelada antes do fechamento CP11.

44. Como estender Coral sem quebrar sua identidade

1. Português corrente, não mera tradução lexical

Coral deve ser escrito e lido como um subconjunto controlado do português. A forma preferida é:

```

se idade for maior ou igual a 18 então
    mostre "Acesso permitido"
senão
    mostre "Acesso negado"
fim

```

A linguagem evita fazer o usuário pensar primeiro em Python para depois traduzir. Operadores simbólicos e chamadas convencionais continuam disponíveis quando necessários à interoperabilidade, mas não definem a identidade do Coral.

2. Naturalidade controlada

Português totalmente livre seria ambíguo. Coral aceita formas deliberadamente limitadas, documentadas e determinísticas. A leitura deve soar natural sem depender de IA ou de adivinhação da intenção do programador.

3. Legibilidade antes de concisão

Entre duas sintaxes equivalentes, a API oficial deve preferir a que comunica intenção com menos conhecimento prévio de convenções de outras linguagens.

```

■ adicione 5 a pontos

```

é preferível, na documentação introdutória, a uma mutação simbólica equivalente.

4. APIs oficiais devem ter uma superfície Coral

Bibliotecas podem usar classes e métodos internamente, mas templates, tutoriais e exemplos devem oferecer uma camada que permaneça coerente com a leitura da linguagem. Chamadas como `objeto.metodo(...)` são válidas e importantes para interoperabilidade, porém não devem ser a única forma de ensinar os recursos centrais da biblioteca oficial.

5. Previsibilidade acima de adivinhação

Chamadas naturais são declarativas (chamada como) e regras/eventos possuem gramática formal. A mesma entrada deve sempre ter a mesma interpretação.

6. Python como plataforma, não como aparência

Coral transpila para Python para aproveitar runtime, bibliotecas, portabilidade e ecossistema. Essa decisão de implementação não deve obrigar o código Coral a parecer código Python.

7. Interoperabilidade progressiva

O prefixo `python:` permanece como escape explícito. Ele é uma ferramenta de integração, não a forma recomendada para funcionalidades que já possuem equivalente Coral.

8. AST e semântica próprias

A sintaxe oficial deve ser reconhecida pelo parser e representada na AST Coral. Recursos novos não devem ser introduzidos como simples substituições textuais frágeis.

9. Ensino sem infantilização

Coral busca reduzir barreiras linguísticas e conceituais sem esconder que programação possui estado, fluxo, funções, objetos, erros e abstrações. O objetivo é tornar esses conceitos observáveis e legíveis, não fingir que eles não existem.

10. Critério para novos recursos

Um recurso candidato deve responder positivamente a estas perguntas:

1. É determinístico?
2. É legível em português sem conhecer sua tradução Python?
3. É consistente com construções já existentes?
4. Pode ser diagnosticado pelo compilador em termos Coral?
5. Pode ser documentado e testado sem depender do backend Python?

Se uma nova API importante só puder ser demonstrada com chamadas Pythonicas, ela ainda não atingiu a experiência ideal da biblioteca oficial Coral.

Checklist de implementação

Ao adicionar sintaxe nova:

1. defina a forma portuguesa e casos ambigualmente parecidos;
2. adicione testes de parser antes do backend;
3. represente a construção em AST ou em um tipo estrutural existente adequado;
4. acrescente diagnósticos semânticos quando houver restrições de contexto;
5. gere Python no backend estrutural;
6. preserve mapa de fonte;
7. acrescente formatação, highlighting/snippet/LSP quando relevante;
8. atualize conformidade e naturalidade;
9. documente a forma natural como principal e a API OO/Python como avançada;
10. execute gates e testes de regressão.

Uma nova API que só pode ser ensinada como `objeto.metodo(...)` ainda não atingiu a experiência Coral ideal quando poderia existir uma frase determinística melhor.

11. Separar estabilidade de experimentação

Na 1.5.5, essa regra também vale para comunicação serial, Pygame, áudio, janela, entrada real e recursos dependentes do sistema: contratos automatizados cobrem o que o ambiente permite, enquanto `Testes/Executar_Testes_Reais.coral` registra a evidência complementar em máquina física. Uma funcionalidade não deve ganhar o rótulo de estável apenas porque compila ou passa por mocks.

Autoridade documental da base estável

A edição corrente é formada por `Livro/Coral_1.6.16_Livro_Fonte.md` e `Livro/Coral_1.7.0_Livro_Oficial.pdf`. O histórico de evolução é preservado principalmente pelo changelog e pela documentação de release; cópias editoriais antigas não precisam permanecer indefinidamente na árvore principal. O índice corrente, os manuais operacionais e o README da distribuição devem apontar para esta edição.

PARTE VI: REFERÊNCIA COMPLETA

Sintaxe, APIs, CLI, configuração e consulta rápida para uso diário.

45. Referência sintática consolidada

Consolidada na linha 1.6 e revalidada para a edição 1.6.16. As construções abaixo incluem trechos que dependem de variáveis definidas pelo programa. Os capítulos introdutórios e Livro/Exemplos/ contêm exemplos completos e executáveis.

1. Escopo

A Coral 1.2 preserva a compatibilidade da família 1.x e acrescenta uma superfície voltada a projetos de hobby, jogos 2D, programação reativa e aprendizado assistido.

2. Princípio lexical e de leitura

Código Coral é um subconjunto controlado do português. Palavras com acento são aceitas na forma canônica e várias formas sem acento continuam reconhecidas por compatibilidade. A linguagem não usa processamento de linguagem natural livre: cada construção possui gramática determinística.

3. Blocos

Blocos são delimitados por `fim`. A indentação é formatada oficialmente, mas `fim` permanece o delimitador sintático explícito.

```
se vida for menor ou igual a 0 então
  mostre "Fim"
senão
  subtraia 1 de vida
fim
```

4. Estado e operações básicas

```
defina pontos como 10
adicione 5 a pontos
subtraia 2 de pontos
mostre pontos
```

5. Funções e chamadas naturais

```
crie a função dobro com numero chamada como "dobro {numero}"
  retorne numero vezes 2
fim
defina resultado como dobro 10
```

A forma natural é declarativa e participa da AST de expressões; não há interpretação por IA.

6. Regras, eventos e tempo

```
regra "ferido" quando vida for menor ou igual a 5 então
    defina estado como "ferido"
fim

quando ocorrer "cura" recebendo valor
    adicione valor a vida
fim

a cada 1 segundo faça
    subtraia 1 de energia
fim
```

Regras usam MotorRegras. Modos disponíveis: ao ativar, sempre e uma vez.

7. Superfície natural de mundos

```
crie um mundo chamado campanha com nome "Minha Campanha"
crie uma entidade chamada vila no mundo campanha
crie uma entidade chamada bosque no mundo campanha
relacione vila com bosque como "estrada" no mundo campanha
encontre um caminho de vila até bosque no mundo campanha como rota
```

coral.mundo representa entidades e relações como grafo dirigido e oferece busca de caminho.

8. Superfície natural de RPG

```
crie um personagem chamado heroína no mundo campanha com 12 de vida
role um dado de 20 lados como ataque
cause 3 de dano a heroína
cure 2 de vida de heroína
crie um combate chamado batalha com heroína e monstro
inicie o combate batalha
```

Rolagens aceitam sementes para reprodutibilidade.

9. Superfície natural de jogos

```
de coral.jogos importe AZUL

crie um jogo chamado jogo com largura 800 e altura 450
crie uma cena chamada principal
crie um sprite chamado jogador em 100 e 180 com largura 40 e altura 40
adicione jogador à cena principal
mude jogo para a cena principal

quando o jogo jogo atualizar com dt
    se a tecla "direita" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a direita
    fim
fim

quando o jogo jogo desenhar
    limpe jogo com AZUL
    desenhe o texto "Olá" em 20 e 20 no jogo jogo
fim

inicie jogo
```

A implementação gráfica oficial usa Pygame CE; BackendNu1o permite testes sem janela.

10. Modo Aprender

`coral explicar`, `coral aprender` e `coral rastrear` executam o programa com rastreamento pedagógico de linhas, variáveis, escopos e saída. A saída JSON é consumida pela extensão VS Code.

11. Auditoria de Naturalidade

`coral naturalidade arquivo.coral` não verifica correção semântica. Ele avalia se o código está usando escapes ou superfícies de baixo nível quando já existe forma Coral oficial.

12. Ferramentas

A distribuição inclui formatter, linter, REPL, test runner, documentação automática, LSP, Debug Adapter, templates, cache de compilação, conformidade e gates de release.

13. Compatibilidade

Código válido em Coral 1.0 e 1.1 deve permanecer válido na 1.2, salvo defeito previamente não especificado. As novas construções da 1.2 são aditivas.

14. Fora do escopo

A linha científica/acadêmica, NumPy, PyTorch e Experimentos Coral permanecem reservados para estudo futuro da 1.3 e não fazem parte do contrato da 1.2.

15. Contrato de naturalidade estável

A distribuição 1.2 inclui `coral naturalidade ARQUIVO_OU_PROJETO`. A auditoria é uma ferramenta de estilo: não torna formas interoperáveis inválidas, mas sinaliza quando uma API oficial já possui uma forma Coral mais legível. Em projetos, a pontuação geral é a média por arquivo para não punir artificialmente repositórios maiores. - - json fornece saída destinada a CI, extensões de editor e outras ferramentas.

O contrato editorial da 1.2 exige que templates, tutoriais e vitrines oficiais priorizem a superfície natural sempre que ela exista.

A tabela resume formas reconhecidas. Os nomes em maiúsculas são espaços reservados explicativos, não palavras que você deve copiar literalmente. Cada bloco precisa de corpo e encerramento adequados.

Construção	Forma de referência
Atribuição	defina NOME como EXPRESSAO
Atribuição alternativa	NOME recebe EXPRESSAO
Tipo opcional	defina NOME do tipo TIPO como EXPRESSAO
Saída	mostre EXPRESSAO
Atualização	adicione VALOR a NOME, subtraia VALOR de NOME
Condicional	se CONDICA0 então ... senão ... fim

Construção	Forma de referência
Repetição	<code>enquanto CONDICAO faça ... fim</code>
Intervalo	<code>para NOME de INICIO até FIM faça ... fim</code>
Coleção	<code>para cada NOME em EXPRESSAO faça ... fim</code>
Função	<code>crie a função NOME com PARAMETROS ... fim</code>
Chamada natural	<code>chamada como "PADRAO {parametro}" no cabeçalho da função</code>
Retorno	<code>retorne EXPRESSAO</code>
Gerador	<code>produza EXPRESSAO, produza de EXPRESSAO</code>
Chamada por efeito	<code>execute EXPRESSAO ou chame FUNCAO com ARGUMENTOS</code>
Importação	<code>importe MODULO, de MODULO importe NOME e OUTRO</code>
Classe	<code>crie a classe NOME ... fim</code>
Teste	<code>teste "NOME" ... fim</code>
Verificação	<code>garanta que CONDICAO</code>
Exceção	<code>tente ... se der erro ... finalmente ... fim</code>
Regra	<code>regra "NOME" quando CONDICAO então ... fim</code>
Evento	<code>quando ocorrer "NOME" recebendo PARAMETRO ... fim</code>
Temporizador	<code>a cada INTERVALO segundos faça ... fim</code>
Escape	<code>python: INSTRUCAO_PYTHON</code>

Léxico e blocos

Use arquivos UTF-8 e comentários iniciados com `#`. A política pública de identificadores é única: o primeiro caractere aceita `_`, letras ASCII, `ª`, `º` e letras latinas de Latin-1; os seguintes também aceitam os dígitos ASCII 0 a 9. Outros alfabetos Unicode não pertencem ao contrato lexical atual. Maiúsculas e minúsculas importam nos nomes de variáveis.

Strings admitem aspas simples ou duplas. Dentro delas, espaços são dados. Escapes como `\n` representam caracteres especiais. Uma expressão numérica usa ponto decimal. A indentação recomendada é de quatro espaços; `fim` é o delimitador explícito.

O que não presumir

A linguagem não interpreta português arbitrário. `leia` aparece em material histórico e sugestões, mas não é uma instrução implementada pelo parser atual; o capítulo de entrada ensina a ponte explícita usada nesta entrega. A sintaxe de `modulo importe nome como apelido` também não é ensinada como válida: use uma atribuição explícita a outro nome após importar.

A gramática `coral-1.0.ebnf` é um artefato histórico. A linha 1.6.1 mantém uma referência sintática versionada gerada a partir do esquema corrente; parser e testes sentinela permanecem a autoridade operacional.

16. Superfície científica e de sistema da 1.3

```

crie um vetor chamado dados com [1, 2, 3]
calcule a média de dados como media_dados

crie uma matriz chamada a com [[2, 0], [0, 4]]
crie um vetor chamado b com [8, 20]
resolva o sistema a com b como solucao

experimento "amostra" como estudo repetindo 3 vezes usando a semente 10
    retorne 42
fim
salve o experimento estudo em "resultado.json"
mostre o resumo do experimento estudo

obtenha informações do sistema como sistema
leia a variável de ambiente "PATH" como caminho_path
execute o comando "python" com ["--version"] como execucao

conecte à porta serial "COM3" a 115200 como placa
envie "PING" para placa com quebra de linha
leia uma linha de placa como resposta
feche a conexão placa

```

Essas formas são reconhecidas pelo backend estrutural. A API serial e o backend simulado são testados; a comunicação física continua dependente de evidência em dispositivo real.

46. Referência da biblioteca padrão

A lista abaixo registra as assinaturas implementadas. Parâmetros sem valor padrão são obrigatórios. Chamadas podem usar argumentos nomeados quando a assinatura permite.

coral.arquivos

Caminhos relativos usam a pasta de trabalho. Escrita substitui conteúdo; adicionar_texto acrescenta ao final. A pasta pai deve existir, salvo quando criada explicitamente.

Função	Assinatura
ler_texto	ler_texto(caminho, codificacao='utf-8')
escrever_texto	escrever_texto(caminho, texto, codificacao='utf-8')
adicionar_texto	adicionar_texto(caminho, texto, codificacao='utf-8')
existe	existe(caminho)
listar	listar(caminho='.')
criar_pasta	criar_pasta(caminho)

coral.texto

As funções convertem seu argumento principal para texto. dividir retorna lista; juntar recebe separador e coleção.

Função	Assinatura
maiusculas	maiusculas(valor)

Função	Assinatura
<code>minusculas</code>	<code>minusculas(valor)</code>
<code>apapar</code>	<code>apapar(valor)</code>
<code>dividir</code>	<code>dividir(valor, separador=None)</code>
<code>juntar</code>	<code>juntar(separador, valores)</code>
<code>substituir</code>	<code>substituir(valor, antigo, novo)</code>
<code>comeca_com</code>	<code>comeca_com(valor, prefixo)</code>
<code>termina_com</code>	<code>termina_com(valor, sufixo)</code>

coral.matematica

`coral.matematica` reúne matemática escalar e discreta sem NumPy. `media` rejeita coleção vazia. `fatorial`, `combinacoes` e `arranjos` exigem inteiros nos seus domínios. `aproximadamente_igual` usa tolerâncias explícitas; NaN nunca compara como aproximadamente igual. `interpolar_linear` também permite extrapolação quando a fração fica fora de 0 a 1. `log10` e `log2` exigem valor positivo. `raiz_nesima` oferece raiz real para índice inteiro positivo e só admite radicando negativo com índice ímpar. `sinal` e operações geométricas novas exigem valores finitos. Constantes: `pi` e `e`.

Função	Assinatura
<code>raiz</code>	<code>raiz(valor)</code>
<code>potencia</code>	<code>potencia(base, expoente)</code>
<code>absoluto</code>	<code>absoluto(valor)</code>
<code>arredondar</code>	<code>arredondar(valor, casas=0)</code>
<code>minimo</code>	<code>minimo(valores)</code>
<code>maximo</code>	<code>maximo(valores)</code>
<code>soma</code>	<code>soma(valores)</code>
<code>media</code>	<code>media(valores)</code>
<code>fatorial</code>	<code>fatorial(n)</code>
<code>combinacoes</code>	<code>combinacoes(n, k)</code>
<code>arranjos</code>	<code>arranjos(n, k)</code>
<code>hipotenusa</code>	<code>hipotenusa(a, b)</code>
<code>aproximadamente_igual</code>	<code>aproximadamente_igual(a, b, tolerancia_relativa=1e-9, tolerancia_absoluta=0.0)</code>
<code>interpolar_linear</code>	<code>interpolar_linear(inicio, fim, fracao)</code>
<code>log10</code>	<code>log10(valor)</code>
<code>log2</code>	<code>log2(valor)</code>

Função	Assinatura
arco_seno	arco_seno(valor)
arco_cosseno	arco_cosseno(valor)
arco_tangente	arco_tangente(valor)
raiz_nesima	raiz_nesima(valor, indice)
produto	produto(valores)
sinal	sinal(valor)
volume_bola3	volume_bola3(raio)

coral.datas

Datas suportam ISO 8601, durações e fusos horários explícitos. Uma data/hora sem fuso não é convertida silenciosamente para outro fuso.

Função	Assinatura
agora	agora(fuso=None)
hoje	hoje(fuso=None)
dias	dias(valor)
segundos	segundos(valor)
minutos	minutos(valor)
horas	horas(valor)
semanas	semanas(valor)
formatar	formatar(valor, formato='%d/%m/%Y')
analisar	analisar(texto, formato=None)
de_iso	de_iso(texto)
para_iso	para_iso(valor, vezespec='auto')
fuso_horario	fuso_horario(nome='UTC')
converter_fuso	converter_fuso(valor, fuso)
diferenca	diferenca(final, inicial)

coral.json

para_json/de_json transformam entre dados e texto; ler_json/escrever_json trabalham com arquivos. Objetos arbitrários precisam de conversão explícita.

Função	Assinatura
para_json	para_json(valor, indentar=2)

Função	Assinatura
<code>de_json</code>	<code>de_json(texto)</code>
<code>ler_json</code>	<code>ler_json(caminho)</code>
<code>escrever_json</code>	<code>escrever_json(caminho, valor, indentar=2)</code>

coral.colecoes

`ordenar` retorna nova lista; `unicos` preserva a primeira ocorrência; `contar` devolve frequências; `agrupar_pares` combina duas sequências até a mais curta.

Função	Assinatura
<code>ordenar</code>	<code>ordenar(valores, reverso=False)</code>
<code>unicos</code>	<code>unicos(valores)</code>
<code>contar</code>	<code>contar(valores)</code>
<code>primeiro</code>	<code>primeiro(valores, padrao=None)</code>
<code>agrupar_pares</code>	<code>agrupar_pares(chaves, valores)</code>

coral.aleatorio

As funções convenientes continuam podendo usar o gerador comum, mas a 1.5.8 acrescenta `FonteAleatoria` e `fonte(mente)` para simulações reproduzíveis sem depender do estado aleatório global. Uma mesma fonte explícita produz a mesma sequência quando recebe a mesma semente. Esta API é voltada a simulação, testes e jogos, não a segredos.

Função ou tipo	Assinatura
<code>FonteAleatoria</code>	<code>FonteAleatoria(mente=None)</code>
<code>fonte</code>	<code>fonte(mente=None)</code>
<code>inteiro</code>	<code>inteiro(minimo, maximo, fonte=None)</code>
<code>decimal</code>	<code>decimal(minimo=0.0, maximo=1.0, fonte=None)</code>
<code>escolher</code>	<code>escolher(valores, fonte=None)</code>
<code>amostra</code>	<code>amostra(valores, quantidade, fonte=None)</code>
<code>escolha_ponderada</code>	<code>escolha_ponderada(valores, pesos, fonte=None)</code>
<code>embaralhar</code>	<code>embaralhar(valores, fonte=None)</code>

Superfícies fundamentais acrescentadas na 1.5.8

Módulo	Operações principais
coral.entrada	ler_linha, usar_fonte_entrada, FonteEntradaSequencial
coral.conversoes	inteiro, decimal, texto, booleano
coral.formatacao	montar_texto, formatar_valor
coral.tipos	tipo_de, nome_tipo, e_tipo
coral.persistencia	salvar, carregar, valor_portatil, para_dados, de_dados, texto_canonico, serializar_canonico
coral.caminhos	normalizar_caminho, juntar_caminho
coral.assincrono	aguardar_com_timeout, cancelar_tarefa, grupo_tarefas

A persistência portátil preserva as categorias declaradas como portáteis, incluindo tupla, conjunto, bytes, data, data/hora, duração e caminho, além dos valores básicos. Serializações canônicas mantêm ordem determinística onde a ordem lógica não é significativa.

Assinaturas confirmadas no runtime 1.6.9

Módulo	Assinatura
arquivos	ler_texto(caminho, codificacao='utf-8')
arquivos	escrever_texto(caminho, texto, codificacao='utf-8')
arquivos	adicionar_texto(caminho, texto, codificacao='utf-8')
arquivos	existe(caminho)
arquivos	listar(caminho='.')
arquivos	criar_pasta(caminho)
texto	maiusculas(valor)
texto	minusculas(valor)
texto	aparar(valor)
texto	dividir(valor, separador=None)
texto	juntar(separador, valores)
texto	substituir(valor, antigo, novo)
texto	comeca_com(valor, prefixo)
texto	termina_com(valor, sufixo)
matematica	raiz(valor)
matematica	potencia(base, expoente)
matematica	absoluto(valor)
matematica	arredondar(valor, casas=0)
matematica	minimo(valores)
matematica	maximo(valores)
matematica	soma(valores)
matematica	media(valores)
matematica	fatorial(n)
matematica	combinacoes(n, k)
matematica	arranjos(n, k)
matematica	hipotenusa(a, b)
matematica	aproximadamente_igual(a, b, tolerancia_relativa=1e-9, tolerancia_absoluta=0.0)
matematica	interpolador_linear(inicio, fim, fracao)
matematica	log10(valor)
matematica	log2(valor)

Módulo	Assinatura
matematica	arco_seno(valor)
matematica	arco_cosseno(valor)
matematica	arco_tangente(valor)
matematica	raiz_nesima(valor, indice)
matematica	produto(valores)
matematica	sinal(valor)
matematica	volume_bola3(raio)
datas	agora()
datas	hoje()
datas	dias(valor)
datas	segundos(valor)
datas	formatar(valor, formato='%d/%m/%Y')
json	para_json(valor, indentar=2)
json	de_json(texto)
json	ler_json(caminho)
json	escrever_json(caminho, valor, indentar=2)
colecoes	ordenar(valores, reverso=False)
colecoes	unicos(valores)
colecoes	contar(valores)
colecoes	primeiro(valores, padrao=None)
colecoes	agrupar_pares(chaves, valores)
aleatorio	inteiro(minimo, maximo)
aleatorio	escolher(valores)
aleatorio	embaralhar(valores)
aleatorio	decimal()

APIs acrescentadas na linha 1.3

coral.numerico

Na 1.6.1, `espaco_linear` exige que `quantidade` represente um inteiro exato. Valores fracionários não são truncados silenciosamente; formas compatíveis que representam exatamente um inteiro, como 3.0 e "3", continuam aceitas.

Função ou tipo	Assinatura resumida
vetor	vetor(valores, tipo=None)
matriz	matriz(linhas, tipo=None)
zeros	zeros(forma, tipo=float)
uns	uns(forma, tipo=float)
intervalo	intervalo(inicio, fim=None, passo=1, tipo=None)
espaco_linear	espaco_linear(inicio, fim, quantidade=50)
identidade	identidade(ordem)
remodelar	remodelar(valores, forma)
concatenar	concatenar(valores, eixo=0)
selecionar_coluna	selecionar_coluna(registros, campo, ausentes="erro")
media	media(valores)
mediana	mediana(valores)
variancia	variancia(valores, populacional=True)
desvio_padrao	desvio_padrao(valores, populacional=True)
percentil	percentil(valores, percentual)
covariancia	covariancia(a, b, populacional=True)
correlacao	correlacao(a, b)
frequencias	frequencias(valores)
moda	moda(valores)
quartis	quartis(valores)
amplitude	amplitude(valores)
intervalo_interquartil	intervalo_interquartil(valores)
mediana_desvios_absolutos	mediana_desvios_absolutos(valores)
media_ponderada	media_ponderada(valores, pesos)
media_geometrica	media_geometrica(valores)
media_harmonica	media_harmonica(valores)
histograma	histograma(valores, faixas)
erro_padrao	erro_padrao(valores)
assimetria	assimetria(valores)
curtose	curtose(valores)
matriz_covariancia	matriz_covariancia(dados, populacional=False)

Função ou tipo	Assinatura resumida
<code>matriz_correlacao</code>	<code>matriz_correlacao(dados)</code>
<code>regressao_linear</code>	<code>regressao_linear(x, y)</code>
<code>intervalo_bootstrap</code>	<code>intervalo_bootstrap(valores, estatistica, confianca, repeticoes, semente)</code>
<code>teste_permutacao</code>	<code>teste_permutacao(a, b, estatistica, repeticoes, semente)</code>
<code>norma</code>	<code>norma(valores)</code>
<code>normalizar</code>	<code>normalizar(valores)</code>
<code>produto_vetorial</code>	<code>produto_vetorial(a, b)</code>
<code>posto</code>	<code>posto(matriz)</code>
<code>minimos_quadrados</code>	<code>minimos_quadrados(coeficientes, termos)</code>
<code>vetor3</code>	<code>vetor3(x, y, z)</code>
<code>distancia_vetores</code>	<code>distancia_vetores(a, b)</code>
<code>angulo_entre_vetores</code>	<code>angulo_entre_vetores(a, b)</code>
<code>projecao_vetorial</code>	<code>projecao_vetorial(a, sobre)</code>
<code>produto_misto3</code>	<code>produto_misto3(a, b, c)</code>
<code>volume_paralelepipedo3</code>	<code>volume_paralelepipedo3(a, b, c)</code>
<code>rotacionar3</code>	<code>rotacionar3(vetor, eixo, angulo)</code>
<code>transformar_ponto3</code>	<code>transformar_ponto3(ponto, transformacao)</code>
<code>matriz_rotacao3</code>	<code>matriz_rotacao3(eixo, angulo)</code>
<code>matriz_translacao3</code>	<code>matriz_translacao3(dx, dy, dz)</code>
<code>matriz_escalas3</code>	<code>matriz_escalas3(sx, sy=None, sz=None)</code>
<code>compor_transformacoes3</code>	<code>compor_transformacoes3(*transformacoes)</code>
<code>transformar_vetor3</code>	<code>transformar_vetor3(vetor, transformacao)</code>
<code>distancia_ponto_plano3</code>	<code>distancia_ponto_plano3(ponto, ponto_plano, normal)</code>
<code>projetar_ponto_plano3</code>	<code>projetar_ponto_plano3(ponto, ponto_plano, normal)</code>
<code>intersecao_reta_plano3</code>	<code>intersecao_reta_plano3(origem, direcao, ponto_plano, normal, tolerancia=1e-12)</code>
<code>coordenadas_esfericas3</code>	<code>coordenadas_esfericas3(ponto)</code>
<code>ponto_de_esfericas3</code>	<code>ponto_de_esfericas3(raio, azimuth, elevacao)</code>
<code>area_triangulo3</code>	<code>area_triangulo3(a, b, c)</code>
<code>volume_tetraedro3</code>	<code>volume_tetraedro3(a, b, c, d)</code>
<code>produto_escalar</code>	<code>produto_escalar(a, b)</code>

Função ou tipo	Assinatura resumida
<code>multiplicar_matrizes</code>	<code>multiplicar_matrizes(a, b)</code>
<code>determinante</code>	<code>determinante(m)</code>
<code>inversa</code>	<code>inversa(m)</code>
<code>resolver_sistema</code>	<code>resolver_sistema(coeficientes, termos)</code>
<code>autovalores</code>	<code>autovalores(m)</code>
<code>forma</code>	<code>forma(valor)</code>

Erros públicos principais: `DependenciaNumericaAusente` e `ErroNumerico`.

coral.graficos

Função ou tipo	Assinatura resumida
Grafico	especificação imutável de apresentação
Serie3D	série tridimensional imutável para pontos ou trajetórias
Superficie3D	grade tridimensional retangular explícita
linha	linha(x, y, nome=None)
dispersao	dispersao(x, y, nome=None)
barras	barras(categorias, valores, nome=None)
grafico_histograma	grafico_histograma(valores, faixas, nome=None)
grafico_caixa	grafico_caixa(grupos)
grafico_funcao	grafico_funcao(funcao, inicio, fim, amostras=200)
series_de_registros	series_de_registros(registros, campo_x, campo_y)
mapa_calor	mapa_calor(matriz, rotulos_linhas=None, rotulos_colunas=None, nome=None)
mapa_correlacao	mapa_correlacao(dados, rotulos=None, nome=None)
barras_erro	barras_erro(x, y, erros, nome=None)
faixa_incerteza	faixa_incerteza(x, centro, inferior, superior, nome=None)
grafico_experimento	grafico_experimento(resultado, eixo_x, metrica)
painel	painel(graficos, colunas=2, titulo="")
dispersao3d	dispersao3d(x, y, z, nome=None)
trajetoria3d	trajetoria3d(x, y, z, nome=None)
superficie3d	superficie3d(x, y, z, nome=None)
configurar	configurar(grafico, titulo=None, eixos=None, unidades=None, legenda=None, tema=None, ...)
para_svg	para_svg(grafico, largura=800, altura=500)
salvar	salvar(grafico, caminho)

Erros públicos principais: ErroGrafico. SVG usa somente a biblioteca padrão; PNG e PDF exigem Matplotlib opcional.

coral.laboratorio

Função ou tipo	Assinatura resumida
executar_experimento	executar_experimento(nome, funcao, repeticoes=1, parametros=None, semente=None, metricas=None, continuar_em_erro=True)
grade_parametros	grade_parametros(parametros)
planejar_experimento	planejar_experimento(parametros, repeticoes=1, limite=None)
selecionar_medicoes	selecionar_medicoes(resultado, criterio)
resumir_por_parametro	resumir_por_parametro(resultado, nome)
validar_resultado	validar_resultado(resultado)
carregar_resultado_json	carregar_resultado_json(caminho)
comparar_experimentos	comparar_experimentos(a, b)
protocolo_medicao	protocolo_medicao()
diagnosticar_laboratorio	diagnosticar_laboratorio()
ResultadoExperimento.resumo	resumo()
ResultadoExperimento.salvar	salvar(caminho)
ResultadoExperimento.salvar_json	salvar_json(caminho)
ResultadoExperimento.salvar_csv	salvar_csv(caminho)
ResultadoExperimento.salvar_markdown	salvar_markdown(caminho)

coral.sistema

Função ou tipo	Assinatura resumida
informacoes	informacoes()
localizar_programa	localizar_programa(nome)
programa_disponivel	programa_disponivel(nome)
quantidade_processadores	quantidade_processadores()
variavel_ambiente	variavel_ambiente(nome, padrao=None)
variavel_ambiente_obrigatoria	variavel_ambiente_obrigatoria(nome)
ambiente_selecionado	ambiente_selecionado(nomes)
pasta_temporaria	pasta_temporaria()
terminal_interativo	terminal_interativo(canal)
argumentos_programa	argumentos_programa()
caminho_executavel	caminho_executavel()
executar_comando	executar_comando(programa, argumentos=(), pasta=None, timeout=None, ambiente=None, entrada=None, codificacao=None, erros_decodificacao="estrito", limite_saida_bytes=None)
processo_atual	processo_atual()
ResultadoComando.linhas_saida	linhas_saida()
ResultadoComando.linhas_erro	linhas_erro()
ResultadoComando.codigo_saida	codigo_saida()
ResultadoComando.tempo_decorrido	tempo_decorrido()
ResultadoComando.exigir_sucesso	exigir_sucesso()

ResultadoComando preserva codigo, saida, erro, duracao_segundos e sucesso, e acrescenta saida_truncada e erro_truncado. O limite de captura é aplicado durante a leitura dos fluxos; timeout tem prioridade e processos continuam sendo iniciados sem shell.

coral.hardware

Função ou tipo	Assinatura resumida
cpu	cpu()
memoria	memoria()
discos	discos()
bateria	bateria()
diagnosticar_hardware	diagnosticar_hardware()
portas_seriais	portas_seriais()
conectar_serial	conectar_serial(porta, velocidade=115200, timeout=1.0)
ConexaoSerial.enviar	enviar(texto, terminar_linha=False)
ConexaoSerial.ler	ler(quantidade=1)
ConexaoSerial.ler_linha	ler_linha()
ConexaoSerial.fechar	fechar()

A ausência de pyserial produz DependenciaSerialAusente. A ausência de psutil reduz os dados disponíveis de inventário sem impedir a importação do pacote.

47. Referência das APIs regras, mundo, RPG e jogos

A tabela abaixo documenta a superfície Python/OO avançada consolidada na linha 1.5 e revisada para o runtime 1.6.1; para código introdutório, prefira as frases naturais mostradas nos capítulos anteriores.

Módulo	Símbolo / assinatura
coral.regras	<pre> classe Regra(nome: 'str', condicao: 'Condicao', acao: 'Acao', modo: 'str' = 'ao_ativar', habilitada: 'bool' = True, ativa: 'bool' = False, execucoes: 'int' = 0) -> None </pre>
coral.regras	<pre> Regra.avaliar(self , contexto: 'Any') -> 'bool' </pre>

Módulo	Símbolo / assinatura
coral.regras	<pre> classe `Temporizador (intervalo: 'float', acao: 'Acao', repetir: 'bool' = False, nome: 'str' = 'temporizador', restante: 'float` None' = None, ativo: 'bool' = True) -> None` </pre>
coral.regras	<pre> Temporizador.atual izar(self, contexto: 'Any', dt: 'float') -> 'int' </pre>
coral.regras	<pre> classe Eventos() </pre>
coral.regras	<pre> Eventos.emitir(sel f, nome: 'str', *args, **kwargs) -> 'int' </pre>
coral.regras	<pre> Eventos.quando(sel f, nome: 'str', funcao: 'Callable[... Any]') </pre>
coral.regras	<pre> classe MotorRegras() </pre>
coral.regras	<pre> MotorRegras.a_cada (self, segundos: 'float', acao: 'Acao', *, nome: 'str' = 'intervalo') -> 'Temporizador' </pre>
coral.regras	<pre> MotorRegras.adicio nar(self, regra: 'Regra') -> 'Regra' </pre>
coral.regras	<pre> MotorRegras.atuali zar(self, contexto: 'Any', dt: 'float') -> 'tuple[str, ...]' </pre>
coral.regras	<pre> MotorRegras.avalia r(self, contexto: 'Any') -> 'tuple[str, ...]' </pre>
coral.regras	<pre> MotorRegras.depois _de(self, segundos: 'float', acao: 'Acao', *, nome: 'str' = 'depois') -> 'Temporizador' </pre>

Módulo	Símbolo / assinatura
coral.regras	MotorRegras.regra(self, nome: 'str', condicao: 'Condicao', acao: 'Acao', *, modo: 'str' = 'ao_ativar') -> 'Regra'
coral.regras	função regra(nome: 'str', quando: 'Condicao', entao: 'Acao', *, modo: 'str' = 'ao_ativar') -> 'Regra'
coral.mundo	classe Entidade(nome: 'str', atributos: 'dict[str, Any]' = <factory>, tags: 'set[str]' = <factory>, inventario: 'list[Any]' = <factory>) -> None
coral.mundo	Entidade.adicionar_ _tag(self, tag: 'str') -> "Entidade"
coral.mundo	Entidade.definir(s elf, nome: 'str', valor: 'Any') -> "Entidade"
coral.mundo	Entidade.guardar(s elf, item: 'Any') -> 'Any'
coral.mundo	Entidade.obter(sel f, nome: 'str', padrao: 'Any' = None) -> 'Any'
coral.mundo	Entidade.possui_ta g(self, tag: 'str') -> 'bool'
coral.mundo	Entidade.remover_i tem(self, item: 'Any') -> 'bool'
coral.mundo	Entidade.remover_t ag(self, tag: 'str') -> 'None'

Módulo	Símbolo / assinatura
coral.mundo	<pre> classe Relacao(origem: 'Entidade', destino: 'Entidade', tipo: 'str', dados: 'dict[str, Any]' = <factory>) -> None </pre>
coral.mundo	<pre> classe Mundo(nome: 'str' = 'mundo') </pre>
coral.mundo	<pre> Mundo.adicionar(se lf, entidade: 'Entidade') -> 'Entidade' </pre>
coral.mundo	<pre> Mundo.atualizar(se lf, dt: 'float' = 0.0) -> 'tuple[str, ...]' </pre>
coral.mundo	<pre> `Mundo.buscar(self,      None` nome: 'str') -> 'Entidade \ </pre>
coral.mundo	<pre> `Mundo.caminho(self,   str', destino:      str', *, tipo: 'str \ origem: 'Entidade \   'Entidade \ </pre> <p>None' = None, bidirecional: 'bool' = True) -> 'tuple[Entidade, ...]`</p>
coral.mundo	<pre> Mundo.com_tag(self , tag: 'str') -> 'tuple[Entidade, ...]' </pre>
coral.mundo	<pre> Mundo.criar_entida de(self, nome: 'str', **atributos: 'Any') -> 'Entidade' </pre>
coral.mundo	<pre> Mundo.emitir(self, evento: 'str', *args, **kwargs) -> 'int' </pre>
coral.mundo	<pre> Mundo.exigir(self, nome: 'str') -> 'Entidade' </pre>
coral.mundo	<pre> Mundo.para_dict(se lf) -> 'dict[str, Any]' </pre>
coral.mundo	<pre> Mundo.para_json(se lf, **kwargs) -> 'str' </pre>

Módulo	Símbolo / assinatura
coral.mundo	<pre>`Mundo.relacionados( self, entidade: 'Entidade \ str', tipo: 'str \ None' = None, *, direcao: 'str' = 'saida') -> 'tuple[Entidade, ...]`</pre>
coral.mundo	<pre>`Mundo.relacionar(sel f, origem: 'Entidade \ str', destino: 'Entidade \ str', tipo: 'str', **dados: 'Any') -> 'Relacao`</pre>
coral.mundo	<pre>`Mundo.remove(self, entidade: 'Entidade \ str') -> 'bool`</pre>
coral.mundo	<pre>Mundo.remove_rela cao(self, relacao: 'Relacao') -> 'bool'</pre>
coral.mundo	<pre>função criar_mundo(nome: 'str' = 'mundo') -> 'Mundo'</pre>
coral.rpg	<pre>classe Rolagem(expressao: 'str', dados: 'tuple[int, ...]', modificador: 'int' = 0) -> None</pre>
coral.rpg	<pre>classe `Personagem(nome: 'str', vida: 'int' = 10, *, vida_max: 'int \ None' = None, **atributos: 'Any')`</pre>
coral.rpg	<pre>Personagem.adicion ar_tag(self, tag: 'str') -> "'Entidade'"</pre>
coral.rpg	<pre>Personagem.curar(s elf, quantidade: 'int') -> 'int'</pre>
coral.rpg	<pre>Personagem.definir (self, nome: 'str', valor: 'Any') -> "'Entidade'"</pre>
coral.rpg	<pre>Personagem.guardar (self, item: 'Any') -> 'Any'</pre>
coral.rpg	<pre>Personagem.obter(s elf, nome: 'str', padrao: 'Any' = None) -> 'Any'</pre>

Módulo	Símbolo / assinatura		
coral.rpg	Personagem.possui_tag(self, tag: 'str') -> 'bool'		
coral.rpg	Personagem.receber_dano(self, quantidade: 'int') -> 'int'		
coral.rpg	Personagem.remover_item(self, item: 'Any') -> 'bool'		
coral.rpg	Personagem.remover_tag(self, tag: 'str') -> 'None'		
coral.rpg	`Personagem.teste(se If, atributo: 'str', dificuldade: 'int' = 10, *, semente: 'int \	None' = None) -> 'tuple[Rolagem, bool]`	
coral.rpg	classe CombateTurnos(participantes: 'Iterable[Personagem]' = ())		
coral.rpg	CombateTurnos.adicionar(self, personagem: 'Personagem') -> 'Personagem'		
coral.rpg	`CombateTurnos.iniciar(self, *, iniciativas: 'dict[str, int] \	None' = None, semente: 'int \	None' = None) -> 'tuple[Personagem, ...]`
coral.rpg	`CombateTurnos.proximo(self) -> 'Personagem \	None`	
coral.rpg	função `rolar(expressao: 'str' = '1d20', semente: 'int \	None' = None, *, gerador: 'random.Random \	None' = None) -> 'Rolagem`
coral.rpg	função criar_personagem(mundo: 'Mundo', nome: 'str', vida: 'int' = 10, **atributos: 'Any') -> 'Personagem'		

Módulo	Símbolo / assinatura
coral.jogos	<pre> classe Jogo(titulo: 'str', largura: 'int' = 800, altura: 'int' = 450, fps: 'int' = 60, *, backend: 'BackendJogos', tela_cheia: 'bool' = False) </pre>
coral.jogos	<pre> Jogo.desenhar_circ ulo(self, x: 'float', y: 'float', raio: 'float', cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), *, contorno: 'int' = 0) -> 'None' </pre>
coral.jogos	<pre> Jogo.desenhar_reta ngulo(self, x: 'float', y: 'float', largura: 'float', altura: 'float', cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), *, contorno: 'int' = 0) -> 'Retangulo' </pre>
coral.jogos	<pre> Jogo.desenhar_text o(self, texto: 'str', x: 'float', y: 'float', tamanho: 'int' = 24, cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255)) -> 'None' </pre>
coral.jogos	<pre> `Jogo.executar(self, *, max_quadros: 'int \ None' = None) -> 'None` </pre>
coral.jogos	<pre> Jogo.limpar(self, cor: 'Cor' = Cor(vermelho=0, verde=0, azul=0, alfa=255)) -> 'None' </pre>

Módulo	Símbolo / assinatura
coral.jogos	Jogo.mudar_cena(self, cena) -> 'None'
coral.jogos	Jogo.parar(self) -> 'None'
coral.jogos	Jogo.quando_atualizar(self, funcao: "Callable[['Jogo', float], Any]")
coral.jogos	Jogo.quando_desenhar(self, funcao: "Callable[['Jogo'], Any]")
coral.jogos	Jogo.quando_encerrar(self, funcao: "Callable[['Jogo'], Any]")
coral.jogos	Jogo.quando_iniciar(self, funcao: "Callable[['Jogo'], Any]")
coral.jogos	Jogo.tecla_pressionada(self, tecla: 'str') -> 'bool'
coral.jogos	Jogo.posicao_mouse(self) -> 'tuple[int, int]'
coral.jogos	Jogo.botao_mouse_pressionado(self, botao: 'str' = 'esquerdo') -> 'bool'
coral.jogos	Jogo.mouse_clicado(self, botao: 'str' = 'esquerdo') -> 'bool'
coral.jogos	Jogo.mouse_solto(self, botao: 'str' = 'esquerdo') -> 'bool'
coral.jogos	Jogo.roda_mouse(self) -> 'tuple[int, int]'
coral.jogos	Jogo.em_tela_cheia(self) -> 'bool'

Módulo	Símbolo / assinatura
coral.jogos	Jogo.definir_tela_cheia(self, ativa: 'bool') -> 'None'
coral.jogos	Jogo.alternar_tela_cheia(self) -> 'bool'
coral.jogos	Jogo.tocar_som(self, caminho: 'str', volume: 'float' = 1.0) -> 'None'
coral.jogos	classe Cor(vermelho: 'int', verde: 'int', azul: 'int', alfa: 'int' = 255) -> None
coral.jogos	Cor.rgb(self) -> 'tuple[int, int, int]'
coral.jogos	classe Retangulo(x: 'float', y: 'float', largura: 'float', altura: 'float') -> None
coral.jogos	Retangulo.colide_com(self, outro: "Retangulo") -> 'bool'
coral.jogos	classe BackendNulo()
coral.jogos	BackendNulo.apresentar(self) -> 'None'
coral.jogos	BackendNulo.desenhar_circulo(self, x: 'float', y: 'float', raio: 'float', cor: 'Cor', *, contorno: 'int' = 0) -> 'None'
coral.jogos	BackendNulo.desenhar_retangulo(self, retangulo: 'Retangulo', cor: 'Cor', *, contorno: 'int' = 0) -> 'None'

Módulo	Símbolo / assinatura
coral.jogos	BackendNulo.desenhar_texto(self, texto: 'str', x: 'float', y: 'float', tamanho: 'int', cor: 'Cor') -> 'None'
coral.jogos	BackendNulo.encerrar(self) -> 'None'
coral.jogos	BackendNulo.iniciar(self, titulo: 'str', largura: 'int', altura: 'int') -> 'None'
coral.jogos	BackendNulo.limitar_fps(self, fps: 'int') -> 'float'
coral.jogos	BackendNulo.limpar(self, cor: 'Cor') -> 'None'
coral.jogos	BackendNulo.processar_eventos(self) -> 'bool'
coral.jogos	BackendNulo.tecla_pressionada(self, tecla: 'str') -> 'bool'
coral.jogos	BackendNulo.tocar_som(self, caminho: 'str', *, volume: 'float' = 1.0) -> 'None'
coral.jogos	classe `Assets(raiz: Path' = 'assets')` 'str`
coral.jogos	`Assets.imagem(self, nome: 'str` Path') -> 'Path`
coral.jogos	`Assets.som(self, nome: 'str` Path') -> 'Path`

Módulo	Símbolo / assinatura
coral.jogos	<pre> classe `Sprite(x: Path \ 'float', y: 'float', largura: 'float', altura: 'float', imagem: 'str \ None' = None, cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), velocidade_x: 'float' = 0.0, velocidade_y: 'float' = 0.0, visivel: 'bool' = True, nome: 'str' = 'sprite', dados: 'dict[str, Any]' = <factory>, animacao: 'Any' = None) -> None` </pre>
coral.jogos	<pre> Sprite.atualizar(s elf, dt: 'float') -> 'None' </pre>
coral.jogos	<pre> Sprite.colide_com(self, outro: "'Sprite'") -> 'bool' </pre>
coral.jogos	<pre> Sprite.mover(self, dx: 'float', dy: 'float') -> 'None' </pre>
coral.jogos	<pre> classe Cena(nome: 'str') </pre>
coral.jogos	<pre> Cena.adicionar(sel f, sprite: 'Sprite') -> 'Sprite' </pre>
coral.jogos	<pre> Cena.atualizar(sel f, jogo, dt: 'float') </pre>
coral.jogos	<pre> Cena.desenhar(self , jogo) </pre>
coral.jogos	<pre> Cena.entrar(self, jogo) </pre>
coral.jogos	<pre> Cena.quando_atuali zar(self, f) </pre>
coral.jogos	<pre> Cena.quando_desenh ar(self, f) </pre>
coral.jogos	<pre> Cena.quando_entrar (self, f) </pre>
coral.jogos	<pre> Cena.quando_sair(s elf, f) </pre>

Módulo	Símbolo / assinatura
coral.jogos	<code>Cena.remover(self, sprite: 'Sprite') -> 'None'</code>
coral.jogos	<code>Cena.sair(self, jogo)</code>
coral.jogos	classe `Animacao(quadros:  'tuple[str \ </div> <div> Path, ...]', fps: 'float' =  10.0, repetir: 'bool' =  True, _tempo: 'float' =  0.0, _indice: 'int' = 0,  finalizada: 'bool' =  False) -> None`
coral.jogos	<code>Animacao.atualizar(self, dt: 'float')</code>
coral.jogos	<code>Animacao.reiniciar(self)</code>
coral.jogos	classe `Camera(x:  'float' = 0.0, y: 'float'  = 0.0, largura: 'float'  = 800.0, altura: 'float'  = 450.0, alvo: 'Sprite \ </div> <div> None' = None,  suavidade: 'float' =  1.0) -> None`
coral.jogos	<code>Camera.atualizar(self)</code>
coral.jogos	<code>Camera.seguir(self, sprite: 'Sprite', suavidade: 'float' = 1.0)</code>
coral.jogos	<code>Camera.tela(self, x: 'float', y: 'float') -> 'tuple[float, float]'</code>
coral.jogos	função criar_jogo(titulo: 'str', largura: 'int' = 800, altura: 'int' = 450, fps: 'int' = 60, *, backend=None) -> 'Jogo'
coral.jogos	<code>função pygame_dispo_nivel() -> 'bool'</code>

Constantes de cor em coral.jogos

PRETO, BRANCO, VERMELHO, VERDE, AZUL e AMARELO são objetos Cor prontos para uso.

48. API Python pública para ferramentas externas

Coral mantém uma API pública estável da família 1.x para ferramentas que precisam analisar ou compilar sem invocar a CLI.

`coral.api.info_api()` informa:

```
{
  "api": "1.0",
  "funcoes": [
    "parse_programa",
    "parse_expressao",
    "analisar",
    "compilar",
    "formatar",
    "info_api"
  ],
  "tipos": [
    "ResultadoAnalise",
    "ResultadoCompilacao",
    "DiagnosticoEstruturado",
    "ErroCoral",
    "ErroDeTraducao"
  ]
}
```

Funções

- `parse_programa(codigo)` - obtém AST estrutural de programa.
- `parse_expressao(texto)` - obtém AST de expressão.
- `analisar(codigo, nome_fonte=...)` - devolve diagnósticos estruturados.
- `compilar(codigo, nome_fonte=...)` - devolve resultado de compilação e Python gerado.
- `formatar(codigo)` - aplica o formatador Coral.
- `info_api()` - descreve versão e superfície da API.

Tipos públicos

`ResultadoAnalise`, `ResultadoCompilacao`, `DiagnosticoEstruturado`, `ErroCoral` e `ErroDeTraducao`.

Ferramentas externas devem preferir essa API à importação de módulos internos como `parser_programa` ou `backend_estrutural`, pois estes podem evoluir entre releases sem a mesma promessa de estabilidade.

49. Referência do CLI

Use `coral --help` para a lista exata da versão instalada. Os fluxos principais da 1.6.1 são:

Tarefa	Comando
Executar arquivo	<code>coral arquivo.coral</code>
Verificar	<code>coral verificar arquivo.coral</code>
Lint	<code>coral lint arquivo.coral</code>
Formatar	<code>coral formatar arquivo.coral</code>
Testar projeto	<code>coral testar</code>
Documentar	<code>coral documentar</code>
REPL	<code>coral repl</code>
Versão	<code>coral --versao</code>
Instalação	<code>coral diagnostico</code>
VS Code	<code>coral vscode-check</code>
Jogos	<code>coral jogos-check</code>
NumPy	<code>coral --numerico-check</code>
Sistema	<code>coral --sistema-info</code>
Hardware	<code>coral --hardware-check</code>
Laboratório	<code>coral --laboratorio-check</code>
Modo Aprender	<code>coral explicar arquivo.coral</code>
Ensinar	<code>coral ensinar arquivo.coral</code>
Naturalidade	<code>coral naturalidade .</code>
Contratos 1.3	<code>coral --contratos-1-3</code>
Conformidade 1.3	<code>coral --conformidade-1-3</code>
Gate 1.3	<code>coral --gate-1-3</code>
Auditoria de release	<code>coral --release-check-1-3</code>

Ajuda integral capturada da versão 1.6.1

```
usage: coral [-h] [--versao] [--repl] [--mostrar-python]
            [--compilar ARQUIVO.py] [--verificar] [--lint] [--formatar]
            [--formatar-em-lugar] [--checar-formatacao]
            [--ast-expressao EXPRESSAO] [--tokens-expressao EXPRESSAO]
            [--ast-programa] [--ast-json] [--semantica] [--simbolos]
            [--mapa-linhas] [--diagnosticos-json] [--conformidade]
            [--conformidade-1-1] [--conformidade-1-2] [--conformidade-1-3]
            [--auditar-exemplos] [--robustez] [--robustez-intensa] [--corpus]
            [--sem-cache] [--cache-info] [--limpar-cache] [--ambiente]
            [--ambiente-json] [--self-check] [--lsp] [--dap] [--api-info]
            [--projeto-info] [--projeto-executar] [--projeto-construir]
            [--dependencias] [--testar [CAMINHO]] [--testes-json [CAMINHO]]
            [--filtro-teste NOME] [--documentar [DESTINO]]
            [--contratos-stdlib] [--contratos-1-3] [--contratos-1-3-json]
            [--gate-1-1] [--gate-rc-1-1] [--vscode-check]
            [--vscode-check-json] [--gate-vscode] [--gate-1-2] [--gate-1-3]
            [--release-check-1-3] [--release-check-1-3-json]
            [--validar-vsix ARQUIVO] [--listar-templates]
            [--novo-projeto NOME] [--template-projeto ID]
            [--destino-projeto PASTA] [--jogos-check] [--numerico-check]
            [--sistema-info] [--sistema-info-json] [--hardware-check]
            [--hardware-check-json] [--laboratorio-check] [--explicar]
            [--explicar-json] [--naturalidade] [--naturalidade-json]
            [--diagnostico-instalacao] [--diagnostico-instalacao-json]
            [arquivo]
```

Executa, analisa, formata e traduz programas escritos na linguagem Coral.

positional arguments:

arquivo arquivo .coral

options:

-h, --help	show this help message and exit
--versao	mostra a versão do Coral
--repl	abre o console interativo do Coral
--mostrar-python	mostra o Python gerado antes da execução
--compilar ARQUIVO.py	gera o Python correspondente sem executar
--verificar	apenas verifica tradução e sintaxe
--lint	analisa o arquivo e mostra diagnósticos
--formatar	imprime o código formatado
--formatar-em-lugar	formata o arquivo no próprio caminho
--checar-formatacao	retorna erro se o arquivo/diretório precisar de formatação
--ast-expressao EXPRESSAO	mostra AST e Python de uma expressão Coral
--tokens-expressao EXPRESSAO	mostra os tokens de uma expressão Coral
--ast-programa	mostra a AST estrutural de um arquivo Coral
--ast-json	mostra a AST estrutural em JSON
--semantica	executa somente a análise semântica estrutural
--simbolos	mostra a tabela de símbolos e escopos
--mapa-linhas	mostra o mapa de linhas Python -> Coral
--diagnosticos-json	emite diagnósticos estáticos em JSON
--conformidade	executa a suíte normativa de conformidade Coral 1.0
--conformidade-1-1	executa a suíte normativa candidata do Coral 1.1
--conformidade-1-2	executa a suíte normativa da linha Coral 1.2
--conformidade-1-3	executa a suíte normativa candidata da linha Coral 1.3
--auditar-exemplos	confirma que exemplos usam apenas o backend estrutural
--robustez	executa testes determinísticos de robustez do compilador
--robustez-intensa	executa o gate histórico de robustez, originado na linha 1.0
--corpus	executa o corpus de programas Coral de estabilização
--sem-cache	desativa o cache persistente desta compilação
--cache-info	mostra informações do cache de compilação
--limpar-cache	remove o cache persistente associado ao arquivo/diretório atual
--ambiente	mostra informações de plataforma e runtime
--ambiente-json	mostra o ambiente em JSON

```

--self-check          verifica rapidamente a instalação do Coral
--lsp                 inicia o Coral Language Server experimental via stdio
--dap                 inicia o Coral Debug Adapter via stdio
--api-info            mostra a versão e a superfície da API pública Coral
--projeto-info        mostra informações do coral.toml encontrado
--projeto-executar    constrói e executa o projeto coral.toml
--projeto-construir    compila todos os módulos do projeto
--dependencias        mostra o grafo de módulos Coral locais
--testar [CAMINHO]    executa testes Coral
--testes-json [CAMINHO]
                        executa testes Coral e retorna JSON
--filtro-teste NOME    executa somente o teste com este nome
--documentar [DESTINO]
                        gera documentação do projeto Coral
--contratos-stdlib    executa os contratos normativos da biblioteca padrão
                        Coral
--contratos-1-3        executa os contratos públicos de comportamento da
                        Coral 1.3
--contratos-1-3-json  emite os contratos públicos da Coral 1.3 em JSON
--gate-1-1            executa o gate integrado da linha Coral 1.1
--gate-rc-1-1         executa os gates de release candidate Coral 1.1
--vscode-check        diagnostica a integração local/remota com VS Code
--vscode-check-json   diagnostica a integração VS Code em JSON
--gate-vscode         executa o gate de compatibilidade VS Code da linha
                        Coral 1.2
--gate-1-2            executa o gate integrado da linha Coral 1.2
--gate-1-3            executa o gate integrado da linha Coral 1.3
--release-check-1-3   audita o candidato/release Coral 1.3
--release-check-1-3-json
                        audita a release Coral 1.3 em JSON
--validar-vsix ARQUIVO
                        valida estruturalmente um pacote VSIX Coral
--listar-templates    lista templates oficiais de projeto
--novo-projeto NOME    cria um novo projeto Coral
--template-projeto ID
                        template usado por --novo-projeto
--destino-projeto PASTA
                        pasta de destino para --novo-projeto
--jogos-check         verifica a integração opcional Pygame CE
--numerico-check      verifica a integração opcional NumPy do Coral 1.3
--sistema-info        mostra informações do sistema operacional para Coral
                        1.3
--sistema-info-json   mostra informações do sistema operacional em JSON
--hardware-check      diagnostica recursos e dependências de hardware do
                        Coral 1.3
--hardware-check-json
                        diagnostica hardware em JSON
--laboratorio-check   mostra capacidades do laboratório reprodutível Coral
                        1.3
--explicar            executa o programa no Modo Aprender e explica os
                        passos
--explicar-json       executa o Modo Aprender e retorna o rastreo em JSON
--naturalidade        audita se o arquivo ou projeto preserva o estilo
                        natural e idiomático do Coral
--naturalidade-json   emite a auditoria de naturalidade em JSON
--diagnostico-instalacao
                        verifica instalação persistente e execução fora do
                        projeto
--diagnostico-instalacao-json
                        diagnóstico de instalação em JSON

```

50. Configurações de referência: coral.toml, settings e launch.json

coral.toml

```
[projeto]
nome = "meu-app"
versao = "0.1.0"

[coral]
versao = ">=1.3"

[entrada]
arquivo = "principal.coral"

[modulos]
caminhos = ["."]

[testes]
caminhos = ["testes"]
```

Instalação persistente e VS Code

A instalação gera instalacao.json e vscode-settings.json. Um exemplo de configuração explícita:

```
{
  "coral.pythonPath": "/caminho/para/coral/ambiente/bin/python",
  "coral.pyzPath": "/caminho/para/coral/coral.pyz",
  "coral.executablePath": "",
  "coral.format.enable": true,
  "coral.inlayHints.enable": true,
  "coral.trace.server": "off"
}
```

Quando esses caminhos não são definidos explicitamente, a extensão 0.64.0 pode consultar o manifesto persistente da instalação do usuário. Use **Coral: Mostrar runtime selecionado** para conferir a decisão.

launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "coral",
      "request": "launch",
      "name": "Coral: Depurar arquivo",
      "program": "${file}",
      "cwd": "${workspaceFolder}",
      "args": [],
      "stopOnEntry": false
    }
  ]
}
```

51. Solução de problemas por sintomas

Sintoma	Ação sugerida
coral não é encontrado	Reabra o terminal, execute o instalador e consulte coral diagnostico.
Versão antiga	Execute coral --versao, confira instalacao.json e use Coral: Mostrar runtime selecionado .
F5 pede outro debugger	Abra um .coral, recarregue a janela e confirme Coral Language 0.72.0.
Run funciona e F5 não	Exporte Coral: Log e execute Diagnosticar_Extensao.
Breakpoint não é verificado	Use linha executável do arquivo de entrada; breakpoints internos em módulos locais continuam limitados.
LSP não inicia	Execute coral vscode-check e confira o runtime selecionado.
NumPy ausente	Instale NumPy no Python da instalação Coral e execute coral --numerico-check.
Estatística vazia falha	É contrato preservado na 1.5.5; forneça ao menos um valor. Variância amostral exige dois.
Experimento com zero repetições falha	Use repeticoes >= 1; zero é inválido por contrato.
pyserial ausente	Instale pyserial no ambiente Coral antes de usar portas seriais.
Porta serial falha	Execute coral --hardware-check; a serial física depende de evidência em dispositivo real.
Pygame ausente	Instale pygame-ce no Python do ambiente Coral e execute coral jogos-check.
Tela não abre em CI ou SSH	Use BackendNulo ou execute em ambiente gráfico.
Programa externo não inicia	executar_comando não usa shell; passe programa e argumentos separadamente.
Diagnósticos aparecem enquanto digita	Atualize para 0.72.0; a edição tolerante reduz cascatas, mas alguns limites estruturais continuam.

Antes de pedir ajuda

```

coral --versao
coral --self-check
coral diagnostico
coral vscode-check
coral --gate-1-3
coral --release-check-1-3

```

Se o problema estiver no editor, exporte **Coral: Log**. Para instalação, VSIX ou DAP, execute o assistente externo e compartilhe o ZIP de logs.

52. Exercícios e percurso de prática

Etapa 1: estado e fluxo

Crie um programa que acompanhe uma energia inicial de 10. A cada passagem, reduza a energia em dois, mostre o valor e encerre quando chegar a zero. Antes de executar, anote a sequência esperada. Compare sua estrutura com `Exemplos/Fundamentos/01_fluxo_e_colecoes.coral`.

Etapa 2: dados e funções

Receba três notas, converta os textos para números e calcule uma média. Extraia o cálculo para uma função. Recuse uma lista vazia em vez de dividir por zero. Os arquivos

`Exemplos/Dados/01_entrada_conversoes_e_texto.coral`,

`Exemplos/Fundamentos/02_funcoes_tipos_e_escopo.coral` e

`Exemplos/Dados/05_utilitarios_e_interoperabilidade.coral` oferecem as peças necessárias.

Etapa 3: projeto real

Copie `projeto_notas/` para uma pasta sua. Acrescente uma função que determine a situação do aluno e um teste para cada faixa de nota. Execute o projeto e os testes pelo comando instalado. Confirme que mover a pasta do projeto não muda o comportamento quando os caminhos são relativos à raiz.

Etapa 4: persistência

Grave um resumo em JSON e leia o arquivo novamente. Crie um teste que use um arquivo temporário. Estude a diferença entre serializar dados e serializar uma instância de classe. Compare `Exemplos/Dados/02_arquivos_json_e_caminhos.coral`.

Etapa 5: reatividade

Modele um personagem cuja condição muda quando a vida cai. Use uma regra, um evento de cura e um temporizador. Preveja quantas vezes cada ação deve ocorrer antes de executar. Compare `Exemplos/Regras/01_eventos_observacao_e_tempo.coral`.

Critério de domínio

Você está pronto para avançar quando consegue explicar o estado antes e depois de cada trecho, justificar a escolha de uma estrutura, reproduzir um erro e escrever um teste que o detecte. Decorar palavras reservadas não substitui compreender fluxo, dados e composição.

Os exemplos resolvidos acompanham o código; os exercícios são propostas de extensão, não funcionalidades adicionais do compilador.

Projeto final sugerido

Construa um pequeno projeto que combine pelo menos quatro áreas do livro:

1. um `coral.toml` e dois módulos locais;
2. testes nativos;
3. uma API oficial (`coral.mundo`, `coral.rpg` ou `coral.jogos`);
4. uma Regra Coral ou evento;
5. depuração por F5 e pelo menos um breakpoint;
6. `coral naturalidade` . antes de finalizar;
7. `coral` documentar para produzir documentação do projeto.

Etapa 6: ciência e reprodutibilidade

Crie duas funções de ordenação. Antes de medir, teste ambas contra a mesma saída esperada. Use uma semente conhecida, repita o experimento, salve JSON e CSV e explique por que tempos de outra máquina não podem ser tratados como resultado universal.

Compare seu projeto com `vitrine/laboratorio_algoritmos`.

Etapa 7: sistema e hardware sem riscos desnecessários

Use obtenha informações do sistema e execute o comando `...` com `[...]` para executar um programa simples que apenas imprima texto. Depois, se houver um dispositivo serial disponível, limite seu teste a um protocolo de eco ou mensagem segura. A comunicação serial física continua como validação de campo; o backend simulado é coberto por contratos automatizados.

Estados de animação em desenvolvimento na linha 1.5.3

No DEV9, estado visual e direção passam a ser dimensões independentes. Um conjunto de estados pode reunir animações simples e direcionais sem exigir a mesma quantidade de direções entre elas.

```

crie um conjunto de estados de animação chamado personagem
  estado parado usa parado
  estado andando usa andar
  estado atacando usa ataque
fim

use movimento em 8 direções no jogador
ligue os estados personagem ao movimento de jogador no sprite visual
mova jogador 10 para cima_direita
use o estado andando de personagem no sprite visual

```

A ligação com movimento fornece apenas a direção de olhar para o estado direcional atual. Ela não decide automaticamente se o personagem está parado, andando, correndo ou atacando. Parar o movimento preserva a última direção visual, e uma animação simples independe de direção. Essa separação mantém `coral.mundo` livre de dependências gráficas e evita uma máquina de estados implícita.

As validações detalhadas de estados direcionais permanecem nos testes históricos. A biblioteca de exemplos corrente evita reconstruir um arquivo específico para cada checkpoint antigo.

Transições declarativas de estados em desenvolvimento na linha 1.5.3

No DEV10, transições automáticas deixam de ser uma hipótese futura e passam a existir como uma camada opt-in sobre os estados já definidos. A declaração continua explícita:

```

crie transições de animação chamadas locomocao para personagem
    ao mover use andando
    ao parar use parado
    ao corrida_ativada use correndo
    ao corrida_desativada use andando
fim

use movimento em 8 direções no jogador
ligue as transições locomocao ao movimento de jogador no sprite visual

```

O evento mover acontece somente quando o movimento realmente começa; trocar de direção enquanto já existe movimento não o dispara novamente. Isso permite que um estado como correndo sobreviva a uma curva. parar ocorre apenas quando o movimento termina.

Eventos fora desse ciclo são nomes definidos pelo programa:

```

ative a transição corrida_ativada de locomocao no sprite visual

```

O núcleo não conhece corrida nem qualquer outro comportamento específico. Essa neutralidade permite que uma etapa futura de coral.regras acione os mesmos eventos sem duplicar uma máquina de estados dentro de coral.jogos. Sem a frase `ligue as transições ...`, os estados permanecem totalmente manuais.

As transições continuam documentadas aqui como parte da evolução do recurso, mas o antigo exemplo dedicado ao checkpoint foi retirado da biblioteca corrente; sua cobertura de regressão permanece nos testes históricos.

53. Glossário, limites conhecidos e referências

Glossário

- **AST**: árvore sintática abstrata do programa.
- **Backend**: etapa que transforma a representação estrutural em Python executável.
- **DAP**: Debug Adapter Protocol usado pelo debugger.
- **LSP**: Language Server Protocol usado pelo servidor de linguagem.
- **PYZ**: aplicação Python empacotada como ZIP executável.
- **VSIX**: pacote de extensão do Visual Studio Code.
- **Superfície natural**: forma idiomática e determinística em português para uma operação Coral.
- **Contrato público**: comportamento verificado automaticamente e tratado como parte da superfície estável.
- **BackendNulo**: backend de jogos que permite testar o loop sem abrir janela.
- **Coral Manager**: orquestrador público da instalação e manutenção.
- **Coral Setup**: interface gráfica que consome o contrato do Manager.

Limites conhecidos da 1.6.1

1. Coral depende de Python 3.10 ou superior; não é uma compilação nativa independente.
2. O código executado possui as permissões do processo Python; não há sandbox de segurança geral.
3. A análise semântica é contextual e não constitui prova completa de correção.
4. O parser do editor possui recuperação parcial, mas não é um parser incremental total.
5. Pygame CE, NumPy, psutil e pyserial continuam dependências externas opcionais de áreas específicas.
6. Medições de tempo e memória do laboratório são locais e não devem ser tratadas como resultados universais.
7. Windows nativo e a interface gráfica real do VS Code dependem da suíte de campo para evidência no host do usuário.
8. Arduino e ESP32 físicos dependem de validação em dispositivo real; o backend serial simulado não substitui esse teste.

Referências internas

- Projeto/README.md e Documentacao/Desenvolvimento/Projeto/INDICE.md.
- Documentacao/Desenvolvimento/Projeto/PRINCIPIOS.md, ARQUITETURA_COMPILADOR.md e GRAMATICA.md.
- Documentacao/Desenvolvimento/Projeto/CORAL_SETUP_MANAGER.md, TUTORIAL_VSCODE.md e VALIDACOES_DE_CAMPO_1_4.md.
- Documentacao/Desenvolvimento/Projeto/PRONTIDAO_CORAL_1_4.md e PIPELINE_RELEASE.md.
- Projeto/vscode-coral/package.json, cliente LSP e adaptador DAP.
- Testes/ e Testes/Fixtures/runtimes/ para regressões históricas e Testes/Executar_Testes_Reais.coral para evidência complementar em host real.

Referências externas oficiais

- Python: <https://docs.python.org/>
- Visual Studio Code: <https://code.visualstudio.com/docs>
- Language Server Protocol: <https://microsoft.github.io/language-server-protocol/>
- Debug Adapter Protocol: <https://microsoft.github.io/debug-adapter-protocol/>
- NumPy: <https://numpy.org/doc/>
- Matplotlib: <https://matplotlib.org/stable/>
- Plotly: <https://plotly.com/python/>
- psutil: <https://psutil.readthedocs.io/>
- pySerial: <https://pyserial.readthedocs.io/>
- Pygame CE: <https://pyga.me/docs/>

Encerramento

A Coral 1.7.0 parte de uma arquitetura fechada na 1.6.0, preserva a organização em seis áreas e mantém português corrente controlado, bibliotecas coesas, ferramentas integradas, instalação unificada e testes proporcionais ao impacto. Esta edição sincroniza o Livro com o runtime, preserva a geometria e visualização científica 3D da 1.6.12 e integra registros comuns e experimentos com Gráficos e Web, preservando a apresentação HTML interativa introduzida na 1.6.13. A linha 1.6.x não mantém uma frente pública dedicada a quatro dimensões espaciais. A implementação em Python continua sendo uma escolha de plataforma; a identidade da linguagem é definida por seus contratos, sua sintaxe e sua forma de explicar programas em português.

APÊNDICE A: CONSOLIDAÇÃO DO CICLO 1.4 → 1.5.5

A linha 1.5 evoluiu por consolidações incrementais. A 1.5.5 preserva essa disciplina e combina reorganização física com uma baseline espacial geral, sem transformar Jogos no centro desses conceitos.

Marco	Consolidação principal
1.4.7	arquitetura de testes rápida, slow e histórica
1.4.8	coesão Numérico + Laboratório + Sistema e fluxo científico reproduzível
1.4.9	auditoria de Hardware, coral.pontes/1 e ferramentas conscientes das integrações
1.4.10	ciclo mínimo de instalação, Setup diagnosticável e coral.campo/2
1.4.11	atualização visual do instalador Windows/Linux preservando coral.manager/1
1.5.0	limpeza estrutural, documentação única, Livro Oficial renovado e promoção da nova base estável
1.5.1	consolidação estrutural e disciplina de regressão sem crescimento descontrolado da suíte
1.5.2	ampliação prática de coral.jogos, entrada, formas e uso em tela
1.5.3	sprites, transformações, animações direcionais e integração entre regras e apresentação
1.5.4	transformações configuráveis, automações matemáticas, colisores e ensino/editor consolidados
1.5.5	distribuição em seis áreas e baseline geral 2D/3D de espaço, geometria, movimento e consultas

Procedimento recomendado de atualização

1. Faça backup de seus projetos pessoais.
2. Para uma cópia nova, extraia Coral_1.5.5_Completo.zip.
3. Para atualizar uma árvore 1.5.4 limpa, aplique Coral_1.5.5_Patch_desde_1.5.4.zip.
4. Use Coral_Setup ou coral manager atualizar para uma instalação persistente.
5. Confirme coral --versao, coral --self-check, coral --gate-1-4 e coral --gate-1-5.

Mudanças estruturais da distribuição

- Projeto/dist/ volta a significar somente artefatos correntes.
- runtimes antigos ainda necessários por regressão ficam em Testes/Fixtures/runtimes/ com inventário e SHA256;
- Instalacao/Runtime/ contém o runtime corrente e Instalacao/Extensao_VSCode/ contém o VSIX e seu pacote fonte;
- documentos de versões anteriores permanecem históricos, enquanto esta fonte e Livro/Coral_1.7.0_Livro_Oficial.pdf formam a edição corrente;
- o roadmap 1.5 de coesão passa a registrar um ciclo encerrado, não uma lista aberta de próximas versões.

APÊNDICE B: MATRIZ DE VALIDAÇÃO DA RELEASE

A fonte normativa da 1.7.0 é Projeto/VALIDACAO_INCREMENTAL_1_6_18.json, complementada pelo relatório de congelamento. Evidências herdadas não são repetidas quando suas entradas permanecem idênticas; alterações invalidam apenas os gates diretamente afetados.

Área	Política 1.7.0
Regressão funcional	executar apenas superfícies alteradas e preservar evidências válidas da 1.6.17
Slow/histórica	não repetir sem mudança de entrada; falhas novas nunca são promovidas por inferência
Windows nativo	nao_executado na 1.7.0; obrigatório no fluxo da 1.7.0
Gates históricos	devem permanecer verdes no escopo aplicável
Gate 1.5	deve passar 20/20
Runtime PYZ	deve responder 1.7.0 e passar --self-check
VSIX	deve corresponder à extensão 0.72.0 corrente e empacotar corretamente
Ciclo mínimo	instalar, diagnosticar, executar launcher, reparar, atualizar e remover em destino isolado
Patch anterior	aplicar em base 1.6.17 limpa e convergir por hashes
Patch desde estável	aplicar em base estável declarada e convergir por hashes
Limpeza	não conter caches, temporários nem artefatos históricos em superfícies correntes
Testes reais	evidência complementar por Testes/Executar_Testes_Reais.coral, incluindo GUI, Pygame, áudio e entrada quando aplicável
Hardware físico	evidência externa em dispositivo real

APÊNDICE C: CONTRATOS CONSOLIDADOS

A tabela abaixo resume contratos que aparecem ao longo do livro. Ela não substitui suas referências técnicas, mas oferece um mapa rápido da arquitetura estável.

Contrato	Papel
<code>coral.manager/1</code>	ciclo de instalação, atualização, reparo e remoção
<code>coral.setup/1</code>	interface gráfica sobre o Manager
<code>coral.pontes/1</code>	catálogo único de integrações consolidadas
<code>coral.persistencia/1</code>	envelope portátil de dados persistentes
<code>coral.campo/2</code>	evidência de plataforma e estado aprovado/falhou/não executado
<code>coral.gate/1.5</code>	critérios finais de promoção da base estável

Na 1.5.5, esses contratos continuam sendo compromissos verificáveis. Evoluções futuras podem ampliá los por versionamento explícito, não por alteração silenciosa.

APÊNDICE D: REGISTRO HISTÓRICO DA ARQUITETURA DA LINHA 1.5.3

Este apêndice preserva decisões tomadas durante a linha 1.5.3. Naquele ciclo, a independência de domínios foi tratada como gate de arquitetura. `coral.mundo` deve funcionar sem RPG ou Jogos; `coral.jogos` deve funcionar sem Mundo ou RPG; `coral.rpg` pode reutilizar Mundo, mas não depende de Jogos; `coral.regras` permanece infraestrutura reativa sem importar os domínios consumidores.

No DEV11, o registro e a persistência de combates contextuais foram transferidos para `coral.rpg`. A persistência de Mundo oferece apenas um mecanismo genérico de extensões e não conhece `CombateTurnos`. A extensão `coral.rpg/combates` é registrada pelo próprio RPG. Compatibilidade histórica é fornecida por shims instalados por RPG, não por imports de RPG dentro de Mundo.

O mesmo marco introduz uma primeira ponte estrutural entre Regras e apresentação. `MotorRegras.emitir_evento(...)` publica eventos sem conhecer consumidores, e `TransicoesEstadosAnimacao.ligar_eventos(...)` pode optar por reagir a esses eventos sem importar Regras. Nenhuma nova frase Coral é promovida nesse passo; parser, AST e ferramentas só serão ampliados após estabilização do contrato estrutural.

Adendo da candidata 1.5.3 — Regras, eventos e apresentação

A linha 1.5.3 fecha a ponte entre Regras e Jogos sem tornar um módulo dependente do outro. Depois de declarar estados e transições visuais, o programa pode ligar o barramento padrão de regras ao sprite:

■ `ligue as transições locomocao aos eventos de regras no sprite visual`

Ou escolher um motor explícito:

■ `ligue as transições locomocao aos eventos de regras em motor_extra no sprite visual`

A ligação é opt-in. `coral.regras` continua responsável por decidir quando um evento é emitido; `coral.jogos` responde ao evento apenas se uma transição correspondente tiver sido declarada; `coral.mundo` continua representando o estado do domínio sem conhecer apresentação ou RPG.

A aceitação integrada da candidata usa

`Testes/Fixtures/Aceitacao_Integrada/teste_completo.coral` e um spritesheet distribuído junto do exemplo. O fechamento exige também uma prova gráfica com Pygame CE e driver de vídeo real vinculada por fingerprint à candidata testada.